SOFTWARE ARCHITECTURE SPECIFICATION

Vidyalayaa

Complete Website, Backend, Database &

Multi-Device Attendance Architecture

Multi-tenant school management SaaS with NFC attendance devices: web application, REST API, PostgreSQL schema, concurrent multi-device attendance engine, notifications, security, deployment and test strategy.

Document type	Software Architecture Specification (SAS)
Document version	1.0
Date	11 October 2026
Status	Draft for engineering review
Product category	Multi-tenant School Management SaaS with IoT/NFC Attendance Hardware
Platform contact	vidyalayaainfo@gmail.com
Classification	Confidential

Audience: frontend, backend, database, embedded-systems, UI/UX and DevOps engineers.
Requirement IDs, decision-status tags (Confirmed / Proposed / Validate) and a full traceability matrix are included.

Table of Contents

Right-click the table and choose **Update Field** (or press F9) if page numbers are not shown after opening. Section numbering follows the mandated structure; Section 1 is the cover page.

2. Document Control and Version History 7
- 2.1 Document information 7
- 2.2 Version history 7
- 2.3 Review and approval 7
- 2.4 How to read this document 8

3. Executive Summary 8
- 3.1 Purpose 8
- 3.2 System in one paragraph 9
- 3.3 Technology stack at a glance 9
- 3.4 Most important architecture decisions 9
- 3.5 Scope of this document 10

4. Product Requirements 10
- 4.1 Functional scope 10
- 4.2 Initial scale target 11
- 4.3 Quality attributes 11
- 4.4 Assumptions and constraints 12

5. Technology Stack 12
- 5.1 Frontend 13
- 5.2 Backend 13
- 5.3 Database and authentication 14
- 5.4 Hosting, delivery and tooling 14
- 5.5 Deviations from the proposed stack 15

6. System Architecture 15
- 6.1 Design principles 15
- 6.2 Components and responsibilities 15
- 6.3 Primary data flows 18
- 6.4 Trust boundaries 18
- 6.5 Host separation 19

7. User Roles and Permissions 19
- 7.1 Roles 19
- 7.2 Permission matrix 19
- 7.3 Scope rules enforced beyond the role 21
- 7.4 Super Admin and student records 21
- 7.5 Roles and permissions in the database 21
- 7.6 Multi-factor authentication policy 22

8. Multi-Tenant Architecture 22
- 8.1 Tenancy model and rationale 22
- 8.2 Defence in depth 23
- 8.3 School ownership in the schema 23
- 8.4 Row-Level Security 24
- 8.5 Preventing cross-school access — the request path 24
- 8.6 Platform-level (cross-tenant) operations 25
- 8.7 Multi-campus and enterprise tenants 25
- 8.8 Tenant offboarding and export 25
- 8.9 Automated cross-school tests 25

9. Frontend Architecture 26
- 9.1 Principles 26
- 9.2 Folder structure 26

9.3 Routing and protected routes 27
9.4 Permission checks in the UI 28
9.5 API client and query caching 28
9.6 Loading, empty and error states 29
9.7 Forms and validation 29
9.8 Responsive layouts and accessibility 29
9.9 Authentication expiry handling 29
9.10 Configuration and security headers 29

10. Backend Architecture 30
10.1 Modular monolith 30
10.2 Folder structure 30
10.3 Layering 31
10.4 Request lifecycle (user routes) 31
10.5 Validation 31
10.6 Transactions 32
10.7 Authorisation middleware 32
10.8 Error handling 32
10.9 Structured logging 33
10.10 Rate limiting 33
10.11 API conventions 33

11. Database Architecture 34
11.1 Relational design principles 34
11.2 Entity relationship overview 34
11.3 Keys, types and naming 36
11.4 Transactions and isolation 37
11.5 Unique constraints that carry business rules 37
11.6 Indexing policy 37
11.7 Row-Level Security 37
11.8 Migrations 37
11.9 Connection pooling 38
11.10 Backup, restore and point-in-time recovery 38
11.11 Audit history 38
11.12 Database monitoring 38
11.13 Retention and growth 38

12. Complete Database Schema 39
12.1 Platform and identity 39
12.2 Academic structure 42
12.3 NFC cards, devices and attendance 48
12.4 Notifications, communication and fees 53
12.5 Subscriptions, audit and utilities 55
12.6 Core SQL definitions 56

13. Multiple NFC Device Architecture 59
13.1 Example deployment 59
13.2 Device attributes the platform tracks 60
13.3 Registration, claiming and revocation 61
13.4 Concurrent attendance processing 62
13.5 Duplicate protection: four different concepts 63
13.6 Location roles 63
13.7 Multi-gate and movement scenarios 63
13.8 Device health monitoring 64

14. Attendance Rule Engine 65
14.1 Two separate state dimensions 65
14.2 Example schedule (configurable, per school and per class) 66
14.3 Window evaluation rules 67

14.4 Rule resolution and versioning 67
14.5 Timestamps and trust 67
14.6 Projection and replay 68
14.7 Absent calculation 68
14.8 Holidays, weekly offs and exam days 68
14.9 Manual corrections 68
14.10 Notification triggers 69
15. Device Firmware Architecture 70
15.1 Prototype hardware 70
15.2 Firmware responsibilities 70
15.3 Firmware module structure 70
15.4 Tap handling 71
15.5 Durable event queue 71
15.6 Synchronisation and retry 72
15.7 Time handling 72
15.8 Health monitoring and self-tests 72
15.9 Feedback patterns 72
15.10 Secure configuration and OTA 73
15.11 Prototype-to-production transition 73
16. Device API Specification 74
16.1 Transport and authentication 74
16.2 Endpoint summary 74
16.3 Event payload 74
16.4 POST /api/v1/device/events/batch 75
16.5 POST /api/v1/device/heartbeat 77
16.6 GET /api/v1/device/config 77
16.7 POST /api/v1/device/sync-ack 77
16.8 POST /api/v1/device/ota/status 78
16.9 Batching, retries, idempotency and limits 78
16.10 Device error responses 78
17. Concurrency and Idempotency 79
17.1 Event processing steps 79
17.2 Transaction pseudocode 80
17.3 Race conditions and how each is closed 82
17.4 Worked example: simultaneous taps at DEV001 and DEV002 83
17.5 Isolation level and locking choices 83
17.6 Example SQL constraints used by the above 83
17.7 Example attendance record and outbox entry 83
18. Offline Synchronization 84
18.1 Guarantees 85
18.2 Synchronisation protocol 86
18.3 Out-of-order and cross-device ordering 86
18.4 Time reconstruction 86
18.5 Backlog bursts 86
18.6 Effects on notifications and absent finalisation 86
18.7 Outage matrix 87
19. Notification Architecture 87
19.1 Design 87
19.2 Channels and messages 89
19.3 Worker algorithm 89
19.4 Retry, failure and dead-letter handling 90
19.5 Preferences, multiple parents and duplicate prevention 90
19.6 Audit history 90
19.7 Push token lifecycle 90

20. Subscription Architecture 91
20.1 Packages and entitlements 91
20.2 Enforcement points 91
20.3 Subscription lifecycle and grace policy 92
20.4 Operations 92
21. Security and Privacy 93
21.1 Threat overview 93
21.2 Controls 93
21.3 NFC cards: cloning risk and lost-card handling 94
21.4 Parent-child and teacher-class validation 95
21.5 Secure report downloads 95
21.6 Data retention, deletion and child privacy 95
21.7 Application security hygiene 95
22. Performance and Scalability 97
22.1 Workload model (per school, ≈ 1,000 students) 97
22.2 Database performance measures 98
22.3 Initial performance targets (to be validated) 98
22.4 Load-testing plan 98
22.5 Capacity triggers 99
23. Deployment Architecture 100
23.1 Environments 100
23.2 Host responsibilities 101
23.3 CI/CD pipeline 101
23.4 Configuration and secrets 101
23.5 Release management for firmware 102
24. Backups and Disaster Recovery 102
24.1 Objectives 102
24.2 Backup design 102
24.3 Restore procedure and drills 102
24.4 Disaster scenarios 103
24.5 Incident-response checklists 103
25. Monitoring and Logging 104
25.1 Signals 104
25.2 Alerts (initial thresholds — tune after load tests) 104
25.3 Logging standards 105
25.4 Uptime and dashboards 105
26. Project Directory Structure 105
26.1 Sharing API contracts without sharing secrets 106
27. Development Roadmap 107
27.1 Phase 1 — Project setup, database schema, authentication and school isolation 107
27.2 Phase 2 — School admin, teachers, parents and academic records 107
27.3 Phase 3 — NFC device registration and basic attendance 108
27.4 Phase 4 — Multiple devices per school, concurrency control, idempotency and offline synchronisation 108
27.5 Phase 5 — Parent notifications and background workers 109
27.6 Phase 6 — Subscription billing, monitoring and reports 109
27.7 Phase 7 — Security testing, performance testing, backup recovery and pilot deployment 109
28. Testing Strategy 110
28.1 Test pyramid and tooling 110
28.2 Test cases 110
28.3 Quality gates 114
29. Risks and Mitigation 114
30. Production Readiness Checklist 115

31. Final Engineering Recommendations 116
31.1 Open decisions requiring owner input 117
32. Glossary 117
Appendix A — Requirement Traceability Matrix 119
Reverse view: test case to requirements 125
Appendix B — Editable Diagram Sources 127
Appendix C — Items Requiring Validation 135

2. Document Control and Version History

This document is Section 2 onward of the mandated structure; **Section 1 is the cover page**. Section numbers therefore start at 2 so that they match the required document outline exactly.

2.1 Document information

Field	Value
Title	Vidyalayaa — Complete Website, Backend, Database & Multi-Device Attendance Architecture
Document type	Software Architecture Specification (SAS)
Version	1.0
Date	11 October 2026
Status	Draft for engineering review (not yet approved)
Product category	Multi-tenant School Management SaaS with IoT/NFC attendance hardware
Platform name	Vidyalayaa
Official contact	vidyalayaainfo@gmail.com
Classification	Confidential — internal engineering and partner-review use
Primary audience	Frontend, backend, database, embedded-systems, UI/UX and DevOps engineers; product owner; security reviewer

2.2 Version history

Version	Date	Author	Summary of change
1.0	2026-10-11	Architecture team (drafted with Claude, Anthropic)	First complete specification: platform, database, multi-device attendance, notifications, security, operations, roadmap and tests.

2.3 Review and approval

Role	Name	Decision	Date
Product owner			
Lead backend engineer			
Lead frontend engineer			
Database engineer			
Embedded / firmware engineer			
Security reviewer			
DevOps engineer			

2.4 How to read this document

2.4.1 Requirement identifiers

Normative statements carry an identifier of the form `PREFIX-REQ-NNN`. The words SHALL / SHALL NOT mean mandatory; SHOULD means recommended unless a documented reason exists. Prefixes in use:

Prefix	Area	Prefix	Area
ARCH-REQ	Overall architecture	DEV-REQ	Devices, firmware, device protocol
TEN-REQ	Multi-tenancy and isolation	NOT-REQ	Notifications
FE-REQ	Frontend	SUB-REQ	Subscriptions and entitlements
API-REQ	API behaviour	SEC-REQ	Security and privacy
DB-REQ	Database	PERF-REQ	Performance targets
ATT-REQ	Attendance engine	OPS-REQ	Operations, deployment, DR, monitoring
TEST-REQ	Test cases (Section 28)		

2.4.2 Decision status tags

Every requirement carries one of three tags so that nobody mistakes a suggestion for a fixed fact:

Tag	Meaning
Confirmed	Stated by the product brief as a binding design decision.
Proposed	A recommended default chosen by the architecture team. It may be changed by an engineering decision record without re-opening the whole design.
Validate	Depends on a vendor feature, legal interpretation, hardware measurement or load test that must be verified before it is relied upon. No figure tagged Validate is a guarantee.

No invented figures

This document contains no vendor prices and no measured performance results. Every performance number is a **target to be validated by load tests** (Section 22). Vendor capabilities (backup windows, PITR, connection limits, SMS regulation) must be confirmed against the vendor's current documentation and the school's contract before they are relied upon.

3. Executive Summary

3.1 Purpose

Vidyalayaa is a commercial, multi-tenant school-management SaaS product that the company sells to partner schools on subscription. Beyond the usual administration, academic and parent-communication functions, it integrates NFC attendance devices at school gates and in classroom blocks. When a student taps a card, the platform records arrival and departure, applies configurable attendance rules, and tells the student's parents in near real time.

This specification tells every engineering discipline exactly how to build the system: which technologies to use, how data is isolated between schools, how several attendance devices in one school write concurrently to one authoritative database without duplicates or lost events, and how the platform is deployed, secured, monitored, tested and rolled out.

3.2 System in one paragraph

A React/TypeScript single-page application hosted on Firebase Hosting talks to a Node.js/Express REST API on Render. The API is a modular monolith whose modules own the business rules. All authoritative data lives in Supabase PostgreSQL, protected by server-side authorisation, composite foreign keys and Row-Level Security. Supabase Auth issues identities; Supabase Storage holds report cards and documents. ESP32 + PN532 NFC devices authenticate with per-device credentials and upload tap events in idempotent batches over HTTPS; the

server — never the device — decides what a tap means. Every attendance change writes a notification-outbox row in the same transaction; a separate Render background worker drains the outbox to Firebase Cloud Messaging, email and SMS.

3.3 Technology stack at a glance

Layer	Selection	Host
Frontend	React, TypeScript, Vite, Tailwind CSS, shadcn/ui, React Router, TanStack Query, React Hook Form, Zod, Recharts	Firebase Hosting (static + CDN)
Backend API	Node.js, TypeScript, Express.js, REST, Zod validation, pino structured logging, rate limiting, auth middleware	Render web service
Background worker	Same codebase family; notification outbox processor and scheduled jobs	Render background worker
Database	Supabase PostgreSQL (authoritative), RLS, SQL migrations, pooled connections	Supabase (managed)
Authentication	Supabase Auth (email/password, recovery, sessions, optional MFA) bound to application users and school memberships	Supabase (managed)
File storage	Supabase Storage (private buckets, signed URLs)	Supabase (managed)
Notifications	Firebase Cloud Messaging push, email provider, SMS provider (provider-agnostic adapters)	External providers
Device	ESP32 / ESP32-S3, PN532 NFC, DS3231 RTC, durable flash queue, HTTPS REST	School premises
CI/CD and source	GitHub, GitHub Actions, staging and production environments	GitHub

3.4 Most important architecture decisions

#	Decision	Why it matters	Status
1	PostgreSQL is the only authoritative store; no Firebase Realtime Database for attendance.	One source of truth removes reconciliation bugs between two databases.	**Confirmed**
2	Shared database, shared schema, school_id on every school-owned row, composite foreign keys and FORCED RLS.	Cheapest model to operate; isolation enforced in four layers (identity, RBAC, API tenant context, database).	**Confirmed**
3	The device identifies itself, never its school or student. The server derives the school from the authenticated device registration.	A stolen or tampered device cannot read or write another school's data.	**Confirmed**
4	Idempotency key = (device_id, device_event_id), claimed with INSERT … ON CONFLICT DO NOTHING inside the processing transaction.	Retries and replays are safe; concurrent duplicates are impossible, not merely unlikely.	**Confirmed**
5	daily_attendance is guarded by UNIQUE (school_id, student_id, attendance_date) and updated under a row lock (SELECT … FOR UPDATE).	Two gates tapped within milliseconds still yield one daily record and one notification.	**Confirmed**
6	Attendance status and movement status are separate columns; the daily row is a deterministic projection of the ordered raw events.	Departure never erases arrival; out-of-order offline uploads are corrected by replay.	Confirmed / Proposed
7	Rules use the event's trusted original timestamp (effective_ts); suspicious timestamps are quarantined.	Offline uploads and wrong RTC clocks cannot silently corrupt attendance.	**Confirmed**
8	Notification outbox written in the attendance transaction; separate worker with SKIP LOCKED, retries and dead-letter handling.	Attendance is never lost because a push provider is down; no duplicate alerts.	**Confirmed**
9	Devices never delete pending events before explicit server acknowledgement; each result carries device_action = DELETE or RETAIN_AND_RETRY.	Power loss and network loss cannot lose accepted taps.	**Confirmed**

#	Decision	Why it matters	Status
10	Modular monolith + one worker; no microservices, Kubernetes or Kafka until measurements justify them.	Small team, fewer moving parts, workload (≈0.7–6 events/s per school at peak) is modest.	**Confirmed**
11	Per-device bearer credential (hashed server-side) over TLS in V1, with a documented upgrade path to asymmetric device keys / mTLS and secure-element storage.	Revocable per device, simple to implement on ESP32, upgradeable.	**Proposed**
12	Subscription expiry restricts features but never deletes data and never stops device event ingestion.	Recorded taps are safety-relevant child data and must not vanish over a billing dispute.	**Confirmed**

3.5 Scope of this document

- **In scope:** complete architecture for web, API, worker, database, device firmware and protocol, notifications, subscriptions, security, deployment, DR, monitoring, testing and roadmap.
- **Not in scope:** visual design system and wireframes (UI/UX will produce them against Section 9), legal advice (Section 21 lists validation items), payment-provider selection and pricing (Section 20), PCB/enclosure design (Section 15 lists prototype-to-production considerations).

4. Product Requirements

4.1 Functional scope

The platform provides the capabilities below. The "Plan" column shows the initial package mapping (details and enforcement in Section 20).

Capability	Description	Primary roles	Plan
School administration	School profile, academic sessions, classes and sections, user and membership management.	School Admin	Basic
Dashboards	Principal dashboard (school-wide analytics), teacher dashboard (assigned classes), parent portal.	Principal, Teacher, Parent	Basic
Marks and results	Exams, subjects, mark entry, results, report cards.	Teacher, Parent	Basic
Homework and materials	Assignments, study materials, due dates.	Teacher, Parent	Professional
Timetable and classes	Class timetable, teacher assignments, subjects.	School Admin, Teacher	Professional
Attendance management	Daily attendance, manual classroom attendance, corrections, reports.	Teacher, Principal, Admin	Basic (reports) / Professional (NFC)
NFC arrival/departure tracking	Cards, devices, locations, rule engine, movement tracking.	School Admin, Principal	Professional
Multiple devices per school	Any number of devices within plan limits, concurrent and offline-tolerant.	School Admin, Super Admin	Professional / Enterprise
Real-time parent notifications	Push, email, SMS with preferences and delivery audit.	Parent	Professional
Fee management	Fee records, status, reminders.	School Admin, Parent	Basic
Announcements	School-wide and class-level notices.	Admin, Principal, Teacher	Basic
Academic reports	Term reports, attendance reports, exports.	Principal, Admin	Basic / Professional (advanced)
Device health monitoring	Heartbeats, offline alerts, queue depth, RTC drift, firmware.	School Admin, Super Admin	Professional

Capability	Description	Primary roles	Plan
Subscription management	Plans, limits, grace and suspension, usage.	Super Admin	Platform
Multi-school data isolation	Strict per-school separation in a shared platform.	All	Platform

4.2 Initial scale target

Dimension	Initial target	Design headroom
Students per school	≈ 1,000	Up to ≈ 5,000 without redesign (validate)
Users per school	≈ 1,000 students + ≈ 1,500 parent accounts + ≈ 60–100 staff	Proportional
NFC devices per school	1 – 10 typical	Plan-limited; no architectural limit
Partner schools	Pilot: 1 – 3	Tens to low hundreds on the same architecture; load-test before each growth step
Attendance events	≈ 440,000 / year / school (before extra taps)	See workload model in Section 22

4.3 Quality attributes

Attribute	Expectation
Correctness	No lost, duplicated or mis-attributed attendance events; no cross-school data exposure.
Availability	API and worker always-on; devices keep working during outages and catch up later. Availability SLO to be defined after pilot measurement.
Security and privacy	Child data treated as sensitive; least privilege; audited access.
Maintainability	Modular monolith, typed contracts, SQL-first migrations, automated tests that gate merges.
Operability	Structured logs, alerts for device, queue and database health, documented runbooks.
Usability	Responsive and accessible UI; parent portal mobile-first.

4.4 Assumptions and constraints

- Each school has reliable electrical power at device locations and Wi-Fi coverage most of the time; devices must still tolerate outages (Section 18).
- School timezone is a property of the school (IANA name, e.g. `Asia/Kolkata`); the platform never assumes server timezone.
- Parent contact data (email, phone) is supplied by the school and may be shared between siblings.
- The company, not the school, owns and ships device hardware in the first release.
- Payment collection and invoicing use an external billing provider selected later; the platform only tracks subscription state (Section 20).

Architecture requirements

ID	Requirement	Status
ARCH-REQ-001	The platform SHALL be delivered as one modular-monolith API plus one separate background worker. Microservices, Kubernetes and Kafka SHALL NOT be introduced in V1 unless measurements justify them.	**Confirmed**

ID	Requirement	Status
ARCH-REQ-002	PostgreSQL (Supabase) SHALL be the single authoritative store for school, student, attendance and academic records. No second source of truth (e.g. Firebase Realtime Database) is permitted for attendance.	**Confirmed**
ARCH-REQ-003	Frontend host, API host, database host, file storage and background worker SHALL be distinct deployables with independent release and scaling.	**Confirmed**
ARCH-REQ-004	Browser bundles SHALL contain only public configuration. Database service-role credentials and device secrets SHALL never appear in browser code.	**Confirmed**
ARCH-REQ-005	All business rules (attendance, school isolation, marks, notifications, device registration, subscription enforcement) SHALL execute on the backend; the frontend is a presentation layer only.	**Confirmed**
ARCH-REQ-006	A single machine-readable API contract (OpenAPI / JSON Schema generated from Zod schemas) SHALL be the shared source for web client types, API validation and firmware documentation.	**Proposed**
ARCH-REQ-007	Staging and production SHALL use separate Supabase projects, separate secrets and separate device credential sets.	**Proposed**
ARCH-REQ-008	Adding a new partner school SHALL require configuration/data only (no code change and no schema change).	**Confirmed**
ARCH-REQ-009	The public REST API SHALL be versioned under /api/v1 and breaking changes SHALL ship under a new version prefix.	**Proposed**
ARCH-REQ-010	The worker SHALL be restart-safe: a worker crash or deploy SHALL NOT lose or duplicate notification jobs.	**Confirmed**

> **Traceability**
> Appendix A maps every requirement ID to the implementing module(s) and the test case(s) in Section 28.

5. Technology Stack

The proposed stack from the brief is adopted without change. Where the brief leaves a choice open (query layer, migration tool, test tools, notification providers) a recommendation and alternatives are given. No technically important reason was found to replace any mandated technology.

5.1 Frontend

Technology	Purpose	Integration method	Alternatives	Implementation requirements
React 18+ / TypeScript (strict)	UI framework and type safety	SPA built with Vite; feature-based modules	Next.js (SSR not needed), Vue	`strict: true`; no `any` in feature code; shared types from `packages/contracts`.
Vite	Build tool and dev server	Static build output uploaded to Firebase Hosting	Webpack	Environment variables only `VITE_*`; code-splitting per role layout.
Tailwind CSS	Utility-first styling and design tokens	Tailwind config holds colour, spacing and typography tokens	CSS Modules	Tokens for light/dark; responsive breakpoints sm/md/lg/xl.
shadcn/ui (Radix primitives)	Accessible component primitives copied into repo	Components live in `components/ui`; customised via tokens	MUI, Chakra	Keep Radix a11y behaviours; do not strip aria attributes.
React Router	Routing, nested role layouts, route guards	Data router with lazy route modules	TanStack Router	Protected-route wrapper; permission-aware navigation.
TanStack Query	Server-state fetching, caching, invalidation	One QueryClient; query-key factory per feature	SWR, RTK Query	Default `staleTime` per data type; invalidate on mutation; no server data in global stores.

Technology	Purpose	Integration method	Alternatives	Implementation requirements
React Hook Form + Zod	Forms and client-side validation	`zodResolver`; schemas shared from `packages/contracts`	Formik + Yup	Client validation is UX only; server validates again.
Recharts	Analytics and charts	Lazy-loaded chart components	Chart.js, Visx	Accessible summaries (table alternative) for each chart.

5.2 Backend

Technology	Purpose	Integration method	Alternatives	Implementation requirements
Node.js (current LTS) + TypeScript	Runtime and type safety	Compiled to JS for production	Deno, Bun	Pin LTS in `.nvmrc` and Dockerfile; `strict` TS.
Express.js	HTTP framework	Modular routers per module under `/api/v1`	Fastify, NestJS	Async error wrapper; no business logic in routes.
Zod	Request/response validation and OpenAPI generation	Schemas in `packages/contracts`, applied by `validate()` middleware	Valibot, Joi	Reject unknown keys on device endpoints; ignore `school_id` in bodies.
pino	Structured JSON logging	Request-scoped child logger with `request_id`	winston	Redact tokens, device secrets, card UIDs in logs (hash).
Helmet, CORS, express-rate-limit (or equivalent)	Security headers, origin allow-list, throttling	Global middleware; stricter on auth and device routes	API gateway	Rate-limit store: in-memory V1 per instance; Redis/Postgres-backed if multi-instance accuracy is needed (validate).
`pg` + Kysely (typed SQL builder)	Database access with explicit SQL control (ON CONFLICT, FOR UPDATE)	Repository layer; `withTenantTx()` helper	Prisma, Drizzle, Knex	Raw SQL allowed where clearer; every query runs inside a tenant-scoped transaction.
Sentry (or equivalent)	Error tracking	SDK in API, worker, web	Rollbar, self-hosted	Scrub PII; release tagging from CI.

5.3 Database and authentication

Technology	Purpose	Integration method	Alternatives	Implementation requirements
Supabase PostgreSQL	Authoritative relational database	API connects through the pooler as a least-privilege role; migrations via Supabase CLI SQL files	Neon, RDS, Cloud SQL	Never expose service-role key outside the API/worker; PostgREST access for `anon`/`authenticated` revoked on application tables.
Supabase pooler (Supavisor)	Connection pooling	Transaction-mode URL for API; direct/session URL for migrations and worker locks	PgBouncer	Avoid session-level state across statements; use `set_config(..., true)` inside transactions.
Supabase Auth	Identity, passwords, recovery, sessions, MFA	Browser obtains JWT from Supabase Auth; API verifies the JWT and maps `sub` to `users.id`	Auth0, Clerk, Keycloak	JWT verified server-side on every request; roles read from DB memberships, not from client-editable claims.
Supabase Storage	Private object storage for documents and report cards	API issues short-lived signed URLs after authorisation	S3 / GCS	Buckets private; path convention `school_id/…`; storage policies deny direct browser listing.
Supabase CLI	Migrations, local stack for tests	SQL files in `db/migrations`, applied by CI	node-pg-migrate, Flyway	Forward-only; expand/contract.

5.4 Hosting, delivery and tooling

Component	Choice	Role and separation	Alternatives
Frontend host	Firebase Hosting	Serves static SPA from CDN; security headers in `firebase.json`; separate staging site.	Cloudflare Pages, Netlify, Vercel
Backend host	Render web service	Runs the Node API; always-on paid instance recommended because devices upload continuously (free-tier spin-down is unsuitable; validate current plan features).	Fly.io, Railway, AWS ECS
Background worker	Render background worker	Separate process for outbox delivery and scheduled jobs; independent restarts and scaling.	Same host with different process type
Database host	Supabase PostgreSQL	Managed Postgres; backups/PITR per plan (validate).	As above
File storage	Supabase Storage	Report cards, study materials, firmware images (private).	S3-compatible
Source control and CI/CD	GitHub + GitHub Actions	Pull-request checks, staging auto-deploy, production approval gate.	GitLab CI
Push notifications	Firebase Cloud Messaging	Delivery channel only; **not** a data store.	OneSignal
Email	Provider adapter (e.g. SES, Resend, SendGrid)	Transactional email; provider chosen after cost and deliverability review (validate).	—
SMS	Provider adapter	Indian SMS requires operator-regulated sender/template registration (DLT); confirm with the chosen provider before launch (validate).	—
Testing	Vitest, Supertest, Playwright, k6, Testcontainers / `supabase start`	Unit, API, E2E, load and DB tests.	Jest, Cypress, Artillery
Firmware toolchain	PlatformIO + Arduino-ESP32 for prototype; ESP-IDF for production	ESP-IDF provides secure boot, flash encryption and OTA partitioning.	Zephyr

Query layer
Attendance processing depends on `INSERT … ON CONFLICT`, `SELECT … FOR UPDATE`, `SKIP LOCKED` and explicit transaction control. A typed SQL builder (Kysely) keeps these visible and testable. A heavy ORM is not recommended for the attendance and outbox modules. (Proposed)

5.5 Deviations from the proposed stack

None. Additions that the brief did not name (pino, Helmet, Kysely, Sentry, Playwright, k6) are tooling choices that do not change the mandated stack.

6. System Architecture

6.1 Design principles

- **One source of truth.** PostgreSQL holds every authoritative record. Caches, push messages and device queues are derived or temporary.
- **The server decides.** Devices and browsers report facts and requests; the API decides what they mean and whether they are allowed.
- **Defence in depth for tenant isolation.** Identity → membership → RBAC/scope → tenant context → RLS and composite keys.
- **Never lose an accepted event.** Durable device queue, idempotent ingestion, acknowledged-after-commit, outbox for side effects.

- **Boring technology first.** Modular monolith, managed Postgres, a worker process. Add complexity only when measurements demand it.

6.2 Components and responsibilities

Component	Responsibility	Runs on	Holds secrets?
Web SPA (React)	All user interfaces for the five roles; talks only to the API and to Supabase Auth.	Firebase Hosting / user browser	No (public config only)
API (Express)	Authenticates users and devices, enforces authorisation and tenancy, runs business logic, serves REST, issues signed URLs.	Render web service	Yes: DB credentials, Supabase service key, provider keys
Worker	Drains notification outbox, expands recipients, calls FCM/email/SMS, runs scheduled jobs (absent finalisation, device watchdog, subscription state, retention).	Render background worker	Yes: DB credentials, provider keys
PostgreSQL	Authoritative data, constraints, RLS, audit history.	Supabase	Managed
Supabase Auth	Identities, passwords, recovery, sessions, MFA.	Supabase	Managed
Supabase Storage	Report cards, documents, firmware images.	Supabase	Managed
NFC device	Reads cards, timestamps taps, stores durably, uploads, reports health, applies signed OTA updates.	School premises	Only its own device credential
External providers	FCM, email, SMS, error monitoring, uptime monitoring.	Third parties	Provider-held

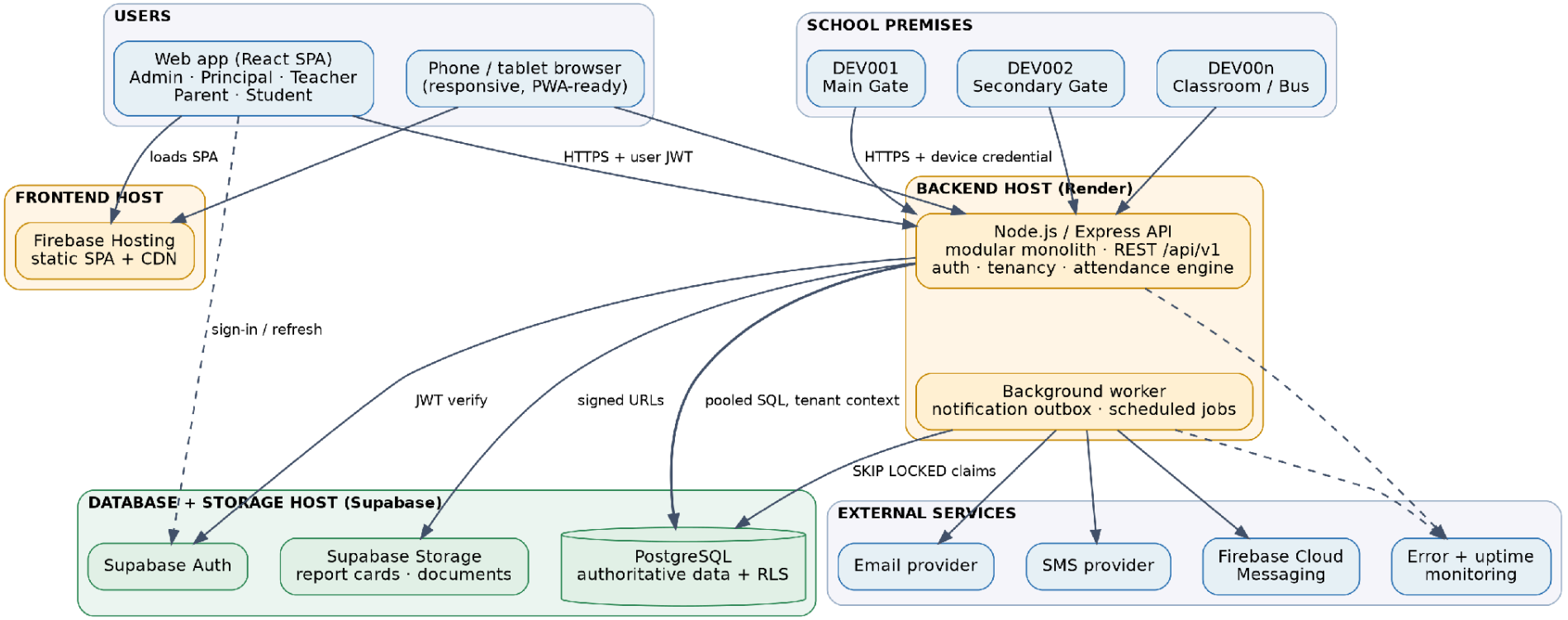


Figure 1 — Complete system architecture

6.3 Primary data flows

6.3.1 Interactive web request

1. Browser signs in with Supabase Auth and receives a short-lived access JWT plus a refresh token.
2. Browser calls `https://api.<domain>/api/v1/...` with `Authorization: Bearer <JWT>`.
3. API verifies the JWT, loads the user's active school membership(s) from the database, resolves the active school, and checks role, scope and subscription entitlements.
4. API opens a transaction, sets the tenant context, executes repository queries (RLS active) and commits.
5. API returns a typed JSON response; TanStack Query caches it according to its freshness policy.

6.3.2 NFC tap

1. Student taps a card; device reads the UID, stamps time, appends an event to its durable queue, and flashes "recorded".
2. Device uploads batches to `/api/v1/device/events/batch` with its credential.
3. API authenticates the device, derives the school, and processes each event in its own transaction (Section 17).
4. Transaction commits the event, daily attendance change and notification outbox row together.
5. API returns per-event results; the device deletes only events marked `DELETE`.

6.3.3 Parent notification

1. Worker claims outbox rows with `FOR UPDATE SKIP LOCKED`.
2. Worker expands the student to linked parents, applies their preferences, creates delivery rows and calls the provider.
3. Delivery outcomes are recorded; transient failures retry with backoff; exhausted jobs become DEAD and raise an alert.

6.4 Trust boundaries

Boundary	Untrusted side	Controls at the boundary
Internet → API (users)	Browser, any HTTP client	TLS, CORS allow-list, JWT verification, rate limits, validation, RBAC, tenant context
Internet → API (devices)	Device firmware and anyone holding a device credential	TLS, per-device credential, school derived server-side, schema validation that ignores school/student fields, idempotency, quotas
API → Database	Application bugs	Least-privilege role, RLS, composite FKs, constraints, transaction-scoped tenant setting
API → Storage	Direct URL guessing	Private buckets; signed URLs issued after authorisation; short TTL
Worker → Providers	Provider outages and rate limits	Retries, backoff, circuit breaker, dead-letter, no secrets in payload logs
Physical device	Anyone with physical access to a unit	Credential scoped to one device; no student PII on device; flash encryption and secure boot in production

6.5 Host separation

Concern	Host	Scaling and release independence
Frontend static files	Firebase Hosting	Deploy without touching API; rollback by previous release.

Concern	Host	Scaling and release independence
REST API	Render web service	Scale instances horizontally; zero-downtime deploys.
Background jobs	Render background worker	Scale and restart independently; no HTTP exposure.
Database	Supabase PostgreSQL	Vertical scaling, read replica later if measured need.
Files	Supabase Storage	Independent bucket policies and lifecycle.

Single-region dependency
Both Supabase and Render run in one region each in V1. Choose regions close to the schools (for India, a South/South-East Asian region if available — validate) and keep API and database in the same or neighbouring region to hold latency down. Multi-region failover is out of scope for V1; the device queue and restore drills (Section 24) cover the realistic failure modes.

7. User Roles and Permissions

7.1 Roles

Role	Scope	Typical user	Authentication notes
Platform Super Admin	Platform-wide, not tied to a school	Company operations / support staff	Stored in `platform_staff`; MFA required (Proposed); no routine access to student records.
School Admin	One school (or campus set)	School office administrator	School membership with role `school_admin`; MFA strongly recommended.
Principal	One school	Head of school	Membership `principal`; MFA recommended.
Teacher	One school; further scoped to assigned classes/subjects	Teaching staff	Membership `teacher` + `teacher_class_assignments`.
Parent / Student	One school (may have children or records in several schools: one membership per school)	Parent, guardian or student	Membership `parent` + `student_parents` links.

7.2 Permission matrix

Legend: **Y** = permitted; **S** = permitted within scope (own school, assigned classes, or linked student/child/self); **R** = read-only; **A** = by approval workflow; **—** = not permitted. The matrix is enforced in the API; the frontend only hides what the API would refuse (FE-REQ-002).

Capability	Super Admin	School Admin	Principal	Teacher	Parent
Register partner schools	Y	—	—	—	—
Configure plans; activate / suspend subscriptions; set school limits	Y	—	—	—	—
View platform health, usage, support requests	Y	—	—	—	—
Register hardware devices for a school; revoke any device	Y	—	—	—	—
Claim, name, assign location, disable or revoke own school's devices	—	Y	R	—	—

Capability	Super Admin	School Admin	Principal	Teacher	Parent
View device status, heartbeat, queue depth	Y	Y	R	—	—
Manage school profile and academic sessions	Grant only	Y	R	—	—
Manage users and memberships	Grant only	Y	—	—	—
Create classes and sections; manage timetable	Grant only	Y	R	R (assigned)	R (linked student)
Manage student and parent records; link parents to students	Grant only	Y	R	R (assigned classes)	R (linked student/self)
Assign / block / replace NFC cards	—	Y	R	—	—
Configure attendance schedules and rules	—	Y	R	—	—
View daily attendance and arrival/departure history	Grant only	Y	Y (school-wide)	S (assigned classes)	S (linked student/self)
Manual classroom attendance	—	Y	R	S	—
Request attendance correction	—	Y	Y	S	S (linked student/self)
Approve attendance correction	—	Y (per policy)	Y	—	—
Enter / update marks and results	—	Y (audited override)	R	S (assigned class + subject)	R (linked student/self)
Upload homework and study materials	—	Y	R	S	R (linked student/self)
Manage fee records	—	Y	R	—	R (linked student/self)
Create announcements	—	Y	Y	S (own classes)	R
School-wide academic and attendance analytics	—	Y	Y	S (own classes)	—
Export reports	—	Y	Y	S	S (linked student/self)
Set notification preferences	—	—	—	—	S (self)
View subscription plan and limits	Y	R	R	—	—
View audit logs	Y (platform)	R (school)	—	—	—
Access student data for support	A (time-boxed grant, audited)	—	—	—	—

7.3 Scope rules enforced beyond the role

Rule	Enforcement
Parent → student	Every student-scoped route checks an active row in `student_parents` for (school, parent/user, student). Removing the link revokes access immediately (no long-lived cache of the relationship beyond a single request).
Teacher → class	Every class- or student-scoped route checks `teacher_class_assignments` (class, subject where relevant, academic session active).
Principal / Admin → school	Scope is the school of the membership; cross-school requests return 404.
Multiple memberships	A person with memberships in several schools selects an active school; the API requires `X-School-Id` and verifies it against the memberships (it is a selector, never an authority).

7.4 Super Admin and student records

The company's staff must not casually read student data. The design therefore separates **platform operations** (schools, plans, device inventory, health) from **school data**:

- Platform-level queries run under a dedicated database role (`app_platform`) that has privileges only on platform tables and on non-personal device/subscription metadata. It has **no** privilege on student, parent, marks, attendance or fee tables.
- For genuine support work a Super Admin creates a **support access grant**: school, reason, ticket reference, scope (tables or features), requested duration (default 60 minutes, maximum 24 hours). Depending on contract, the School Admin must approve it (Proposed).
- While a grant is active, the API opens read-only tenant-scoped transactions on behalf of the staff member, tags every audit log row with the grant id, and shows the school admin a notice.
- All grants and every record read under them are written to `audit_logs`. Expiry is enforced server-side.

7.5 Roles and permissions in the database

Roles are rows in `roles`; permissions are rows in `permissions` (stable codes); `role_permissions` maps them. Membership gives a user exactly one role per school. The code checks **permission codes**, not role names, so a new role is a data change.

Permission code (examples)	Meaning
`students.read` / `students.write`	View / modify student records within scope
`attendance.read` / `attendance.manual_write` / `attendance.correct_request` / `attendance.correct_approve`	Attendance viewing and correction workflow
`marks.read` / `marks.write`	Marks and results
`devices.read` / `devices.manage`	Device status / lifecycle
`cards.manage`	Assign, block, replace NFC cards
`rules.manage`	Attendance rule versions
`fees.read` / `fees.write`	Fee records
`announcements.write`	Create announcements
`reports.export`	Export or download reports
`users.manage`	Memberships and invitations
`audit.read`	View audit logs

7.6 Multi-factor authentication policy

- Super Admin: MFA required (Proposed).
- School Admin and Principal: MFA strongly recommended and enforceable by the school; platform default is "prompt after first login" (Proposed).
- Teachers, parents: optional.
- Sensitive operations (role change, device revoke, support grant, bulk export) may require a recent re-authentication ("step-up") (Proposed).

8. Multi-Tenant Architecture

8.1 Tenancy model and rationale

Partner schools share one application, one API, one worker and one PostgreSQL database with a shared schema. Each school is a tenant identified by `schools.id`. This is the most economical model to operate for tens to low hundreds of schools of ≈1,000 students, and it keeps migrations to a single step.

Model	Isolation strength	Operational cost	Verdict
Shared database, shared schema, `school_id` on every row (chosen)	Logical; strengthened by composite FKs + FORCED RLS + tests	Lowest: one migration, one pool, one backup	Chosen for V1 (Confirmed)
Shared database, schema per school	Stronger namespace separation	Migrations ×N, catalog bloat, harder pooling	Not needed at this scale
Database per school	Strongest	Highest: N backups, N pools, N migrations	Offer only as a contractual Enterprise option later

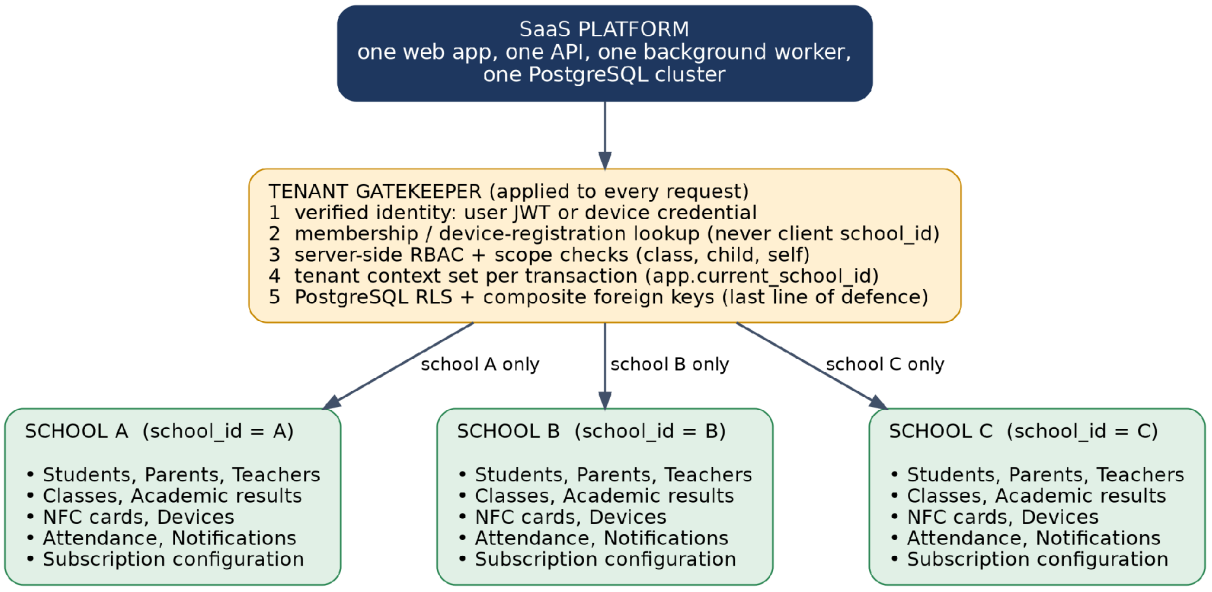


Figure 2 — Multi-school tenant architecture

Each school has its own students, parents, teachers, classes, attendance records, NFC cards, devices, academic results, notifications and subscription configuration. None of these rows is reachable without passing the tenant gatekeeper shown above.

8.2 Defence in depth

Layer	Mechanism	What it stops
1. Identity	Supabase JWT (users) or device credential (devices) verified on every request	Anonymous access; forged identities
2. Membership / registration	School resolved from `school_memberships` or `devices.school_id` — never from a body, query or header value alone	A client choosing someone else's school
3. Authorisation	Permission codes + scope checks (class, child, self) + subscription entitlement	Over-privileged actions within the right school

Layer	Mechanism	What it stops
4. Tenant context	`withTenantTx` sets `app.current_school_id` for the transaction only	A repository bug that forgets a WHERE clause
5. Database	FORCED RLS on every school-owned table; composite FKs `(school_id, id)`; unique constraints scoped by `school_id`	Cross-school joins, mis-assigned references, buggy queries
6. Verification	Automated cross-school tests on every route and table; policy-existence tests in CI	Regressions when new tables or routes are added

8.3 School ownership in the schema

Every school-owned table has `school_id uuid NOT NULL REFERENCES schools(id)`. Every parent table that is referenced by school-owned children also declares `UNIQUE (school_id, id)`. Children reference parents with **composite** foreign keys, so PostgreSQL itself refuses a row that mixes schools:

Composite foreign keys enforce same-school references

```
-- parent
ALTER TABLE students ADD CONSTRAINT students_school_id_id_uk UNIQUE (school_id, id);
ALTER TABLE classes  ADD CONSTRAINT classes_school_id_id_uk  UNIQUE (school_id, id);

-- child: marks reference a student and an exam of the SAME school
CREATE TABLE marks (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id   uuid NOT NULL REFERENCES schools(id),
  exam_id     uuid NOT NULL,
  student_id  uuid NOT NULL,
  subject_id  uuid NOT NULL,
  score       numeric(6,2) NOT NULL CHECK (score >= 0),
  entered_by  uuid NOT NULL REFERENCES users(id),
  FOREIGN KEY (school_id, exam_id)    REFERENCES exams (school_id, id),
  FOREIGN KEY (school_id, student_id) REFERENCES students (school_id, id),
  FOREIGN KEY (school_id, subject_id) REFERENCES subjects (school_id, id),
  UNIQUE (school_id, exam_id, student_id, subject_id)
);
```

8.4 Row-Level Security

Policy template applied to every school-owned table by a migration helper

```
-- Roles: the API connects as a login that is a member of app_api (NOT superuser, NOT BYPASSRLS)
ALTER TABLE students ENABLE ROW LEVEL SECURITY;
ALTER TABLE students FORCE  ROW LEVEL SECURITY;   -- applies even to the table owner

CREATE POLICY tenant_isolation ON students
  AS PERMISSIVE FOR ALL TO app_api
  USING      (school_id = nullif(current_setting('app.current_school_id', true), '')::uuid)
  WITH CHECK (school_id = nullif(current_setting('app.current_school_id', true), '')::uuid);

-- Browser-facing Supabase roles get nothing on application tables (API is the only data path)
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM anon, authenticated;
```

Tenant-scoped transaction helper (transaction-mode pooler safe)

```
// apps/api/src/database/withTenantTx.ts
export async function withTenantTx<T>(ctx: TenantContext, fn: (tx: Tx) => Promise<T>): Promise<T> {
  return db.transaction().execute(async (tx) => {
    // third argument true = transaction-local: reset automatically at COMMIT/ROLLBACK,
    // so a pooled connection can never leak a previous tenant into the next request
    await sql`select set_config('app.current_school_id', ${ctx.schoolId}, true),
                     set_config('app.current_user_id',   ${ctx.userId ?? ''}, true),
                     set_config('app.actor_type',        ${ctx.actorType}, true)`.execute(tx);
    return fn(tx);
  });
}
```

> **Why RLS in addition to API checks**
> API authorisation is the primary control because RLS cannot express "this teacher may edit only class 8A". RLS is the safety net: if a developer forgets `WHERE school_id = …` the database still returns only the active school's rows. The design deliberately does **not** rely on Supabase client-side RLS from the browser; all application data goes through the API.

8.5 Preventing cross-school access — the request path

1. Authenticate (JWT or device credential).
2. Load active memberships (user) or device registration (device). Choose the active school: for users the `X-School-Id` header must match one of their memberships; for devices the school is `devices.school_id`.
3. Build an immutable `TenantContext { schoolId, userId, role, permissions, actorType }` and attach it to the request. Handlers read school identity **only** from this object.
4. Validate input with Zod; any `school_id` field in a body is stripped (browser) or rejected as `FOREIGN_FIELD_IGNORED` (device).
5. Run all queries in `withTenantTx`.
6. For a not-found or foreign-school resource return the same `404 NOT_FOUND`.

8.5.1 Device requests

A device is authenticated by `X-Device-Id` plus bearer secret. The API looks up `devices` by `device_code`, verifies the secret hash and status, and uses `devices.school_id` as the tenant. Any `school_id`, `student_id` or foreign `device_id` in the payload is ignored or rejected. A device therefore cannot write to, or read from, another school even if its firmware is modified.

8.6 Platform-level (cross-tenant) operations

- School registration, plan configuration and platform health use the `app_platform` role with grants limited to platform tables and aggregate/health views.
- Cross-school aggregates (usage, device health) expose counts and statuses, not personal data.
- Scheduled jobs that must scan all schools (absent finalisation, outbox draining) run under `app_worker` and still set the tenant context for each school or event they handle, so RLS stays on.

8.7 Multi-campus and enterprise tenants

Enterprise customers with several campuses are modelled as an `organizations` row owning several `schools` rows (each campus is a school with its own students, devices and rules). Organization-level roles (e.g. group principal) read across the organization's schools through explicit memberships; they are never implicit. This avoids changing the tenancy model later. (Proposed)

8.8 Tenant offboarding and export

- Termination moves the school to `TERMINATED`, blocks logins, stops notifications, and starts the contractual retention period (Section 21).
- A full export is produced per school (SQL/CSV/JSON plus storage objects) because `school_id` partitions every table naturally.
- Deletion after retention is a documented, audited job that deletes in dependency order within one school only.

8.9 Automated cross-school tests

CI must prove isolation, not assume it (TEN-REQ-005):

- **Route sweep:** for every registered route, fixtures with two schools; call with School A credentials and School B identifiers (path, query, body); expect 404 and no side effects.
- **Table sweep:** a test enumerates all tables with a `school_id` column and asserts RLS enabled + forced and policy present; it asserts composite FKs exist for every relationship to a school-owned parent.
- **Role sweep:** connect as `app_api` without a tenant setting and assert zero rows from every table.
- **Device sweep:** a device from School A sends School B ids; assert nothing leaks and nothing is written under School B.

Multi-tenancy requirements

ID	Requirement	Status
TEN-REQ-001	Every school-owned table SHALL carry a NOT NULL school_id referencing schools(id).	**Confirmed**
TEN-REQ-002	References between school-owned rows SHALL use composite foreign keys (school_id, id) so a row cannot reference a parent that belongs to another school.	**Confirmed**
TEN-REQ-003	The API SHALL derive school context from the authenticated membership (users) or the authenticated device registration (devices). A school_id supplied by a browser or device SHALL never be trusted.	**Confirmed**
TEN-REQ-004	PostgreSQL Row-Level Security SHALL be enabled and FORCED on every school-owned table, with policies keyed on a transaction-local tenant setting.	**Confirmed**
TEN-REQ-005	Automated cross-school access tests SHALL run in CI against every API route and every school-owned table; a failing test SHALL block merge.	**Confirmed**
TEN-REQ-006	Requests for another school's resource SHALL return 404 (not 403) to avoid confirming that the resource exists.	**Proposed**
TEN-REQ-007	Platform staff access to school-owned student data SHALL require a time-boxed, reason-bearing support_access_grant and SHALL be written to audit_logs.	**Confirmed**

9. Frontend Architecture

9.1 Principles

- Feature-based modules: each business area owns its pages, components, hooks, API calls and schemas.
- Role-specific layouts compose shared components; roles never fork the component library.
- The frontend renders and collects input. **It contains no secrets and no authoritative business rules** (FE-REQ-002, FE-REQ-006).
- Server state lives in TanStack Query; local UI state in component state; there is no global store for server data.

9.2 Folder structure

apps/web/src

```
apps/web/src/
  app/                    # app bootstrap: providers, QueryClient, router, error boundary
  components/
    ui/                   # shadcn/ui primitives (Button, Dialog, Table, Form ...)
    common/               # shared composites (DataTable, PageHeader, EmptyState, ConfirmDialog)
    charts/               # Recharts wrappers with table alternatives
  layouts/                # PlatformLayout, AdminLayout, PrincipalLayout, TeacherLayout,
                          # ParentLayout (mobile-first), StudentLayout, AuthLayout
  features/
    auth/                 # login, recovery, MFA enrol, session hooks
    schools/              # school profile, academic sessions, (platform) school registry
```

```
    students/  parents/  teachers/  classes/
    attendance/           # daily views, live feed, corrections, rule configuration UI
    exams/  marks/  homework/  timetable/
    fees/  notifications/ # inbox, preferences, push-token registration
    devices/              # device list, health, claim, location assignment, revoke
    subscriptions/        # plan view (school) and plan admin (platform)
    reports/              # report builder, async export status, downloads
      <feature>/
        pages/  components/  hooks/  api.ts  keys.ts  schemas.ts  types.ts
  hooks/                  # cross-feature hooks: useAuth, usePermission, useDebounce, useMediaQuery
  services/               # apiClient.ts, supabaseClient.ts (auth only), fcm.ts, analytics.ts
  schemas/                # Zod schemas re-exported from packages/contracts
  types/                  # generated API types, domain types
  utils/                  # date/time (school timezone), formatters, a11y helpers
  routes/                 # route table, ProtectedRoute, RoleRedirect, route permission map
```

Folder	Responsibility
`app/`	Providers (QueryClientProvider, Router, theme), global error boundary, toast host, session bootstrap.
`components/ui`	Unstyled-to-styled primitives from shadcn/ui; the only place where Radix is wrapped.
`components/common`	Composite components used by several features (DataTable with server pagination, filter bar, form field wrappers, empty and error states).
`layouts/`	Per-role shells: navigation, breadcrumbs, school switcher, notification bell, responsive drawers. Layouts contain no feature logic.
`features/*`	Vertical slices. A feature may import from `components`, `hooks`, `services`, `schemas`, `utils`; features do not import each other's internals (use public `index.ts`).
`hooks/`	`useAuth()` (session + memberships), `usePermission(code)` (UI-only checks), `useSchoolTimezone()`.
`services/`	`apiClient` (fetch wrapper with auth, retries, error normalisation), `supabaseClient` (Auth only), FCM web registration.
`schemas/`	Zod schemas shared with the API through `packages/contracts` so form validation and server validation cannot drift.
`routes/`	Route definitions, lazy loading, guards, permission map; the single place that maps URLs to roles.

9.3 Routing and protected routes

Area	Path prefix	Layout	Allowed roles
Public	`/login`, `/forgot-password`, `/reset-password`	AuthLayout	Anyone
Platform	`/platform/*`	PlatformLayout	Super Admin
School admin	`/admin/*`	AdminLayout	School Admin
Principal	`/principal/*`	PrincipalLayout	Principal
Teacher	`/teacher/*`	TeacherLayout	Teacher
Parent	`/parent/*`	ParentLayout	Parent
Student	`/student/*`	StudentLayout	Student
Root	`/`	—	Redirect by role (RoleRedirect)

Protected route wrapper

```
// routes/ProtectedRoute.tsx
export function ProtectedRoute({ roles, permission, children }: Props) {
  const { status, session, activeMembership } = useAuth();
  const location = useLocation();

  if (status === 'loading') return <FullPageSpinner />;
  if (status === 'unauthenticated')
    return <Navigate to="/login" replace state={{ returnTo: location.pathname + location.search }} />;
  if (roles && !roles.includes(activeMembership.role)) return <ForbiddenPage />;
  if (permission && !activeMembership.permissions.includes(permission)) return <ForbiddenPage />;
```

```
  return children ?? <Outlet />;
}
// NOTE: this only improves UX. The API re-checks every request (FE-REQ-002).
```

9.4 Permission checks in the UI

- `usePermission("marks.write")` toggles buttons, menu items and form fields. A hidden control is **not** a security control.
- Permission lists come from `GET /api/v1/me` (memberships, role, permission codes, subscription features) and are cached briefly; the API remains authoritative.
- Feature availability by subscription (`features.nfc_attendance`) hides modules and shows an upgrade notice; the API returns `SUBSCRIPTION_FEATURE_UNAVAILABLE` if called anyway.

9.5 API client and query caching

Single API entry point

```
// services/apiClient.ts (sketch)
export async function api<T>(path: string, init: ApiInit = {}): Promise<T> {
  const token = await getAccessToken();                       // from Supabase session, refreshed if near
expiry
  const res = await fetch(`${API_BASE}${path}`, {
    ...init,
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
      'X-School-Id': getActiveSchoolId(),                     // selector only; server verifies
membership
      'X-Request-Id': crypto.randomUUID(),
      ...init.headers,
    },
  });
  if (res.status === 401) { await handleExpiredSession(); throw new ApiError('SESSION_EXPIRED'); }
  if (!res.ok) throw await toApiError(res);                   // normalises the error envelope
  return res.status === 204 ? (undefined as T) : res.json();
}
```

Concern	Convention
Query keys	Factory per feature: `studentKeys.list(filters)`, `studentKeys.detail(id)`; always include active school id as first element so switching schools never shows stale data.
Freshness	Reference data (classes, subjects) `staleTime` 10 min; lists 30 s; live attendance feed `refetchInterval` 15 s (Proposed; Supabase Realtime is a later optimisation); marks entry data `staleTime` 0.
Mutations	`useMutation` + targeted `invalidateQueries`; optimistic updates only for low-risk toggles.
Retries	Queries retry twice with backoff on network/5xx; never retry 4xx.
Pagination	Server-side pagination for all lists (`page`, `pageSize` ≤ 100) with `keepPreviousData`; cursor pagination for event feeds.
Persistence	No sensitive query data persisted to localStorage; in-memory only.

9.6 Loading, empty and error states

- Every data view implements four states: **loading** (skeletons, not blocking spinners), **empty** (explains why and what to do), **error** (human message, retry action, request id for support) and **success**.
- Route-level error boundary catches render errors; Sentry receives them with the request id.
- Mutation errors map `VALIDATION_ERROR` field messages to form fields; other codes show a toast with the stable code.
- Offline detection (`navigator.onLine`) shows a banner; mutations are not queued offline in V1.

9.7 Forms and validation

- React Hook Form + `zodResolver` with schemas from `packages/contracts`.
- Client validation gives instant feedback; **the server validates again** and is authoritative.
- Long forms (marks entry, bulk uploads) save drafts in memory and warn on unsaved navigation; large uploads show per-row error reports from the API.
- Dates and times are shown in the **school timezone**, stored and transmitted as ISO-8601 UTC instants or `YYYY-MM-DD` school-local dates.

9.8 Responsive layouts and accessibility

Breakpoint	Layout behaviour
< 640 px (phones)	Bottom navigation for Parent/Student; tables become cards; forms single column; touch targets ≥ 44 px.
640–1023 px (tablets)	Collapsible sidebar; two-column forms; charts resize.
≥ 1024 px (desktop dashboards)	Persistent sidebar, multi-panel dashboards, dense tables with column controls.

- Target WCAG 2.1 AA (FE-REQ-003, Validate): semantic landmarks, visible focus, colour contrast ≥ 4.5:1, labels for every input, `aria-live` for toasts and async results, keyboard-operable tables and dialogs.
- Charts always provide a data-table alternative and text summary.
- Automated axe checks in Playwright; manual keyboard and screen-reader pass before pilot.

9.9 Authentication expiry handling

1. Supabase client refreshes the access token before expiry; `apiClient` reads the current token for each call.
2. On a 401 the client attempts one forced refresh and replays the request once.
3. If refresh fails (revoked, expired refresh token, password changed), the user is sent to `/login` with `returnTo`; in-memory form drafts are kept for the session so a quick re-login does not lose work.
4. Parent portal "remember me" uses the refresh token lifetime configured in Supabase; privileged roles use shorter sessions (Proposed).

9.10 Configuration and security headers

- Only `VITE_API_BASE_URL`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_FCM_*` public keys are bundled. CI fails if the bundle contains a service-role JWT pattern (TEST-REQ-040).
- Firebase Hosting sends CSP (default-src 'self'; connect-src limited to API, Supabase and FCM), HSTS, `X-Content-Type-Options`, `Referrer-Policy`, `frame-ancestors 'none'`.
- No user content is rendered with `dangerouslySetInnerHTML`; announcements use sanitised Markdown.

Frontend requirements

ID	Requirement	Status
FE-REQ-001	The web app SHALL provide role-specific layouts (platform, admin, principal, teacher, parent, student) that share common components.	**Confirmed**
FE-REQ-002	Protected routes and permission-aware UI SHALL be treated as user-experience aids only; every action SHALL be re-authorised by the API.	**Confirmed**
FE-REQ-003	The UI SHALL target WCAG 2.1 AA conformance for core workflows (keyboard navigation, contrast, labels, focus management).	**Validate**

ID	Requirement	Status
FE-REQ-004	Expired or revoked sessions SHALL be handled without data loss: silent refresh first, then redirect to sign-in preserving the user's return path.	**Proposed**
FE-REQ-005	Layouts SHALL be responsive for desktop dashboards, tablets and mobile phones; the parent portal is designed mobile-first.	**Confirmed**
FE-REQ-006	Only variables prefixed VITE_ and containing public values (API base URL, Supabase URL, Supabase anon key, FCM web config) SHALL be bundled.	**Confirmed**

10. Backend Architecture

10.1 Modular monolith

The API is one deployable Node.js service organised into modules. A module owns its routes, controllers, services, repositories and validation schemas and exposes a small public interface to other modules. Modules call each other through those interfaces, never by reaching into another module's tables. If a module later needs to become a service, the seam already exists.

10.2 Folder structure

apps/api/src

```
apps/api/src/
  config/            # env parsing (Zod), constants, feature defaults
  middleware/        # requestId, httpLogger, rateLimit, bodyLimit, authenticate, deviceAuth,
                     # tenant, authorize, entitlements, validate, errors, notFound
  modules/
    auth/            # /me, session info, MFA hooks, invitation acceptance
    schools/         # school profile, registration (platform), support access grants
    students/  parents/  teachers/
    academics/       # sessions, classes, subjects, exams, marks, homework, timetable
    attendance/
      ingest/        # device event ingestion service (transaction)
      engine/        # pure rule functions: classify, movement transition, replay
      rules/         # attendance_rule_versions CRUD and resolution
      corrections/   # correction workflow
      reports/       # attendance queries
    devices/         # registry, locations, credentials, heartbeat, config, OTA, health
    notifications/   # outbox API, preferences, push tokens, templates
    fees/  subscriptions/  reports/  audit/
      <module>/ routes.ts  controller.ts  service.ts  repository.ts  schemas.ts  types.ts  index.ts
  database/          # pool, withTenantTx, Kysely types (generated), migration helpers
  jobs/              # job definitions shared with apps/worker (absentFinalise, deviceWatchdog,
                     # subscriptionState, retention, outboxDrain)
  integrations/      # fcm, email, sms, storage, sentry adapters (interfaces + implementations)
  utils/             # errors, ids, time (school tz), hashing, pagination, result types
  app.ts             # builds the Express app (no listen) for tests
  server.ts          # listens, graceful shutdown
```

10.3 Layering

Layer	Does	Does not
Route	Maps method+path to middleware chain and controller; declares required permission and Zod schemas.	Contain logic.
Controller	Reads validated input and `req.tenant`; calls one service method; shapes the HTTP response.	Query the database; read `school_id` from input.
Service	Business rules; opens `withTenantTx`; orchestrates repositories; emits audit and outbox rows; throws typed errors.	Know about HTTP.
Repository	Typed SQL for one aggregate; takes a `tx`; always relies on tenant context.	Decide authorisation.

Layer	Does	Does not
Engine (pure functions)	Attendance classification and movement transitions with no I/O, so they are unit-testable with thousands of cases.	Touch the database.

10.4 Request lifecycle (user routes)

Middleware order

```
requestId -> httpLogger -> securityHeaders(helmet) -> cors(allow-list) -> bodyLimit(1MB)
  -> rateLimit(ip) -> authenticate(JWT) -> tenant(resolve membership, build TenantContext)
  -> rateLimit(user) -> authorize(permission) -> entitlements(feature/limits) -> validate(zod)
  -> controller -> service(withTenantTx) -> repository -> response
  -> errors (centralised) -> httpLogger(response)
```

Route → service example

```
// modules/marks/routes.ts
router.put(
  '/classes/:classId/exams/:examId/marks',
  authorize('marks.write', { scope: 'teacherClass', param: 'classId' }),
  entitlements.requireFeature('marks'),
  validate({ params: ClassExamParams, body: BulkMarksBody }),
  asyncHandler(marksController.bulkUpsert),
);

// modules/marks/service.ts
export async function bulkUpsert(ctx: TenantContext, input: BulkMarksInput) {
  return withTenantTx(ctx, async (tx) => {
    await classesRepo.assertTeacherAssigned(tx, ctx, input.classId);      // scope check
    const result = await marksRepo.upsertMany(tx, ctx, input);            // composite FKs + unique key
    await audit.record(tx, ctx, 'marks.bulk_upsert', { classId: input.classId, rows: result.count });
    return result;
  });
}
```

10.5 Validation

- Zod schemas per endpoint for params, query and body. Unknown keys are stripped on user endpoints and **rejected on device endpoints**.
- Schemas are exported from `packages/contracts` and generate the OpenAPI document (ARCH-REQ-006).
- Business-rule validation (e.g. "score ≤ exam maximum") happens in services and returns `VALIDATION_ERROR` with field paths.

10.6 Transactions

- Every write path runs in a transaction via `withTenantTx`; read paths also use it so RLS has context.
- Attendance ingestion uses one transaction **per event** so one bad event never blocks the others in a batch (Section 17).
- Bulk academic writes (marks upload) are atomic per file by default, with a "partial accept" option returning a row-error report.
- Retry only on serialization/deadlock errors (SQLSTATE 40001, 40P01) with bounded attempts.

10.7 Authorisation middleware

Fail-closed authorisation

```
export const authorize = (permission: string, opts: ScopeOpts = {}) =>
  async (req: Request, _res: Response, next: NextFunction) => {
    const t = req.tenant;                                          // built by tenant middleware
    if (!t.permissions.has(permission)) throw new ForbiddenError('FORBIDDEN');
    if (opts.scope === 'teacherClass')  await assertTeacherAssigned(t, req.params[opts.param!]);
    if (opts.scope === 'parentChild')   await assertParentLinked(t, req.params[opts.param!]);
    if (opts.scope === 'self')          assertSelf(t, req);
    next();
```

```
  };
// Routes without an authorize() or an explicit "public" marker fail the route-sweep test (SEC-REQ-003).
```

10.8 Error handling

Single error envelope

```
HTTP 422
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Some fields are invalid.",
    "request_id": "req_01J9Z3K8M4Q7",
    "details": [ { "path": "scores[3].score", "message": "Must be <= 100" } ]
  }
}
```

HTTP	Code (examples)	Use
400	BAD_REQUEST	Malformed JSON or parameters
401	UNAUTHENTICATED, SESSION_EXPIRED, DEVICE_REVOKED	Missing/invalid/expired credential
402/403	SUBSCRIPTION_FEATURE_UNAVAILABLE, FORBIDDEN	Entitlement or permission refused
404	NOT_FOUND	Missing **or** other-school resource (same response)
409	CONFLICT, VERSION_MISMATCH	Unique or optimistic-lock conflicts
413 / 422 / 429	PAYLOAD_TOO_LARGE, VALIDATION_ERROR, RATE_LIMITED	Limits and validation (with `Retry-After` for 429)
500 / 503	INTERNAL, DEPENDENCY_UNAVAILABLE	Unexpected / temporary failures (device receives RETRY_LATER semantics)

10.9 Structured logging

- JSON logs with `level, time, request_id, school_id, actor_type, actor_id, route, status, duration_ms`; no request bodies by default.
- Redaction list: `authorization`, `cookie`, device secrets, passwords, tokens; card UIDs logged as a truncated hash.
- Business events (attendance outcome, device state change, outbox transition) are logged with stable event names for alerts.

10.10 Rate limiting

Scope	Default limit (Proposed, tune after load tests)	Notes
Per IP, global	300 requests / minute	Protects against scraping and floods
Login / recovery (Supabase Auth + API wrappers)	10 / 15 minutes per IP + account	Brute-force protection
Authenticated user	120 requests / minute	Higher for exports through async jobs
Device	60 requests / minute per device	Batch endpoint is the high-volume path; response includes `retry_after_seconds`
Bulk/export endpoints	5 / minute per user	Large operations are asynchronous jobs

10.11 API conventions

- Base path `/api/v1`; JSON; ISO-8601 UTC instants; IDs are UUIDs; enumerations are upper-snake strings.

- Pagination: `?page=&pageSize=` (default 25, max 100) returning `{ data, page, pageSize, total }`; cursor variant `?cursor=&limit=` for feeds.
- Idempotency for client mutations that may be retried (bulk uploads, payments webhooks): `Idempotency-Key` header stored in `api_idempotency_keys`.
- Health: `GET /healthz` (liveness), `GET /readyz` (database and migrations reachable); no authentication, no secrets.
- Graceful shutdown: on SIGTERM stop accepting connections, finish in-flight requests (≤ 25 s), close the pool.

API requirements

ID	Requirement	Status
API-REQ-001	All client and device traffic SHALL use HTTPS (TLS 1.2 or later); plain HTTP SHALL be redirected or refused.	**Confirmed**
API-REQ-002	Every endpoint SHALL validate request bodies, params and query strings with Zod schemas. Client/device-supplied school identifiers SHALL be ignored or rejected.	**Proposed**
API-REQ-003	Errors SHALL use one envelope containing a stable machine code, a human message and a request_id; internal details SHALL NOT leak.	**Proposed**
API-REQ-004	Rate limits SHALL apply per IP, per authenticated user and per device, with tighter limits on authentication endpoints.	**Proposed**
API-REQ-005	Payload limits SHALL be enforced: 1 MB default JSON body, 256 KB and 100 events for device batches.	**Proposed**
API-REQ-006	List endpoints SHALL paginate (default 25, max 100); very large or append-only lists SHALL use cursor pagination.	**Proposed**

11. Database Architecture

11.1 Relational design principles

- **Normalise first.** Student, parent, class and school attributes live in one place; child tables reference them by key (DB-REQ-010).
- **Snapshots only when history demands it.** `attendance_events.location_type` and `daily_attendance.class_id` are snapshots because later edits must not rewrite history.
- **Constraints are the last line of correctness.** Unique constraints, CHECKs and composite FKs carry rules that concurrent code could otherwise violate.
- **Raw facts are immutable.** `attendance_events` rows are never edited or deleted; derived state (`daily_attendance`) can be recomputed.

11.2 Entity relationship overview

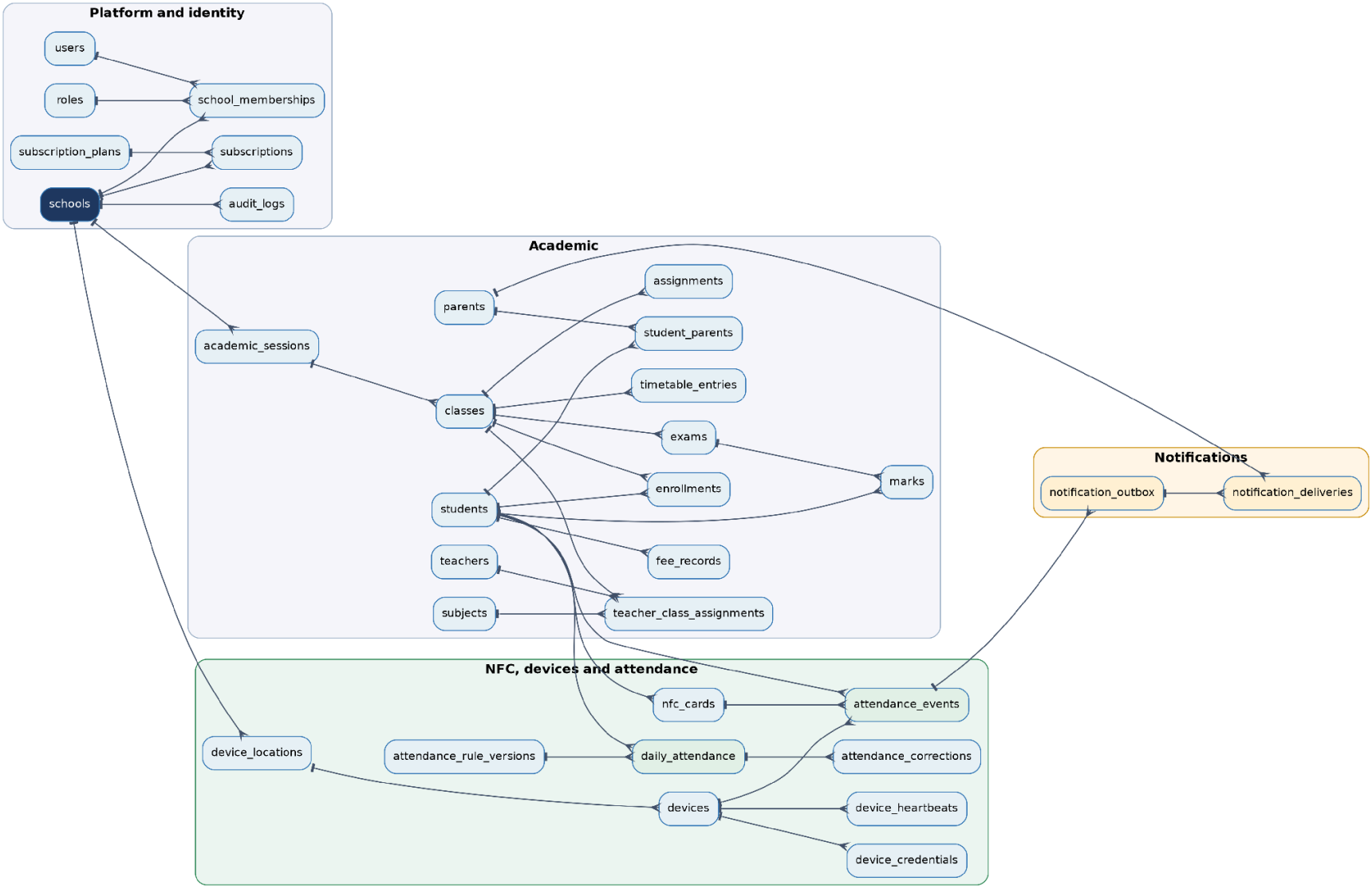


Figure 3 — Database entity relationships (overview)

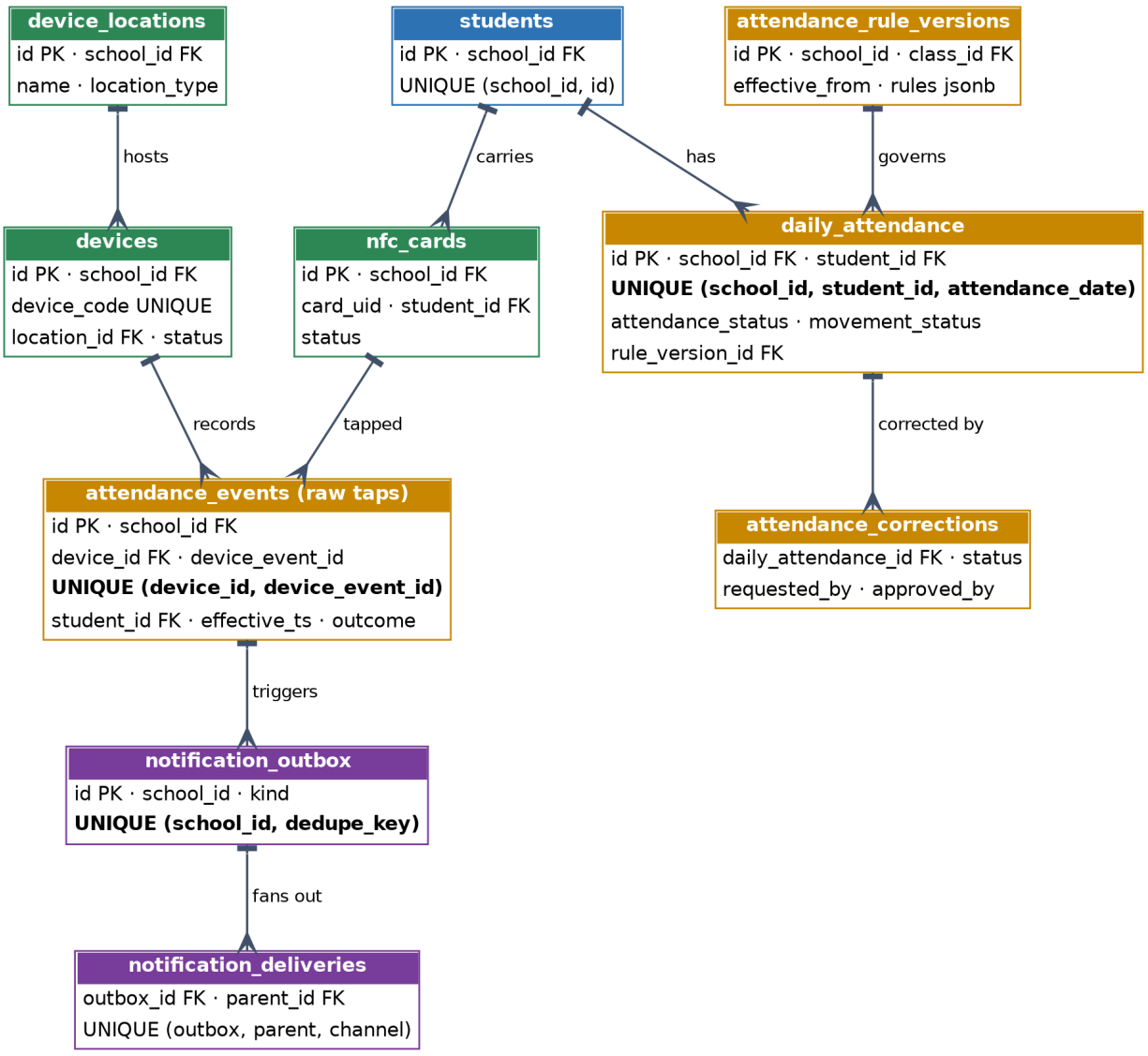


Figure 4 — Attendance, device and notification entities (key columns)

11.3 Keys, types and naming

Topic	Convention
Primary keys	`id uuid PRIMARY KEY DEFAULT gen_random_uuid()` (UUIDv4). Native UUIDv7 requires a newer PostgreSQL; evaluate later for index locality (Validate).
Foreign keys	Always declared; composite `(school_id, x_id)` for school-owned references; `ON DELETE RESTRICT` by default.
Time	`timestamptz` for instants (UTC); `date` for school-local calendar dates; `time` for rule windows.
Enumerations	`text` + `CHECK (col IN (...))` (simpler migrations than native enums).
Money	`numeric(12,2)` plus ISO currency code.
Naming	snake_case, plural table names, `<table>_<cols>_idx` / `_uk` / `_fk` constraint names.
Audit columns	`created_at`, `updated_at` (trigger), `created_by` where user-driven.
Deletion	Soft retire via `status` for academic and attendance entities; hard delete only in retention/offboarding jobs.

11.4 Transactions and isolation

The default isolation level is **READ COMMITTED**. Correctness under concurrency does not depend on SERIALIZABLE; it depends on (a) unique constraints, (b) `INSERT … ON CONFLICT`, and (c) explicit row locks (`SELECT … FOR UPDATE`) on the aggregate being modified (the student-day row). This keeps throughput high and behaviour predictable. SERIALIZABLE with retry is reserved for rare cross-row invariants if they appear.

11.5 Unique constraints that carry business rules

Constraint	Rule enforced
`daily_attendance UNIQUE (school_id, student_id, attendance_date)`	At most one daily attendance record per student per school day.
`attendance_events UNIQUE (device_id, device_event_id)`	Idempotency: one stored event per device event id.
`nfc_cards UNIQUE (school_id, card_uid) WHERE status <> 'RETIRED'`	A physical card belongs to one non-retired record per school.
`nfc_cards UNIQUE (school_id, student_id) WHERE status = 'ACTIVE'`	One active card per student.
`student_parents UNIQUE (school_id, student_id, parent_id)`	No duplicate links.
`marks UNIQUE (school_id, exam_id, student_id, subject_id)`	One mark per student per subject per exam.
`notification_outbox UNIQUE (school_id, dedupe_key)`	One notification per meaningful state change.
`devices UNIQUE (device_code)` and `UNIQUE (school_id, id)`	Globally unique device identity; composite key target.
`subscriptions UNIQUE (school_id) WHERE status NOT IN ('CANCELLED','EXPIRED')`	One live subscription per school.

11.6 Indexing policy

- Index every foreign key used in joins and every filter used by a documented screen or job; each index is listed in Section 12 with its query.
- Prefer composite indexes beginning with `school_id` so a tenant's data is contiguous: `(school_id, student_id, attendance_date)`.
- Use partial indexes for hot subsets (`WHERE status = 'PENDING'` for outbox, `WHERE outcome <> 'OK'` for review queues).
- Verify with `EXPLAIN (ANALYZE, BUFFERS)` on seeded realistic data (≥ 1 M events) before adding or keeping an index (DB-REQ-005).

11.7 Row-Level Security

RLS is enabled and forced on all school-owned tables (Section 8). Policies are generated by a migration helper so that the same template applies everywhere; a CI test lists tables lacking policies. Platform tables (`schools`, `subscription_plans`, `platform_staff`) use separate policies for `app_platform`.

11.8 Migrations

- SQL files in `db/migrations/NNNN_description.sql`, applied in order by `supabase db push` in CI.
- Forward-only. Destructive changes follow **expand → migrate → contract** across at least two releases.
- Each migration is reviewed for locks (`CREATE INDEX CONCURRENTLY` on large tables, `NOT VALID` constraints then `VALIDATE`).

- Schema assertions run after migrations in CI (RLS, composite FKs, key unique constraints).
- Seed scripts create two demo schools for development and cross-school tests; production is never seeded with demo data.

11.9 Connection pooling

Connection	Use	Notes
Pooler, transaction mode	API and worker queries	Do not rely on session state, advisory session locks or `LISTEN`; set tenant context with transaction-local `set_config`. Disable or manage prepared statements per driver guidance.
Direct / session connection	Migrations, admin tasks, optional worker `LISTEN/NOTIFY`	Few connections; restricted network access.

Pool size per API instance starts small (Proposed: 10–15) and is tuned against the plan's connection limit with the headroom defined in PERF-REQ-008 (Validate against current Supabase limits).

11.10 Backup, restore and point-in-time recovery

- Automated daily backups; PITR enabled where the plan provides it (DB-REQ-008, Validate). Without PITR the achievable RPO is the backup interval, which does not meet the 5-minute target — see Section 24.
- Before every risky migration take an on-demand backup/snapshot.
- Quarterly restore drill into a scratch project with integrity checks (TEST-REQ-026).

11.11 Audit history

- `audit_logs` is append-only (INSERT/SELECT only for application roles) and records actor, actor type, school, action, entity, before/after JSON, request id and IP.
- Triggers on the highest-risk tables (`attendance_corrections`, `school_memberships`, `student_parents`, `nfc_cards`, `devices`, `subscriptions`) capture changes even if the application layer misses them.
- Raw attendance events are themselves an immutable audit trail of taps.

11.12 Database monitoring

- `pg_stat_statements` for slow/high-volume queries; slow query logging above 500 ms (Proposed).
- Dashboards and alerts: connections used vs limit, CPU, memory, disk growth, replication/backup status, lock waits, long transactions, table/index bloat.
- Weekly review of the top-10 queries by total time during the pilot.

11.13 Retention and growth

At ≈ 0.6–1.3 M raw events per school-year (Section 22) plain tables with the documented indexes are adequate. Table partitioning (by month on `effective_ts`) is **not** introduced in V1; note that a unique constraint on a partitioned table must include the partition key, which conflicts with the `(device_id, device_event_id)` idempotency key. Revisit only if measurements show index/VACUUM pain, and then archive old months to cold storage instead.

Database requirements

ID	Requirement	Status
DB-REQ-001	Primary keys SHALL be UUIDs generated by gen_random_uuid(); instants SHALL be timestamptz in UTC; enumerations SHALL be text with CHECK constraints.	**Proposed**

ID	Requirement	Status
DB-REQ-002	Foreign keys SHALL be declared for every relationship; ON DELETE defaults to RESTRICT; academic and attendance data SHALL be retired by status, not hard-deleted.	**Proposed**
DB-REQ-003	daily_attendance SHALL carry UNIQUE (school_id, student_id, attendance_date).	**Confirmed**
DB-REQ-004	attendance_events SHALL carry UNIQUE (device_id, device_event_id) as the idempotency key.	**Confirmed**
DB-REQ-005	Every index SHALL be justified by a documented query; new indexes SHALL be verified with EXPLAIN (ANALYZE) on realistic data volumes.	**Proposed**
DB-REQ-006	Schema changes SHALL be forward-only, version-controlled SQL migrations applied by CI, using expand/contract for breaking changes.	**Proposed**
DB-REQ-007	The API SHALL connect through the pooler using a least-privilege role (not superuser, not BYPASSRLS).	**Confirmed**
DB-REQ-008	Automated backups and, where the plan supports it, point-in-time recovery SHALL be enabled and verified by restore drills.	**Validate**
DB-REQ-009	audit_logs SHALL be append-only: application roles SHALL hold INSERT and SELECT only.	**Confirmed**
DB-REQ-010	Child tables SHALL not duplicate student or school attributes; snapshots are allowed only where historical accuracy is required (e.g. location_type at tap time).	**Proposed**
DB-REQ-011	attendance_events rows SHALL be immutable except for explicitly listed processing-state columns; DELETE SHALL be revoked.	**Proposed**
DB-REQ-012	A student SHALL have at most one ACTIVE NFC card, and a card UID SHALL belong to at most one non-retired card record per school (partial unique indexes).	**Confirmed**

12. Complete Database Schema

Tables are grouped by domain. Unless stated, every school-owned table has `school_id NOT NULL` with a composite unique key `(school_id, id)` so that children can use composite foreign keys (Section 8). All identifiers are UUIDs. All timestamps are `timestamptz`. "RLS" means the standard tenant policy from Section 8 is applied. Index lists show the query each index serves.

> **Reading the tables**
> Only the columns that matter for behaviour and integrity are listed; engineering may add descriptive columns (phone, address, photo path) without changing relationships. Foreign keys written as "→ x(school_id,id)" mean a composite foreign key `(school_id, <col>) → x(school_id, id)`.

12.1 Platform and identity

12.1.1 schools

One row per partner school (tenant). Platform table; not itself RLS-scoped by school_id, readable by members for their own school only.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_code	text	NOT NULL UNIQUE (e.g. SCH001)
name	text	NOT NULL
timezone	text	NOT NULL default 'Asia/Kolkata' (IANA name)
status	text	CHECK IN (ONBOARDING, ACTIVE, SUSPENDED, TERMINATED)
organization_id	uuid	NULL, FK → organizations(id) (multi-campus)

Column	Type	Constraints / notes
settings	jsonb	NOT NULL default '{}' (locale, notification defaults)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Indexes: UNIQUE(school_code); (organization_id).

12.1.2 organizations

Optional parent for multi-campus Enterprise customers.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
name	text	NOT NULL
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

12.1.3 users

Application profile for every authenticated person. `id` equals `auth.users.id` from Supabase Auth, linking identity to application data.

Column	Type	Constraints / notes
id	uuid	PK, FK → auth.users(id) ON DELETE RESTRICT
email	citext	NOT NULL UNIQUE
full_name	text	NOT NULL
phone	text	NULL
status	text	CHECK IN (INVITED, ACTIVE, DISABLED)
last_login_at	timestamptz	NULL
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Indexes: UNIQUE(email). Not school-owned: a person may belong to several schools via memberships.

12.1.4 platform_staff

Company staff (Super Admins). Separate from school memberships so platform access never appears as a school role.

Column	Type	Constraints / notes
user_id	uuid	PK, FK → users(id)
platform_role	text	CHECK IN (SUPER_ADMIN, SUPPORT, BILLING)
mfa_required	boolean	NOT NULL default true
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

12.1.5 roles

System-defined roles. Seeded: school_admin, principal, teacher, parent, student.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
code	text	NOT NULL UNIQUE
name	text	NOT NULL
description	text	NULL

12.1.6 permissions

Stable permission codes checked by the API.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
code	text	NOT NULL UNIQUE (e.g. marks.write)
description	text	NULL

12.1.7 role_permissions

Many-to-many mapping of roles to permissions.

Column	Type	Constraints / notes
role_id	uuid	FK → roles(id)
permission_id	uuid	FK → permissions(id)
		PRIMARY KEY (role_id, permission_id)

12.1.8 school_memberships

Links a user to a school with exactly one role. This is how an authentication identity becomes a school member.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
user_id	uuid	NOT NULL, FK → users(id)
role_id	uuid	NOT NULL, FK → roles(id)
status	text	CHECK IN (INVITED, ACTIVE, SUSPENDED, REMOVED)
invited_by	uuid	NULL, FK → users(id)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, user_id). Indexes: (user_id) for login lookup; (school_id, role_id, status) for staff lists. RLS.

12.1.9 support_access_grants

Time-boxed, reasoned permission for platform staff to read a school's data.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
staff_user_id	uuid	NOT NULL, FK → platform_staff(user_id)
reason	text	NOT NULL
ticket_ref	text	NOT NULL

Column	Type	Constraints / notes
scope	jsonb	NOT NULL (tables / features)
approved_by	uuid	NULL, FK → users(id) (school admin approval if contract requires)
starts_at	timestamptz	NOT NULL
expires_at	timestamptz	NOT NULL, CHECK (expires_at <= starts_at + interval '24 hours')
revoked_at	timestamptz	NULL

Indexes: (school_id, expires_at); (staff_user_id, expires_at).

12.2 Academic structure

12.2.1 academic_sessions

A school year/term container.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
name	text	NOT NULL (e.g. 2026-27)
starts_on	date	NOT NULL
ends_on	date	NOT NULL CHECK (ends_on > starts_on)
is_current	boolean	NOT NULL default false
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, id); UNIQUE(school_id, name); partial UNIQUE(school_id) WHERE is_current. Index: (school_id, starts_on).

12.2.2 classes

A class/section in an academic session (e.g. 8-A).

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
academic_session_id	uuid	NOT NULL → academic_sessions(school_id,id)
grade	text	NOT NULL
section	text	NOT NULL
class_teacher_id	uuid	NULL → teachers(school_id,id)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, id); UNIQUE(school_id, academic_session_id, grade, section). Index: (school_id, academic_session_id).

12.2.3 subjects

Subjects taught at the school.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)

Column	Type	Constraints / notes
code	text	NOT NULL
name	text	NOT NULL

UNIQUE(school_id, code); UNIQUE(school_id, id).

12.2.4 students

The student master record. Contains identity only; class membership is in enrollments.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
student_code	text	NOT NULL (e.g. STU1001)
user_id	uuid	NULL, FK → users(id) (optional login)
first_name	text	NOT NULL
last_name	text	NULL
date_of_birth	date	NULL
status	text	CHECK IN (ACTIVE, TRANSFERRED, GRADUATED, WITHDRAWN)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, id); UNIQUE(school_id, student_code). Indexes: (school_id, status); (school_id, last_name, first_name) for search; (user_id).

12.2.5 enrollments

Which class a student is in during an academic session (history preserved).

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
student_id	uuid	NOT NULL → students(school_id,id)
class_id	uuid	NOT NULL → classes(school_id,id)
academic_session_id	uuid	NOT NULL → academic_sessions(school_id,id)
roll_number	text	NULL
starts_on	date	NOT NULL
ends_on	date	NULL
status	text	CHECK IN (ACTIVE, ENDED)

Constraints: partial UNIQUE(school_id, student_id, academic_session_id) WHERE status = 'ACTIVE'. Indexes: (school_id, class_id, status) for class rosters; (school_id, student_id).

12.2.6 parents

Parent/guardian record, optionally linked to a login.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
user_id	uuid	NULL, FK → users(id)

Column	Type	Constraints / notes
full_name	text	NOT NULL
email	citext	NULL
phone	text	NULL
status	text	CHECK IN (ACTIVE, INACTIVE)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, id). Indexes: (school_id, user_id); (school_id, phone).

12.2.7 student_parents

Authorisation link: which parents may see which students. Source of truth for parent-child access.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
student_id	uuid	NOT NULL → students(school_id,id)
parent_id	uuid	NOT NULL → parents(school_id,id)
relationship	text	CHECK IN (MOTHER, FATHER, GUARDIAN, OTHER)
is_primary_contact	boolean	NOT NULL default false
can_receive_alerts	boolean	NOT NULL default true
status	text	CHECK IN (ACTIVE, REVOKED)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, student_id, parent_id). Indexes: (school_id, parent_id, status) — hot path for parent portal; (school_id, student_id, status) — hot path for notification fan-out.

12.2.8 teachers

Teacher/staff record.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
user_id	uuid	NULL, FK → users(id)
staff_code	text	NOT NULL
full_name	text	NOT NULL
status	text	CHECK IN (ACTIVE, INACTIVE)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, id); UNIQUE(school_id, staff_code).

12.2.9 teacher_class_assignments

Authorisation link: which teacher teaches which subject in which class.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()

Column	Type	Constraints / notes
school_id	uuid	NOT NULL, FK → schools(id)
teacher_id	uuid	NOT NULL → teachers(school_id,id)
class_id	uuid	NOT NULL → classes(school_id,id)
subject_id	uuid	NULL → subjects(school_id,id) (NULL = class teacher)
starts_on	date	NOT NULL
ends_on	date	NULL

Constraints: UNIQUE(school_id, teacher_id, class_id, subject_id). Indexes: (school_id, teacher_id) for "my classes"; (school_id, class_id).

12.2.10 exams

An examination for a class in a session.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
class_id	uuid	NOT NULL → classes(school_id,id)
name	text	NOT NULL
exam_type	text	CHECK IN (UNIT, MIDTERM, FINAL, PRACTICAL)
starts_on	date	NULL
max_score	numeric(6,2)	NOT NULL CHECK (max_score > 0)
status	text	CHECK IN (DRAFT, OPEN, LOCKED, PUBLISHED)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, id). Index: (school_id, class_id, status).

12.2.11 marks

A student's score in a subject for an exam. Locked exams reject writes in the service layer.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
exam_id	uuid	NOT NULL → exams(school_id,id)
student_id	uuid	NOT NULL → students(school_id,id)
subject_id	uuid	NOT NULL → subjects(school_id,id)
score	numeric(6,2)	NOT NULL CHECK (score >= 0)
remarks	text	NULL
entered_by	uuid	NOT NULL, FK → users(id)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, exam_id, student_id, subject_id). Indexes: (school_id, student_id, exam_id) for report cards; (school_id, exam_id, subject_id).

12.2.12 assignments

Homework / study materials.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
class_id	uuid	NOT NULL → classes(school_id,id)
subject_id	uuid	NULL → subjects(school_id,id)
kind	text	CHECK IN (HOMEWORK, MATERIAL)
title	text	NOT NULL
body	text	NULL
due_at	timestamptz	NULL
storage_path	text	NULL (private bucket)
created_by	uuid	NOT NULL, FK → users(id)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Index: (school_id, class_id, due_at DESC).

12.2.13 timetable_entries

Weekly timetable slots.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
class_id	uuid	NOT NULL → classes(school_id,id)
subject_id	uuid	NOT NULL → subjects(school_id,id)
teacher_id	uuid	NOT NULL → teachers(school_id,id)
weekday	smallint	CHECK (weekday BETWEEN 1 AND 7)
starts_at	time	NOT NULL
ends_at	time	NOT NULL CHECK (ends_at > starts_at)
room	text	NULL

Constraints: no teacher double-booking via EXCLUDE on (school_id, teacher_id, weekday, timerange) (optional). Index: (school_id, class_id, weekday).

12.2.14 school_holidays

Non-school days and exam-day schedule markers used by the rule engine.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
on_date	date	NOT NULL
kind	text	CHECK IN (HOLIDAY, WEEKEND_OVERRIDE, EXAM_DAY, SHORT_DAY)
class_id	uuid	NULL → classes(school_id,id) (NULL = whole school)
label	text	NULL

Constraints: UNIQUE(school_id, on_date, kind, class_id) NULLS NOT DISTINCT. Weekly offs (e.g. Sunday) are in rule settings.

12.3 NFC cards, devices and attendance

12.3.1 nfc_cards

A physical card assigned to a student. Replacement cards create new rows; old ones become RETIRED/LOST.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
card_uid	text	NOT NULL, uppercase hex (e.g. 04A2B3C4D5E680)
student_id	uuid	NOT NULL → students(school_id,id)
status	text	CHECK IN (ACTIVE, BLOCKED, LOST, RETIRED)
issued_at	timestamptz	NOT NULL default now()
blocked_at	timestamptz	NULL
blocked_reason	text	NULL
issued_by	uuid	NOT NULL, FK → users(id)

Constraints: UNIQUE(school_id, id); partial UNIQUE(school_id, card_uid) WHERE status <> 'RETIRED'; partial UNIQUE(school_id, student_id) WHERE status = 'ACTIVE'. Index: (school_id, card_uid) is the tap-time lookup (covered by the partial unique index).

12.3.2 device_locations

Physical places that have readers, with a rule-relevant type.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
name	text	NOT NULL (Main Gate)
location_type	text	CHECK IN (ENTRY, EXIT, BIDIRECTIONAL, CLASSROOM)
class_id	uuid	NULL → classes(school_id,id) (for CLASSROOM)
campus_label	text	NULL
is_active	boolean	NOT NULL default true

Constraints: UNIQUE(school_id, id); UNIQUE(school_id, name). Several devices may share one location (e.g. two readers on a wide gate).

12.3.3 devices

Registered NFC reader units. Each belongs to exactly one school and has a globally unique code.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
device_code	text	NOT NULL UNIQUE (e.g. DEV001)
name	text	NOT NULL
hardware_serial	text	NULL UNIQUE
location_id	uuid	NULL → device_locations(school_id,id)
status	text	CHECK IN (PROVISIONED, ACTIVE, DISABLED, REVOKED)
claim_code_hash	bytea	NULL (cleared after claim)
firmware_version	text	NULL
config_version	integer	NOT NULL default 1
last_seen_at	timestamptz	NULL (last heartbeat or batch)

Column	Type	Constraints / notes
last_sync_at	timestamptz	NULL (last successful batch)
pending_event_count	integer	NOT NULL default 0
reader_status	text	CHECK IN (OK, DEGRADED, FAIL, UNKNOWN)
rtc_status	text	CHECK IN (OK, DRIFT, LOST_POWER, UNKNOWN)
wifi_rssi_dbm	smallint	NULL
storage_used_pct	smallint	NULL
revoked_at	timestamptz	NULL
revoked_reason	text	NULL
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: UNIQUE(school_id, id). Indexes: (school_id, status); (last_seen_at) WHERE status = 'ACTIVE' for the offline watchdog. "Online/offline" is derived: ACTIVE and last_seen_at within the threshold.

12.3.4 device_credentials

Per-device secrets, stored only as hashes. Two ACTIVE rows allowed during rotation.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
device_id	uuid	NOT NULL → devices(school_id,id)
secret_hash	bytea	NOT NULL (SHA-256 of 256-bit random secret)
status	text	CHECK IN (ACTIVE, ROTATING_OUT, REVOKED)
created_at	timestamptz	NOT NULL default now()
expires_at	timestamptz	NULL
last_used_at	timestamptz	NULL

Index: UNIQUE(secret_hash); (device_id, status).

12.3.5 device_configurations

Versioned configuration delivered to devices (no secrets).

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
device_id	uuid	NULL → devices(school_id,id) (NULL = school default)
version	integer	NOT NULL
config	jsonb	NOT NULL (heartbeat interval, batch size, debounce, LED/buzzer profile, time-sync interval)
created_by	uuid	NOT NULL, FK → users(id)
created_at	timestamptz	NOT NULL default now()

UNIQUE(school_id, device_id, version).

12.3.6 device_heartbeats

Sampled history of device health (the latest values also live on `devices`).

Column	Type	Constraints / notes
id	bigint	PK identity
school_id	uuid	NOT NULL, FK → schools(id)
device_id	uuid	NOT NULL → devices(school_id,id)
received_at	timestamptz	NOT NULL default now()
device_time	timestamptz	NOT NULL
skew_ms	integer	NOT NULL (device_time − server time)
boot_id	uuid	NOT NULL
uptime_ms	bigint	NOT NULL
firmware_version	text	NOT NULL
pending_events	integer	NOT NULL
storage_used_pct	smallint	NOT NULL
wifi_rssi_dbm	smallint	NULL
reader_status	text	NOT NULL
rtc_status	text	NOT NULL

Index: (device_id, received_at DESC). Inserted at most every 5 minutes or on state change; retained 30 days (Proposed).

12.3.7 device_logs

Diagnostic records sent by firmware (queue corruption, reader failure, OTA failure, reboot reasons).

Column	Type	Constraints / notes
id	bigint	PK identity
school_id	uuid	NOT NULL, FK → schools(id)
device_id	uuid	NOT NULL → devices(school_id,id)
device_ts	timestamptz	NULL
received_at	timestamptz	NOT NULL default now()
level	text	CHECK IN (INFO, WARN, ERROR)
code	text	NOT NULL (e.g. QUEUE_CORRUPTION, READER_FAIL, OTA_FAILED)
detail	jsonb	NULL

Index: (device_id, received_at DESC); (school_id, code, received_at DESC). Retained 90 days (Proposed).

12.3.8 firmware_releases and device_ota_status

Signed firmware inventory and per-device update state (added tables, justified by secure OTA).

Column	Type	Constraints / notes
firmware_releases.version	text	PK; channel, sha256, signature, size, min_version, storage_path, created_at
device_ota_status.device_id	uuid	FK; target_version, state IN (OFFERED, DOWNLOADING, VERIFYING, INSTALLED, FAILED, ROLLED_BACK), detail, updated_at

12.3.9 attendance_rule_versions

Immutable, versioned attendance configuration per school (and optionally per class or day type). A change creates a new row.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()

Column	Type	Constraints / notes
school_id	uuid	NOT NULL, FK → schools(id)
class_id	uuid	NULL → classes(school_id,id) (NULL = school-wide)
day_type	text	CHECK IN (REGULAR, EXAM, SHORT_DAY)
version_no	integer	NOT NULL
effective_from	date	NOT NULL
effective_to	date	NULL (end-exclusive)
rules	jsonb	NOT NULL (validated by Zod and a CHECK on required keys)
created_by	uuid	NOT NULL, FK → users(id)
created_at	timestamptz	NOT NULL default now()

Constraints: UNIQUE(school_id, id); EXCLUDE USING gist (school_id =, coalesce(class_id, nil-uuid) =, day_type =, daterange(effective_from, effective_to, '[)') &&) prevents overlapping versions. Index: (school_id, day_type, effective_from DESC).

12.3.10 attendance_events

Raw, immutable record of every tap (and every rejected tap). The audit trail and the input to the daily projection. Full DDL below.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
device_id / device_event_id	uuid / uuid	idempotency key UNIQUE(device_id, device_event_id)
sequence_number, boot_id	bigint, uuid	device ordering and boot session
card_uid	text	as read
nfc_card_id, student_id	uuid, uuid	NULL when unknown card
location_id, location_type	uuid, text	location at tap time (snapshot)
event_ts / effective_ts	timestamptz	device-reported / trusted time used by rules
time_source, timestamp_trust	text	RTC_SYNCED, RTC_UNSYNCED, NTP, MONOTONIC; TRUSTED, RECONSTRUCTED, SUSPECT
status, outcome	text	RECORDED / REJECTED; outcome code
applied_to_daily	boolean	whether it changed daily state

12.3.11 daily_attendance

One row per student per school day: the consolidated attendance and movement state. Full DDL below.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
student_id, attendance_date	uuid, date	UNIQUE(school_id, student_id, attendance_date)
class_id, academic_session_id	uuid	snapshot at day-open
attendance_status	text	PENDING, PRESENT, LATE, HALF_DAY, ABSENT, EXCUSED
movement_status	text	NOT_ARRIVED, ARRIVED, LEFT
arrival_classification / departure_classification	text	ON_TIME, LATE, AFTER_CUTOFF / NORMAL, HALF_DAY, EARLY
first_arrival_event_id, first_arrival_at	uuid, timestamptz	immutable once set (except by replay or correction)

Column	Type	Constraints / notes
last_departure_event_id, last_departure_at	uuid, timestamptz	
rule_version_id	uuid	rule version used
needs_review	boolean	quarantined or ambiguous events
source, version	text, integer	NFC / MANUAL / SYSTEM; optimistic version

12.3.12 attendance_corrections

Manual correction workflow. The raw events are never edited; an approved correction updates the daily projection and is audited.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
daily_attendance_id	uuid	NOT NULL → daily_attendance(school_id,id)
requested_status	text	NOT NULL (attendance status to set)
reason	text	NOT NULL
status	text	CHECK IN (REQUESTED, APPROVED, REJECTED, APPLIED, CANCELLED)
requested_by	uuid	NOT NULL, FK → users(id)
approved_by	uuid	NULL, FK → users(id)
decided_at	timestamptz	NULL
applied_at	timestamptz	NULL
requested_at	timestamptz	NOT NULL default now()

Indexes: (school_id, status, requested_at); (school_id, daily_attendance_id).

12.4 Notifications, communication and fees

12.4.1 notification_outbox

Durable queue of notification jobs written in the same transaction as the cause. Full DDL below.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
kind	text	STUDENT_ARRIVED, STUDENT_DEPARTED, STUDENT_ABSENT, ATTENDANCE_CORRECTED, ANNOUNCEMENT, FEE_REMINDER, DEVICE_ALERT
student_id	uuid	NULL → students(school_id,id)
dedupe_key	text	NOT NULL; UNIQUE(school_id, dedupe_key)
payload	jsonb	NOT NULL (template variables, no secrets)
source_event_id	uuid	NULL → attendance_events(school_id,id)
status	text	PENDING, PROCESSING, COMPLETED, DEAD
attempts	integer	NOT NULL default 0
next_attempt_at	timestamptz	NOT NULL default now()
locked_until	timestamptz	NULL
created_at / completed_at	timestamptz	

Index: (next_attempt_at) WHERE status IN ('PENDING','PROCESSING') — the worker claim query.

12.4.2 notification_deliveries

One attempt target per recipient and channel; the audit record of delivery.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
outbox_id	uuid	NOT NULL → notification_outbox(school_id,id)
parent_id	uuid	NULL → parents(school_id,id)
channel	text	CHECK IN (PUSH, EMAIL, SMS)
address_hash	text	NULL (token/email/phone reference, not plain text in logs)
status	text	PENDING, SENT, FAILED, SUPPRESSED, EXPIRED_STALE, DEAD
attempts	integer	NOT NULL default 0
next_attempt_at	timestamptz	NULL
provider_message_id	text	NULL
last_error	text	NULL
sent_at	timestamptz	NULL

Constraints: UNIQUE(school_id, outbox_id, parent_id, channel). Indexes: (next_attempt_at) WHERE status = 'FAILED'; (school_id, parent_id, created_at DESC).

12.4.3 notification_preferences and push_tokens

Per-parent channel choices and FCM tokens (added tables).

Column	Type	Constraints / notes
notification_preferences	table	school_id, parent_id, kind, channel, enabled; UNIQUE(school_id, parent_id, kind, channel); quiet-hours columns
push_tokens	table	school_id, user_id, token_hash, platform, last_seen_at, is_active; UNIQUE(token_hash)

12.4.4 announcements

Notices to the school or a class.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
audience	text	CHECK IN (SCHOOL, CLASS, PARENTS, STAFF)
class_id	uuid	NULL → classes(school_id,id)
title	text	NOT NULL
body	text	NOT NULL (sanitised Markdown)
published_at	timestamptz	NULL
created_by	uuid	NOT NULL, FK → users(id)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Index: (school_id, published_at DESC).

12.4.5 fee_records

Fee charges and payment status per student.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
student_id	uuid	NOT NULL → students(school_id,id)
academic_session_id	uuid	NOT NULL → academic_sessions(school_id,id)
fee_type	text	NOT NULL
amount_due	numeric(12,2)	NOT NULL CHECK (amount_due >= 0)
amount_paid	numeric(12,2)	NOT NULL default 0
currency	char(3)	NOT NULL default 'INR'
due_on	date	NOT NULL
status	text	CHECK IN (UNPAID, PARTIAL, PAID, WAIVED)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Index: (school_id, student_id, due_on); (school_id, status, due_on).

12.5 Subscriptions, audit and utilities

12.5.1 subscription_plans

Platform-defined packages. Prices are not stored as design facts here; billing amounts live in the billing provider.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
code	text	NOT NULL UNIQUE (BASIC, PROFESSIONAL, ENTERPRISE)
name	text	NOT NULL
features	jsonb	NOT NULL (feature flags, e.g. nfc_attendance: true)
limits	jsonb	NOT NULL (max_students, max_devices, max_campuses, report_retention_days)
is_active	boolean	NOT NULL default true

12.5.2 subscriptions

A school's plan and lifecycle state.

Column	Type	Constraints / notes
id	uuid	PK, default gen_random_uuid()
school_id	uuid	NOT NULL, FK → schools(id)
plan_id	uuid	NOT NULL, FK → subscription_plans(id)
status	text	TRIALING, ACTIVE, PAST_DUE, SUSPENDED_READONLY, SUSPENDED, CANCELLED, EXPIRED
current_period_end	timestamptz	NOT NULL
grace_ends_at	timestamptz	NULL
limit_overrides	jsonb	NULL (contract-specific)
billing_ref	text	NULL (external provider id)
created_at	timestamptz	NOT NULL default now()
updated_at	timestamptz	NOT NULL default now() (trigger)

Constraints: partial UNIQUE(school_id) WHERE status NOT IN ('CANCELLED','EXPIRED'). Plus `subscription_events` (history of state changes).

12.5.3 audit_logs

Append-only audit trail.

Column	Type	Constraints / notes
id	bigint	PK identity
school_id	uuid	NULL (platform events), FK → schools(id)
occurred_at	timestamptz	NOT NULL default now()
actor_type	text	USER, DEVICE, SYSTEM, SUPPORT
actor_id	uuid	NULL
support_grant_id	uuid	NULL → support_access_grants(id)
action	text	NOT NULL (e.g. attendance.correction.approve)
entity_type, entity_id	text, uuid	
before, after	jsonb	NULL
request_id, ip, user_agent	text, inet, text	NULL

Indexes: (school_id, occurred_at DESC); (entity_type, entity_id); REVOKE UPDATE, DELETE.

12.5.4 Additional utility tables

Added where justified.

Column	Type	Constraints / notes
api_idempotency_keys	table	user_id, key, request_hash, response, expires_at — safe client retries
report_documents	table	school_id, kind, student_id/class_id, storage_path, status, requested_by — async generated reports
job_runs	table	job name, school scope, started/finished, status — visibility of scheduled jobs
subscription_events	table	history of subscription state changes

12.6 Core SQL definitions

These definitions are the reference for the attendance tables and are consistent with the examples in Sections 16 and 17.

devices and nfc_cards

```
CREATE TABLE devices (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id           uuid NOT NULL REFERENCES schools(id),
  device_code         text NOT NULL UNIQUE,                      -- 'DEV001'
  name                text NOT NULL,
  location_id         uuid,
  status              text NOT NULL DEFAULT 'PROVISIONED'
                      CHECK (status IN ('PROVISIONED','ACTIVE','DISABLED','REVOKED')),
  firmware_version    text,
  last_seen_at        timestamptz,
  last_sync_at        timestamptz,
  pending_event_count integer NOT NULL DEFAULT 0,
  reader_status       text NOT NULL DEFAULT 'UNKNOWN',
  rtc_status          text NOT NULL DEFAULT 'UNKNOWN',
  wifi_rssi_dbm       smallint,
  revoked_at          timestamptz,
  created_at          timestamptz NOT NULL DEFAULT now(),
  UNIQUE (school_id, id),
  FOREIGN KEY (school_id, location_id) REFERENCES device_locations (school_id, id)
);
```

```
CREATE TABLE nfc_cards (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id   uuid NOT NULL REFERENCES schools(id),
  card_uid    text NOT NULL,                                    -- '04A2B3C4D5E680'
  student_id  uuid NOT NULL,
  status      text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','BLOCKED','LOST','RETIRED')),
  issued_at   timestamptz NOT NULL DEFAULT now(),
  UNIQUE (school_id, id),
  FOREIGN KEY (school_id, student_id) REFERENCES students (school_id, id)
);
CREATE UNIQUE INDEX nfc_cards_uid_live_uk     ON nfc_cards (school_id, card_uid)   WHERE status <>
'RETIRED';
CREATE UNIQUE INDEX nfc_cards_student_live_uk ON nfc_cards (school_id, student_id) WHERE status = 'ACTIVE';
```

attendance_events (raw, immutable taps)

```
CREATE TABLE attendance_events (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id         uuid NOT NULL REFERENCES schools(id),
  device_id         uuid NOT NULL,
  device_event_id   uuid NOT NULL,                -- event_id generated by the device
  sequence_number   bigint NOT NULL,
  boot_id           uuid NOT NULL,
  card_uid          text NOT NULL,
  nfc_card_id       uuid,
  student_id        uuid,
  location_id       uuid NOT NULL,
  location_type     text NOT NULL CHECK (location_type IN ('ENTRY','EXIT','BIDIRECTIONAL','CLASSROOM')),
  event_ts          timestamptz NOT NULL,         -- tap time as reported by the device (UTC)
  effective_ts      timestamptz NOT NULL,         -- trusted time used by the rule engine
  time_source       text NOT NULL CHECK (time_source IN
('RTC_SYNCED','RTC_UNSYNCED','NTP','MONOTONIC','UNKNOWN')),
  timestamp_trust   text NOT NULL CHECK (timestamp_trust IN ('TRUSTED','RECONSTRUCTED','SUSPECT')),
  received_at       timestamptz NOT NULL DEFAULT now(),
  firmware_version  text NOT NULL,
  status            text NOT NULL CHECK (status IN ('RECORDED','REJECTED')),
  outcome           text NOT NULL,                -- e.g. ARRIVAL_ON_TIME, REJECTED_CARD_BLOCKED
  applied_to_daily  boolean NOT NULL DEFAULT false,
  result            jsonb NOT NULL DEFAULT '{}'::jsonb,   -- stored response for idempotent replays
  CONSTRAINT attendance_events_idem_uk      UNIQUE (device_id, device_event_id),
  CONSTRAINT attendance_events_school_id_uk UNIQUE (school_id, id),
  FOREIGN KEY (school_id, device_id)   REFERENCES devices (school_id, id),
  FOREIGN KEY (school_id, location_id) REFERENCES device_locations (school_id, id),
  FOREIGN KEY (school_id, nfc_card_id) REFERENCES nfc_cards (school_id, id),
  FOREIGN KEY (school_id, student_id)  REFERENCES students (school_id, id)
);
CREATE INDEX attendance_events_student_time_idx ON attendance_events (school_id, student_id, effective_ts);
CREATE INDEX attendance_events_school_time_idx  ON attendance_events (school_id, effective_ts DESC);
CREATE INDEX attendance_events_device_seq_idx   ON attendance_events (device_id, sequence_number);
CREATE INDEX attendance_events_review_idx       ON attendance_events (school_id, effective_ts DESC)
  WHERE outcome LIKE 'QUARANTINED%' OR outcome LIKE 'FLAGGED%';
```

daily_attendance (one row per student per school day)

```
CREATE TABLE daily_attendance (
  id                        uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id                 uuid NOT NULL REFERENCES schools(id),
  student_id                uuid NOT NULL,
  attendance_date           date NOT NULL,                  -- school-local calendar date
  academic_session_id       uuid NOT NULL,
  class_id                  uuid NOT NULL,
  attendance_status         text NOT NULL DEFAULT 'PENDING'
      CHECK (attendance_status IN ('PENDING','PRESENT','LATE','HALF_DAY','ABSENT','EXCUSED')),
  movement_status           text NOT NULL DEFAULT 'NOT_ARRIVED'
      CHECK (movement_status IN ('NOT_ARRIVED','ARRIVED','LEFT')),
  arrival_classification    text CHECK (arrival_classification IN ('ON_TIME','LATE','AFTER_CUTOFF')),
  departure_classification  text CHECK (departure_classification IN ('NORMAL','HALF_DAY','EARLY')),
  first_arrival_event_id    uuid,
  first_arrival_at          timestamptz,
  last_departure_event_id   uuid,
  last_departure_at         timestamptz,
  last_applied_event_ts     timestamptz,
  needs_review              boolean NOT NULL DEFAULT false,
  source                    text NOT NULL DEFAULT 'NFC' CHECK (source IN ('NFC','MANUAL','SYSTEM')),
  rule_version_id           uuid NOT NULL REFERENCES attendance_rule_versions(id),
  version                   integer NOT NULL DEFAULT 1,
  created_at                timestamptz NOT NULL DEFAULT now(),
  updated_at                timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT daily_attendance_uk UNIQUE (school_id, student_id, attendance_date),
  UNIQUE (school_id, id),
```

```
  FOREIGN KEY (school_id, student_id)             REFERENCES students (school_id, id),
  FOREIGN KEY (school_id, class_id)               REFERENCES classes (school_id, id),
  FOREIGN KEY (school_id, academic_session_id)    REFERENCES academic_sessions (school_id, id),
  FOREIGN KEY (school_id, first_arrival_event_id) REFERENCES attendance_events (school_id, id),
  FOREIGN KEY (school_id, last_departure_event_id) REFERENCES attendance_events (school_id, id)
);
CREATE INDEX daily_attendance_class_date_idx ON daily_attendance (school_id, class_id, attendance_date);
CREATE INDEX daily_attendance_date_status_idx ON daily_attendance (school_id, attendance_date,
attendance_status);
```

notification_outbox

```
CREATE TABLE notification_outbox (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id        uuid NOT NULL REFERENCES schools(id),
  kind             text NOT NULL,
  student_id       uuid,
  dedupe_key       text NOT NULL,
  payload          jsonb NOT NULL,
  source_event_id  uuid,
  status           text NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING','PROCESSING','COMPLETED','DEAD')),
  attempts         integer NOT NULL DEFAULT 0,
  next_attempt_at  timestamptz NOT NULL DEFAULT now(),
  locked_until     timestamptz,
  created_at       timestamptz NOT NULL DEFAULT now(),
  completed_at     timestamptz,
  CONSTRAINT notification_outbox_dedupe_uk UNIQUE (school_id, dedupe_key),
  UNIQUE (school_id, id),
  FOREIGN KEY (school_id, student_id)      REFERENCES students (school_id, id),
  FOREIGN KEY (school_id, source_event_id) REFERENCES attendance_events (school_id, id)
);
CREATE INDEX notification_outbox_claim_idx ON notification_outbox (next_attempt_at)
  WHERE status IN ('PENDING','PROCESSING');
```

RLS and immutability for attendance_events

```
-- Standard RLS template (generated by a migration helper for every school-owned table)
ALTER TABLE attendance_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE attendance_events FORCE  ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON attendance_events AS PERMISSIVE FOR ALL TO app_api
  USING      (school_id = nullif(current_setting('app.current_school_id', true), '')::uuid)
  WITH CHECK (school_id = nullif(current_setting('app.current_school_id', true), '')::uuid);

-- Immutability of raw events: application roles cannot delete; only processing columns are updatable
REVOKE DELETE ON attendance_events FROM app_api;
REVOKE UPDATE ON attendance_events FROM app_api;
GRANT  UPDATE (status, outcome, applied_to_daily, result, student_id, nfc_card_id, effective_ts,
timestamp_trust)
  ON attendance_events TO app_api;
```

> **Why some columns are updatable**
>
> The ingest transaction first claims the idempotency key (INSERT with provisional values) and then fills in the result in the same transaction. After commit the row is effectively immutable; replay and quarantine-review jobs may update only the listed processing columns, and each such change is audited.

13. Multiple NFC Device Architecture

A school may run one, two, five, ten or more NFC devices. This is a first-class requirement (DEV-REQ-015): the device model, credentials, concurrency control and monitoring are designed for N devices per school from the start.

13.1 Example deployment

Device ID	Location	Location type	School	Role in movement tracking
DEV001	Main Gate	ENTRY	SCH001	Primary arrival reader

Device ID	Location	Location type	School	Role in movement tracking
DEV002	Secondary Gate	BIDIRECTIONAL	SCH001	Arrival or departure depending on state and time window
DEV003	Classroom Block A	CLASSROOM	SCH001	Presence evidence; does not change gate movement state
DEV004	Classroom Block B	CLASSROOM	SCH001	Presence evidence
DEV005	Bus Entrance	EXIT (or ENTRY/EXIT pair)	SCH001	Departure by bus; may be configured as a pair of readers

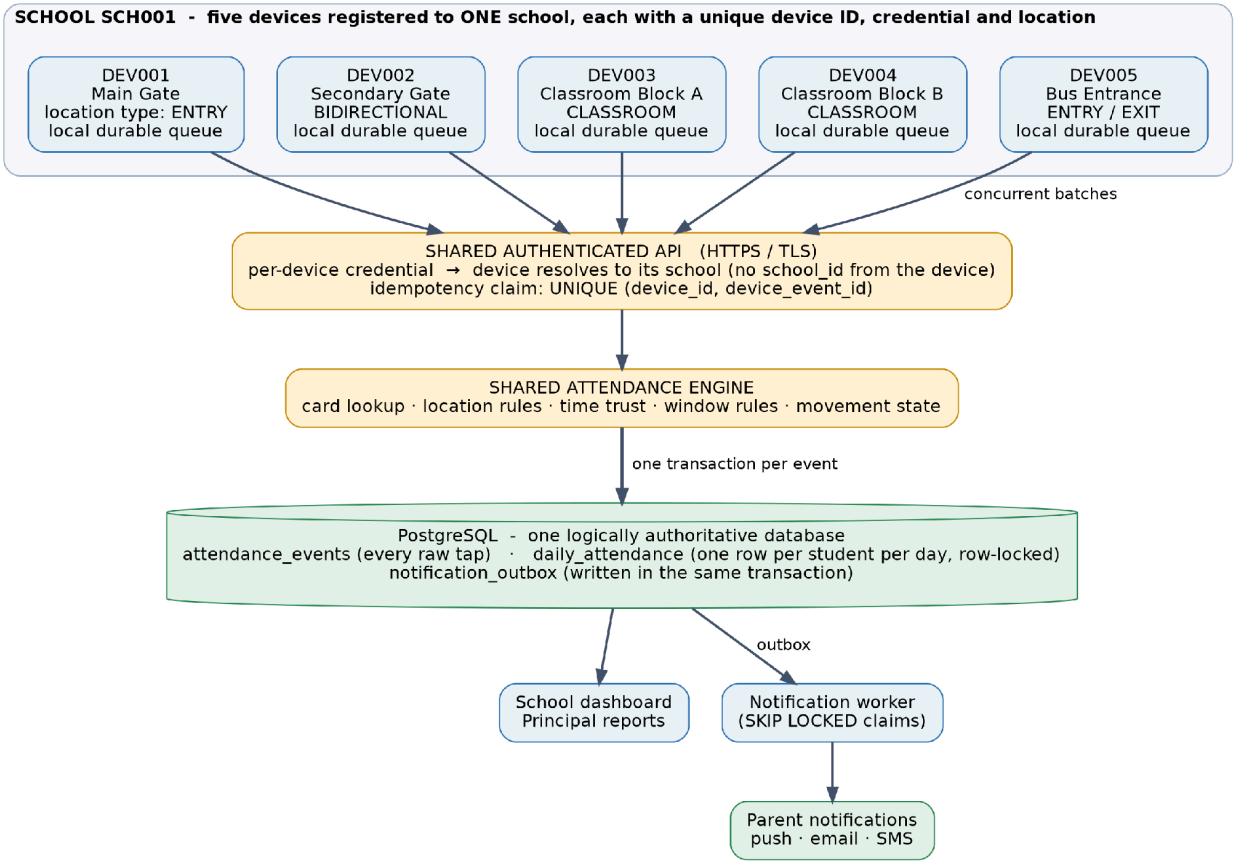


Figure 5 — Multi-device attendance flow: several concurrent readers, one authoritative database

All devices of a school register to the **same** school, have unique device IDs, individually assigned locations and independent credentials. Every tap flows through the same authenticated API, the same attendance engine and into the same PostgreSQL database; the school dashboard and parent notifications are produced from that single source.

13.2 Device attributes the platform tracks

Attribute	Where stored	Updated by
Unique device ID (`device_code`)	`devices.device_code`	Registration
Unique credentials	`device_credentials` (hash only)	Provisioning, rotation
School association	`devices.school_id`	Registration (never by the device)
Location association	`devices.location_id` → `device_locations`	School Admin
Firmware version	`devices.firmware_version`	Heartbeat, OTA status

Attribute	Where stored	Updated by
Online / offline status	Derived: ACTIVE and `last_seen_at` within threshold	Heartbeat, batch upload, watchdog job
Last heartbeat / last sync	`devices.last_seen_at`, `last_sync_at`	Heartbeat / successful batch
Pending event count	`devices.pending_event_count`	Heartbeat, sync-ack
NFC reader health	`devices.reader_status`	Heartbeat
RTC health	`devices.rtc_status`, `device_heartbeats.skew_ms`	Heartbeat
Wi-Fi signal strength	`devices.wifi_rssi_dbm`	Heartbeat
Revocation status	`devices.status`, `revoked_at`, credential status	Admin / Super Admin

13.3 Registration, claiming and revocation

Hardware is registered for a school by the platform (checking the plan's device limit), provisioned with a secret, then claimed by the School Admin, who names it and assigns its location. Revocation is available to the School Admin (own school) and the Super Admin and takes effect within the target of 60 seconds.

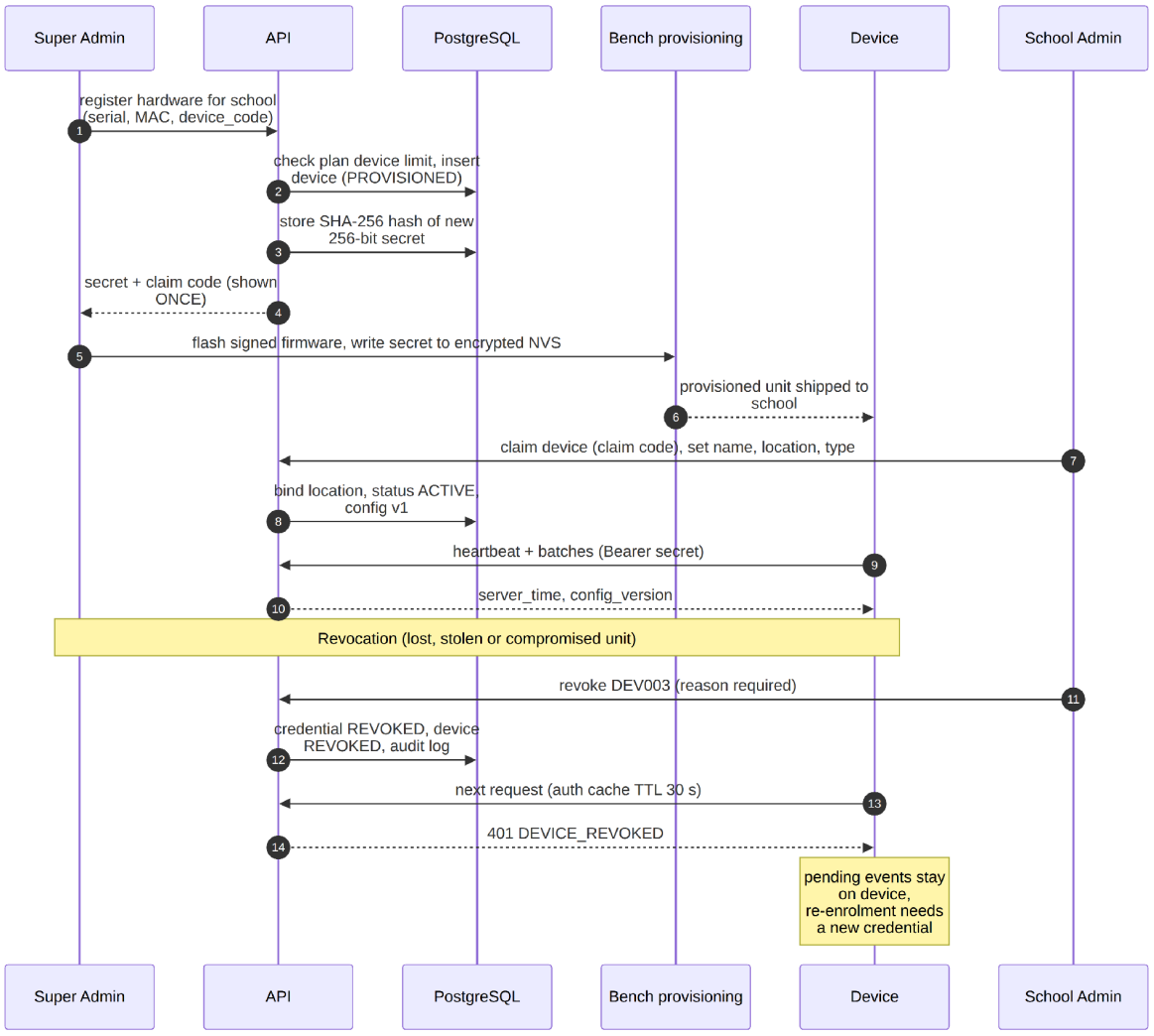


Figure 6 — Device registration and revocation

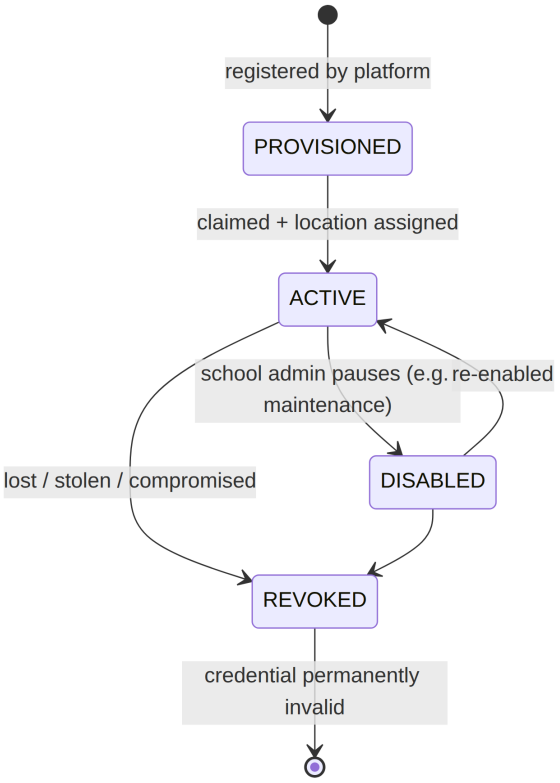


Figure 7 — Device lifecycle states

> **Device authority**
> The backend derives the school from the device's authenticated registration. A device that changes the `school_id` (or `student_id`) in its request gains nothing: those fields are ignored or rejected (DEV-REQ-002, TEST-REQ-028).

13.4 Concurrent attendance processing

Several devices can receive taps for the same student within milliseconds (for example DEV001 and DEV002 for STU1001). For each device batch the backend:

1. Authenticates the device and derives its school.
2. Verifies the card belongs to that school (lookup is always scoped by the device's school).
3. Claims the event's idempotency key by inserting it into `attendance_events` (unique `(device_id, device_event_id)`).
4. Resolves the card to its assigned student and the student's active enrolment.
5. Evaluates the event using its **trusted original timestamp** and the school's rule version.
6. Creates the day row if absent (`INSERT … ON CONFLICT DO NOTHING`), then locks it (`SELECT … FOR UPDATE`).
7. Applies the movement/attendance transition; a second simultaneous tap finds the state already changed and is recorded as a duplicate tap.
8. Writes the notification-outbox row only if the transition is meaningful.
9. Commits, then returns a clear per-event result to each device.

Correctness comes from database constraints and locks, not from an application "check then insert" (Section 17).

13.5 Duplicate protection: four different concepts

Concept	Definition	Stored?	Changes daily state?	Notification?
Duplicate event	The **same** device event (same `device_id` + `event_id`) delivered again, e.g. a retry after a lost response.	No second row; stored result returned	No	No
Repeated tap	A **new** physical tap of the same card at the same device within the debounce window (default 60 s) or while the state already reflects it.	Yes (raw event)	No	No
Duplicate daily attendance	An attempt to create a second attendance row for the same student, school and date (e.g. two gates in parallel).	Raw events stored; one daily row	One row only (UNIQUE constraint)	One
Legitimate arrival and departure	Different movements by the same student on one day (arrive, leave, possibly re-enter).	Yes	Yes, per state machine	Once per meaningful transition

13.6 Location roles

Location type	Meaning	Behaviour
ENTRY	Entrance only	Tap while NOT_ARRIVED → arrival. Tap while ARRIVED → repeat (ignored, flagged if after debounce). Tap while LEFT → re-entry if policy allows.
EXIT	Exit only	Tap while ARRIVED → departure (classified by time window). Tap while NOT_ARRIVED → recorded, flagged `FLAGGED_DEPARTURE_WITHOUT_ARRIVAL`, state unchanged until arrival is reconciled.
BIDIRECTIONAL	Used for both	NOT_ARRIVED → arrival. ARRIVED → departure **only if** the event time lies in a configured departure window; otherwise ignored and flagged (`FLAGGED_OUTSIDE_WINDOW`). LEFT → re-entry per policy.
CLASSROOM	In-building classroom reader	Stored as presence evidence. Does not change gate movement or send parent alerts by default. If `classroom_tap_implies_arrival` is enabled and the student has no arrival, it is treated as arrival at that time. Classroom taps may also feed per-period attendance.

13.7 Multi-gate and movement scenarios

Scenario	Handling	Result
Student arrives through Gate A (DEV001, ENTRY) and exits through Gate B (DEV002, BIDIRECTIONAL)	Both are valid movements. Arrival sets ARRIVED; the Gate B tap lies in a departure window so it sets LEFT.	Arrival + departure recorded; attendance status preserved; one notification each.
Student taps twice at the same gate	Second tap within debounce (or state already ARRIVED) is stored as `REPEAT_TAP_IGNORED`.	No state change; no notification.
Student taps at two gates simultaneously	Both events stored. Row lock serialises; the first applied is the arrival; the other is `DUPLICATE_TAP_OTHER_DEVICE_IGNORED`. If the later-committed event has the earlier timestamp, the day is replayed so the earliest tap is the arrival.	One daily row; one arrival notification.
Student taps a classroom reader and the main entrance reader	Entrance tap defines gate arrival; classroom tap is presence evidence. If the classroom tap came first (entrance device offline) and the policy is off, the day is flagged `PRESENT_WITHOUT_GATE_ARRIVAL` for review.	Gate movement unaffected by classroom tap.
Student exits before the arrival event has synchronised	EXIT event is stored with `FLAGGED_DEPARTURE_WITHOUT_ARRIVAL`. When the arrival event arrives (earlier `effective_ts`), the day is replayed in event-time order: arrival → departure.	Final state correct; notifications deduplicated.

Scenario	Handling	Result
A device goes offline while others remain online	Other devices continue normally. The offline device queues taps; watchdog alerts the admin; absent finalisation is deferred for that school (Section 14). On reconnection its events merge by replay.	No loss; delayed notifications may be suppressed if stale.

13.8 Device health monitoring

- Heartbeat default every 60 s while online; the server snapshots health on `devices` and stores history every 5 minutes or on change.
- Watchdog job (every minute): devices ACTIVE with no heartbeat for 3 minutes (configurable) become OFFLINE and raise a school-admin alert; after 15 minutes the platform on-call is alerted for Professional/Enterprise schools (Proposed).
- Alerts for: pending count > 500, storage > 80 %, RTC skew > 120 s, reader FAIL, firmware outdated, sequence gap.

Device requirements

ID	Requirement	Status
DEV-REQ-001	Each device SHALL have a unique device_code, unique credentials, school association, location, firmware version, online status, last heartbeat, last sync, pending count, reader health, RTC health, Wi-Fi signal and revocation status.	**Confirmed**
DEV-REQ-002	A device's school SHALL be derived from its authenticated registration; a device SHALL never select an arbitrary school or student.	**Confirmed**
DEV-REQ-003	Device secrets SHALL be stored only as hashes server-side, support rotation with overlap, and revocation SHALL take effect within a target of 60 seconds.	**Proposed**
DEV-REQ-004	The device SHALL persist each tap durably before giving "recorded" feedback and SHALL NOT delete any event before explicit server acknowledgement.	**Confirmed**
DEV-REQ-005	Each event SHALL include event_id, device_id, card_uid, event_timestamp, time_source, sequence_number, boot_id and firmware_version.	**Confirmed**
DEV-REQ-006	The device SHALL send periodic heartbeats containing health, queue depth, storage, RTC and connectivity data.	**Confirmed**
DEV-REQ-007	Batch uploads SHALL contain at most 100 events and 256 KB; the response SHALL contain a per-event result and an explicit device_action (DELETE or RETAIN_AND_RETRY).	**Proposed**
DEV-REQ-008	Uploads SHALL retry with exponential backoff plus jitter and SHALL resume automatically after reboot or network loss.	**Confirmed**
DEV-REQ-009	Firmware updates SHALL be cryptographically signed, version-checked against rollback and installed with automatic rollback on failed self-test.	**Confirmed**
DEV-REQ-010	The queue SHALL use per-record CRC, quarantine corrupt records, raise alerts at defined fill levels and never overwrite unacknowledged events.	**Confirmed**
DEV-REQ-011	Devices SHALL NOT store student personal data or backend privileged secrets; the only secret on a device is its own credential.	**Confirmed**
DEV-REQ-012	The server SHALL detect gaps in a device's sequence numbers and raise an alert for review.	**Proposed**
DEV-REQ-013	A device with no heartbeat for the configured threshold (default 3 minutes) SHALL be marked OFFLINE and alerted to the school admin.	**Proposed**
DEV-REQ-014	Device configuration SHALL be versioned, fetched over authenticated TLS and SHALL NOT contain secrets of other systems.	**Proposed**
DEV-REQ-015	A school SHALL be able to operate 1..N devices, limited only by its subscription plan.	**Confirmed**

14. Attendance Rule Engine

14.1 Two separate state dimensions

Attendance status says what the school records for the day. Movement status says where the student is. They are stored separately so that a departure never erases the arrival or the attendance result (ATT-REQ-001).

Dimension	Values	Changed by
Attendance status	PENDING, PRESENT, LATE, HALF_DAY, ABSENT, EXCUSED	Arrival classification, departure classification, absent job, approved correction, leave/excuse
Movement status	NOT_ARRIVED, ARRIVED, LEFT	Tap events through the state machine
Arrival classification	ON_TIME, LATE, AFTER_CUTOFF	First valid arrival (immutable except replay/correction)
Departure classification	NORMAL, HALF_DAY, EARLY	Last valid departure

> **PENDING is an addition**
> The brief lists PRESENT, LATE, HALF_DAY, ABSENT, EXCUSED. A neutral PENDING value is added as a Proposed default for the period before arrival or absence is decided; it avoids marking anyone ABSENT prematurely.

HALF_DAY is derived additively: a student who arrived (PRESENT or LATE) and departs inside the half-day window gets `attendance_status = HALF_DAY` while `arrival_classification` and `first_arrival_at` remain untouched, so the arrival information is never lost. Policy for late-and-half-day combinations is configurable (Validate with each school).

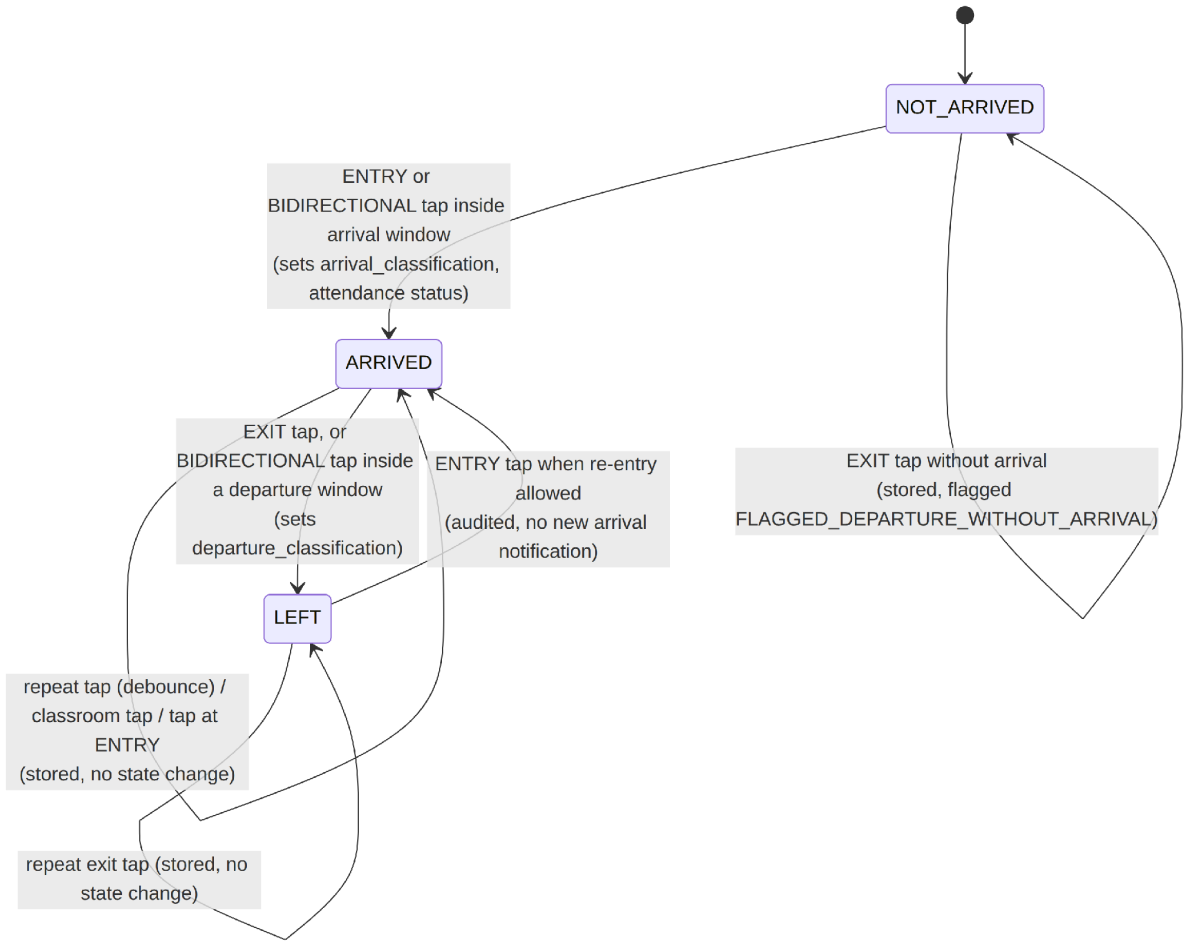


Figure 8 — Movement state machine

14.2 Example schedule (configurable, per school and per class)

Window (school-local)	Meaning	Result for a tap
before 06:30 (`earliest_accept`)	Too early	Stored; flagged `FLAGGED_TOO_EARLY`, not applied
06:30 ≤ t < 08:00	Early arrival	Arrival accepted, classified ON_TIME
08:00 ≤ t < 08:30	On-time arrival window	ARRIVAL_ON_TIME → PRESENT
08:30 ≤ t < 12:00	Late arrival window	ARRIVAL_LATE → LATE (policy)
t ≥ 12:00 (`late_end`)	After arrival cutoff	Stored; `ACCEPTED_AFTER_CUTOFF` → needs_review, not auto-PRESENT
12:00 ≤ t < 13:00	Half-day eligible departure	DEPARTURE_HALF_DAY → HALF_DAY
13:00 ≤ t < 14:00	Between half-day and normal	DEPARTURE_EARLY (flagged; policy: needs approval)
t ≥ 14:00	Normal departure (expected 14:00–15:00; later still NORMAL)	DEPARTURE_NORMAL
departure before 12:00 after arrival	Early departure	DEPARTURE_EARLY (flagged for review)

These windows are **examples only**. Every boundary is data in `attendance_rule_versions.rules`.

attendance_rule_versions.rules (example)

```
{
  "arrival":   { "earliest_accept": "06:30", "on_time_start": "08:00", "on_time_end": "08:30", "late_end":
"12:00" },
  "departure": { "half_day": { "start": "12:00", "end": "13:00" },
                 "normal":   { "start": "14:00", "end": "15:00" } },
```

```
  "absent_cutoff": "12:00",
  "absent_finalize_grace_minutes": 30,
  "repeat_tap_debounce_seconds": 60,
  "allow_reentry": true,
  "bidirectional_outside_window_action": "IGNORE_AND_FLAG",
  "classroom_tap_implies_arrival": false,
  "weekly_off": ["SUN"],
  "notify": { "arrival": true, "late": true, "departure": true, "absent": true },
  "late_counts_as": "LATE",
  "max_clock_skew_seconds": 120
}
```

14.3 Window evaluation rules

- Windows are **start-inclusive, end-exclusive**: 08:30:00.000 belongs to the late window, 08:29:59.999 to on-time (TEST-REQ-032).
- Evaluation converts `effective_ts` (UTC) to school-local time with the school's IANA timezone, and derives `attendance_date` as the local date; DST-safe and independent of server timezone.
- Overnight windows are not supported in V1.

14.4 Rule resolution and versioning

1. Class-specific rule for the date and day type, if any.
2. School-wide rule for the date and day type.
3. Day type is determined by `school_holidays`: EXAM_DAY or SHORT_DAY use those rule versions; otherwise REGULAR.
4. A rule version never changes after creation; changes create a new version with `effective_from`. `daily_attendance.rule_version_id` records which version produced a result (ATT-REQ-015).

14.5 Timestamps and trust

Rules use `effective_ts`, the best trustworthy time of the tap, never the upload time (ATT-REQ-003).

Case	Evidence	timestamp_trust	effective_ts	Action
RTC valid and recently synchronised	time_source RTC_SYNCED, skew ≤ max_clock_skew	TRUSTED	event_ts	Apply normally
Device was offline, RTC valid	Same, event_ts ≤ received_at, age ≤ max offline window (7 days)	TRUSTED	event_ts	Apply by replay if out of order
RTC lost power / unsynchronised	time_source RTC_UNSYNCED but boot has a heartbeat reference and `uptime_ms`	RECONSTRUCTED	boot_reference_time + uptime_ms	Apply; mark for review if reconstruction error > tolerance
Timestamp in the future (> received_at + 5 min) or older than the offline window	Impossible or implausible	SUSPECT	received_at (informational)	Quarantine: stored, `QUARANTINED_TIMESTAMP`, not applied, needs_review
No reliable source	time_source UNKNOWN	SUSPECT	received_at	Quarantine

14.6 Projection and replay

`daily_attendance` is a deterministic fold over the student's events for the day, ordered by `(effective_ts, device_id, sequence_number)`. Normal case: the new event is later than `last_applied_event_ts`, so it is applied incrementally. Out-of-order case: the new event is earlier, so the engine replays all of that student-day's events (typically < 10) inside the same transaction and rewrites the day row. Because the fold is a pure function, replay is safe, testable and idempotent (ATT-REQ-009).

Deterministic projection

```
// engine/replay.ts (pure; no I/O)
export function fold(events: Ev[], rules: RuleSet, base: DayState): DayState {
  let s = { ...base, movement: 'NOT_ARRIVED', arrival: null, departure: null };
  for (const e of sortByEffectiveTime(events)) {
    if (e.trust === 'SUSPECT' || e.status === 'REJECTED') continue;       // quarantined / rejected: never
applied
    s = transition(s, e, rules);   // movement + classification per location type, window and debounce
  }
  return deriveAttendanceStatus(s, rules);   // PRESENT/LATE/HALF_DAY from classifications; ABSENT/EXCUSED
preserved
}
```

14.7 Absent calculation

- At day-open (school-local 00:05) a job creates `PENDING` rows for all actively enrolled students on school days (holidays and weekly-offs excluded).
- The absent-finalisation job runs at `absent_cutoff + absent_finalize_grace_minutes` (default 12:30). Rows still PENDING with no accepted arrival and no approved leave become ABSENT and an absent notification is queued.
- **Deferral:** if any ACTIVE device of the school is OFFLINE or reports pending events, finalisation for that school is deferred and retried every 10 minutes up to a hard limit (default 15:30) after which it finalises with a "provisional" flag; the admin is alerted (Proposed).
- **Retraction:** a delayed event whose `effective_ts` is before the cutoff replays the day, replaces ABSENT with PRESENT/LATE and sends one correction notice (ATT-REQ-010).

14.8 Holidays, weekly offs and exam days

- `school_holidays` and `weekly_off` define non-school days: taps are stored with outcome `NO_SCHOOL_DAY` and not applied (ATT-REQ-011).
- EXAM_DAY / SHORT_DAY use their own rule versions (different windows); class-specific exam days override school-wide ones.

14.9 Manual corrections

1. Teacher, Admin or Principal creates an `attendance_corrections` row (REQUESTED) with a reason.
2. Principal (or Admin per school policy) approves or rejects; the actor cannot approve their own request.
3. On approval the daily row is updated (`source = MANUAL`, `version + 1`), the correction becomes APPLIED, an audit log entry is written and an `ATTENDANCE_CORRECTED` notification is queued if the parent-visible status changed.
4. Raw events are never modified (ATT-REQ-012). Later replays respect an applied correction through a `manual_override` marker.

14.10 Notification triggers

Transition	Dedupe key	Parent message (example)
First valid arrival of the day	`STUDENT_ARRIVED:{school}:{student}:{date}`	"Your child arrived at school at 8:12 AM."
Departure (per departure episode)	`STUDENT_DEPARTED:{school}:{student}:{date}:{n}`	"Your child left school at 2:31 PM."
Absent finalisation	`STUDENT_ABSENT:{school}:{student}:{date}`	"Your child has been marked absent today."
Correction changes visible status	`ATTENDANCE_CORRECTED:{school}:{student}:{date}:{version}`	"Attendance for today was updated to Present."

Attendance requirements

ID	Requirement	Status
ATT-REQ-001	Attendance status and movement status SHALL be stored in separate columns. A departure event SHALL NOT erase attendance status or the recorded arrival.	**Confirmed**
ATT-REQ-002	Attendance windows SHALL be configurable per school and per class, versioned, timezone-aware, start-inclusive and end-exclusive.	**Confirmed**
ATT-REQ-003	Rules SHALL be evaluated against the event's trusted original timestamp (effective_ts), never its upload time.	**Confirmed**
ATT-REQ-004	Events with untrustworthy timestamps SHALL be stored, flagged and quarantined for review rather than applied automatically.	**Confirmed**
ATT-REQ-005	Processing SHALL be idempotent per (device, event_id); a retried event SHALL return the stored result without reprocessing.	**Confirmed**
ATT-REQ-006	Simultaneous taps from several devices SHALL be stored as separate raw events yet resolve to exactly one daily record, using a row lock inside one transaction.	**Confirmed**
ATT-REQ-007	A tap SHALL NOT be blindly treated as departure; location type, current movement state and time window SHALL decide the outcome.	**Confirmed**
ATT-REQ-008	Parent notifications SHALL be created only on meaningful state changes and SHALL carry a deterministic dedupe key.	**Confirmed**
ATT-REQ-009	daily_attendance SHALL be a deterministic projection of the ordered attendance events of that student and date; an out-of-order event SHALL trigger a replay of that day.	**Proposed**
ATT-REQ-010	An absent-finalisation job SHALL run after the absent cutoff plus a synchronisation grace period, defer for schools with offline devices holding pending events, and be retractable.	**Proposed**
ATT-REQ-011	Taps on holidays or non-school days SHALL be recorded but not applied to daily attendance.	**Proposed**
ATT-REQ-012	Manual corrections SHALL use attendance_corrections with requester, approver and reason; raw events SHALL never be edited.	**Confirmed**
ATT-REQ-013	Classroom-reader events SHALL be stored as presence evidence and SHALL NOT change gate movement status unless the school enables classroom_tap_implies_arrival.	**Proposed**
ATT-REQ-014	An event SHALL be acknowledged to the device only after its database transaction has committed.	**Confirmed**
ATT-REQ-015	Each daily_attendance row SHALL record the rule_version_id used, so historical results remain explainable after rule changes.	**Proposed**
ATT-REQ-016	Re-entry after departure (e.g. returning for an appointment) SHALL follow a configurable policy and SHALL be audited.	**Proposed**

15. Device Firmware Architecture

15.1 Prototype hardware

Part	Purpose	Notes
ESP32 DevKit V1 or ESP32-S3 dev board	MCU, Wi-Fi, flash	ESP32-S3 is recommended for production because of secure boot v2 and flash encryption support (verify chip revision before relying on it — Validate).
PN532 NFC reader (I²C or SPI)	ISO14443A card read	Read UID in V1; keep the option to run authenticated card protocols later.
NFC cards (e.g. MIFARE Classic/Ultralight/NTAG class)	Student credential	UID-only mode is clonable (Section 21).
DS3231 RTC with backup cell	Reliable time during Wi-Fi/NTP outages	Check oscillator-stop flag; periodic NTP discipline.

Part	Purpose	Notes
Persistent local storage	Event queue	Prototype: LittleFS partition in flash; production: evaluate external FRAM or industrial flash (Validate).
RGB LED, buzzer, optional OLED	Immediate feedback	OLED shows only "Recorded" — no student name by default.
Regulated 5 V supply	Power	Optional protected rechargeable backup for outage ride-through.

15.2 Firmware responsibilities

- Read NFC cards and debounce physical re-reads (card held on reader).
- Generate a unique `event_id` (UUIDv4 from the hardware RNG) and monotonically increasing `sequence_number` persisted across reboots; create a new `boot_id` at each boot.
- Record `event_timestamp` and its **trust source** (`time_source`) plus `uptime_ms`.
- **Store the event durably before giving "recorded" feedback.**
- Upload in batches in sequence, retry with exponential backoff and jitter, reconnect after failures.
- Delete events only after server acknowledgement with `device_action = DELETE`.
- Monitor hardware health and report it in heartbeats; queue diagnostic logs.
- Fetch versioned configuration; apply signed OTA updates with rollback.

> **Backend is authoritative**
> The device never decides "present", "late" or "departed". It reports what happened and when; the server derives meaning (ATT/DEV requirements). Firmware therefore needs no student data and no attendance rules.

15.3 Firmware module structure

firmware/ layout

```
firmware/
  src/
    main.cpp / app_main.c      # init, task creation
    nfc/        reader.*        # PN532 driver wrapper, UID read, anti-bounce
    time/       clock.*         # DS3231 + NTP discipline, trust classification, uptime
    event/      event.*         # event record, UUID, sequence, boot_id
    queue/      queue.*         # durable append-only log with CRC, ack cursor, segment reclaim
    net/        wifi.*  http.*  # reconnect, TLS (CA pinning), request signing headers
    sync/       sync.*          # batch builder, retry/backoff, result handling
    health/     health.*        # heartbeat, self-tests, thresholds
    config/     config.*        # versioned config, defaults, secure storage access
    ota/        ota.*           # download, hash + signature verify, A/B partition, rollback
    ui/         feedback.*      # LED/buzzer/OLED patterns
  include/contracts_generated.h # event codes and limits generated from packages/contracts
  partitions.csv                # nvs, otadata, ota_0, ota_1, queue(littlefs), coredump
  platformio.ini / sdkconfig.*
```

15.4 Tap handling

Tap path (pseudocode)

```
on_card_detected(uid):
  if (same_uid && now - last_tap_ms < 1500): return             // physical re-read of a held card
  ev.event_id        = uuid_v4()
  ev.sequence_number = next_sequence()                          // persisted counter (write-ahead)
  ev.boot_id         = BOOT_ID
  ev.card_uid        = hex_upper(uid)
  (ev.event_timestamp, ev.time_source) = clock_now_with_source() // RTC_SYNCED | RTC_UNSYNCED | NTP |
MONOTONIC
  ev.uptime_ms       = millis()
  ok = queue_append(ev)                                         // CRC record, flush, verify
  if ok:   feedback(RECORDED)                                   // single short beep, green
```

```
  else:    feedback(STORAGE_ERROR)                                  // distinct pattern, DO NOT pretend
success
  signal(sync_task)
```

15.5 Durable event queue

Aspect	Design
Structure	Append-only log split into fixed-size segments (e.g. 16 KB). A small double-buffered **ack cursor** record stores the sequence acknowledged by the server.
Record format (≈ 96–128 bytes)	`magic(2) | version(1) | length(1) | sequence(8) | event_id(16) | boot_id(16) | uid_len(1)+uid(≤10) | timestamp_ms(8) | time_source(1) | uptime_ms(8) | flags(1) | crc32(4)`
Durability	Record is flushed and re-read for CRC before feedback. Power loss mid-write leaves a torn record that fails CRC and is skipped.
Deletion	Never delete individual events early. A segment is reclaimed only when **every** record in it is at or below the acknowledged cursor.
Capacity estimate	10,000 records × 128 B ≈ 1.3 MB. At ≈ 3,000 taps per day for a 1,000-student school shared across devices, a single busy gate device buffers several days. Size the partition for **≥ 72 hours** of the device's own peak traffic (Validate with a field measurement).
Thresholds	80 % full: WARN in heartbeat. 95 % full: error pattern + alert. 100 %: **stop accepting new taps and signal clearly** (never overwrite unacknowledged events).
Corruption handling	On CRC failure, scan forward to the next magic number, count and quarantine bad bytes, write a `QUEUE_CORRUPTION` device log, continue. If the ack cursor copy is corrupt use the older copy; if both are corrupt, re-send from the oldest present record (server idempotency makes this safe).
Reboot recovery	On boot: mount, validate segments, rebuild in-memory index, increment boot counter, create new `boot_id`, continue sequence numbers from the persisted counter.
Wear	Segments rotate across the partition; programmes avoid rewriting the same block (production: FRAM removes the wear concern).

15.6 Synchronisation and retry

Sync loop (pseudocode)

```
sync_task():
  loop:
    wait_for(wifi_connected && has_pending)
    batch = queue_peek(max_events=50, max_bytes=100_000)         // oldest first, in sequence order
    resp  = POST /device/events/batch (batch)                    // TLS, device credential
    if resp.ok:
        for r in resp.results:
            if r.device_action == "DELETE": mark_acked(r.event_id)
            // RETAIN_AND_RETRY stays in the queue
        advance_ack_cursor(contiguous_acked_sequence())
        POST /device/sync-ack
        backoff.reset()
    else:  // network error, 5xx, 429, timeout
        sleep(backoff.next())                                    // 2 s, 4 s, 8 s ... capped at 5 min, +/-
25 % jitter
                                                                 // honour Retry-After from 429/503
```

- Never skip a failing event: events are sent in sequence order; a permanently rejected event gets `device_action = DELETE` (it is stored server-side as REJECTED), so it cannot block the queue.
- Storage-full alerts and queue depth are reported in every heartbeat.

15.7 Time handling

- DS3231 holds UTC. At boot the firmware reads the RTC, checks the oscillator-stop flag (OSF) and, when online, disciplines it from NTP at most every 6 hours (Proposed). `server_time` in every API response is a second reference.

- `time_source`: `RTC_SYNCED` (RTC valid, last NTP sync recent), `RTC_UNSYNCED` (OSF set or never synced), `NTP` (no RTC, live NTP), `MONOTONIC` (only uptime available).
- Every event carries `uptime_ms` so the server can reconstruct the time of taps in a boot session when the wall clock was wrong (Section 14).

15.8 Health monitoring and self-tests

- PN532 self-test (firmware version query) at boot and every 5 minutes; failure → `reader_status = FAIL`, red pattern, heartbeat alert.
- RTC drift check against `server_time`; skew > 120 s → `rtc_status = DRIFT`.
- Wi-Fi RSSI, free heap, flash fill, reset reason (brownout, watchdog) reported in heartbeat.
- Task watchdog resets a hung firmware; the reason is logged and reported.

15.9 Feedback patterns

Situation	LED	Buzzer
Tap stored (recorded)	Green 300 ms	1 short beep
Server later rejects card (blocked/unknown) — shown only when online and enabled	Red 600 ms	2 short beeps
Storage error / queue full	Red fast blink	3 long beeps
Offline (taps still recorded)	Amber pulse on idle	none
Reader failure	Red solid	periodic beep
OTA in progress	Blue pulse	none

15.10 Secure configuration and OTA

- Wi-Fi credentials and the device secret live in encrypted NVS; production enables flash encryption and secure boot.
- OTA uses dual app partitions. The firmware image is signed; the device verifies hash and signature (public key in the bootloader/firmware) and rejects images with a lower `security_version` (anti-rollback). After first boot a self-test must pass within 2 minutes, otherwise the bootloader rolls back (DEV-REQ-009).
- The device reports state with `POST /device/ota/status` (OFFERED → DOWNLOADING → VERIFYING → INSTALLED / FAILED / ROLLED_BACK).
- The pending queue survives OTA because it lives in its own partition.

> **No privileged secrets on devices**
> The ESP32 holds only its own revocable device credential and the OTA **public** key. It never holds Supabase service keys, database URLs, FCM server keys or other devices' secrets (DEV-REQ-011, SEC-REQ-006).

15.11 Prototype-to-production transition

Topic	Prototype	Production
Board	DevKit + breakout modules on a breadboard / perfboard	Custom PCB or industrial module; connectors, ESD protection, enclosure with tamper switch
MCU security	Plain flash	Flash encryption, secure boot v2, disabled JTAG/UART download mode, per-device credential injected at provisioning
Storage	LittleFS in flash	FRAM or industrial flash; wear and power-fail tested

Topic	Prototype	Production
Power	5 V adapter	Protected supply with brownout handling; optional battery/UPS
Connectivity	Wi-Fi	Wi-Fi with site survey; optional Ethernet (PoE) for gates
Cards	UID read	Authenticated cards (e.g. DESFire/NTAG 424 DNA class — Validate) to counter cloning
Provisioning	Manual	Bench provisioning tool; QR claim code; factory test report
Updates	USB flash	Signed OTA with staged rollout channels
Certification	—	Radio/EMC/safety compliance for the target market (Validate)

16. Device API Specification

16.1 Transport and authentication

- V1 uses HTTPS REST over TLS 1.2+. The device validates the server certificate chain (pinned CA bundle, not "insecure" mode).
- Every request carries `X-Device-Id: DEV001` and `Authorization: Bearer <device_secret>`. The server hashes the secret (SHA-256) and compares it in constant time with an ACTIVE or ROTATING_OUT credential of that device; the device must be ACTIVE (or PROVISIONED for config/heartbeat only).
- The school is **derived** from the device registration. Any `school_id`, `student_id` or other-device `device_id` in a body is ignored (headers/body mismatch of `device_id` → `403 DEVICE_MISMATCH`).
- Server auth results are cached per process for 30 s so revocation takes effect within the 60 s target.
- Requests include `Content-Type: application/json`; responses always include `server_time` so the device can detect clock skew.

16.2 Endpoint summary

Method + path	Purpose	Max body	Rate (per device)
`POST /api/v1/device/events/batch`	Upload up to 100 tap events; returns per-event results	256 KB	30 / min
`POST /api/v1/device/heartbeat`	Health report, logs, queue depth; returns server time and config/OTA hints	16 KB	6 / min
`GET /api/v1/device/config`	Fetch versioned configuration (supports `If-None-Match`)	—	6 / min
`POST /api/v1/device/sync-ack`	Device confirms purge of acknowledged events and reports cursor	4 KB	30 / min
`POST /api/v1/device/ota/status`	Report OTA download/verify/install state	4 KB	12 / min

16.3 Event payload

Field	Type	Required	Description
event_id	UUID	Yes	Unique per tap; generated on device. With `device_id` forms the idempotency key.
device_id	string	Yes	Device code (e.g. DEV001); must match the authenticated device.
card_uid	string	Yes	Uppercase hex UID as read.
event_timestamp	RFC 3339 UTC (ms)	Yes	Tap time per the device clock.
time_source	enum	Yes	RTC_SYNCED, RTC_UNSYNCED, NTP, MONOTONIC, UNKNOWN.

Field	Type	Required	Description
sequence_number	integer	Yes	Monotonic per device, persisted across reboots.
boot_id	UUID	Yes	Random per boot.
firmware_version	string	Yes	Semantic version.
uptime_ms	integer	Recommended	Milliseconds since boot, for time reconstruction.

16.4 POST /api/v1/device/events/batch

Events must be sent in ascending `sequence_number`. The server processes each event in its **own** transaction and returns one result per event.

Example device event request (school-local time 08:12:03 IST = 02:42:03 UTC)

```
POST /api/v1/device/events/batch
X-Device-Id: DEV001
Authorization: Bearer <device secret>
Content-Type: application/json

{
  "batch_id": "b1f0a7c2-3e5d-4c91-8a47-0d6e2f9b1c58",
  "device_id": "DEV001",
  "firmware_version": "1.2.0",
  "sent_at": "2026-10-12T02:42:05.120Z",
  "events": [
    {
      "event_id": "9b2f6c1e-4d7a-4e53-8c19-2a7f5d3b8e60",
      "device_id": "DEV001",
      "card_uid": "04A2B3C4D5E680",
      "event_timestamp": "2026-10-12T02:42:03.412Z",
      "time_source": "RTC_SYNCED",
      "sequence_number": 48213,
      "boot_id": "c7d2e9a1-5b3f-4a86-9e10-3f8b6d2a7c45",
      "uptime_ms": 5231876,
      "firmware_version": "1.2.0"
    }
  ]
}
```

Example batch upload response (success)

```
HTTP/1.1 200 OK
{
  "batch_id": "b1f0a7c2-3e5d-4c91-8a47-0d6e2f9b1c58",
  "server_time": "2026-10-12T02:42:05.480Z",
  "config_version": 3,
  "results": [
    {
      "event_id": "9b2f6c1e-4d7a-4e53-8c19-2a7f5d3b8e60",
      "status": "RECORDED",
      "outcome": "ARRIVAL_ON_TIME",
      "attendance_status": "PRESENT",
      "movement_status": "ARRIVED",
      "notification_queued": true,
      "device_action": "DELETE"
    }
  ],
  "summary": { "received": 1, "recorded": 1, "duplicate": 0, "rejected": 0, "retry_later": 0 },
  "ack_through_sequence": 48213
}
```

Result for the simultaneous tap from a second device (DEV002)

```
{
  "event_id": "6e1a8d4c-3f95-4b27-a0c6-d7e2b5f18a49",
  "status": "RECORDED",
  "outcome": "DUPLICATE_TAP_OTHER_DEVICE_IGNORED",
  "attendance_status": "PRESENT",
  "movement_status": "ARRIVED",
  "notification_queued": false,
  "device_action": "DELETE"
}   // DEV002's tap 277 ms after DEV001's tap for the same student: stored, no state change
```

Example duplicate-event response

```
{
  "event_id": "9b2f6c1e-4d7a-4e53-8c19-2a7f5d3b8e60",
  "status": "DUPLICATE_EVENT",
  "original_status": "RECORDED",
  "original_outcome": "ARRIVAL_ON_TIME",
  "processed_at": "2026-10-12T02:42:05.470Z",
  "notification_queued": false,
  "device_action": "DELETE"
}   // the same event_id retried: no second processing, previous result returned
```

Example rejected-card response

```
{
  "event_id": "2d7c9f10-8a3e-4b65-b1d4-6c0e5a9f3e72",
  "status": "REJECTED",
  "outcome": "REJECTED_CARD_BLOCKED",
  "reason": "Card is blocked.",
  "notification_queued": false,
  "device_action": "DELETE"
}   // unknown cards return outcome REJECTED_UNKNOWN_CARD; both are stored for audit
```

Per-event status and device_action

status	Meaning	device_action
RECORDED	Stored and processed; the business outcome is in `outcome` (including ignored repeat taps, flagged and quarantined events).	DELETE
DUPLICATE_EVENT	Idempotency hit; original result returned.	DELETE
REJECTED	Permanently refused but stored for audit (unknown/blocked card, no active enrolment, malformed but attributable event).	DELETE
RETRY_LATER	Transient failure (database unavailable, lock timeout, deadlock retries exhausted). Nothing was committed.	RETAIN_AND_RETRY

Business outcome codes

outcome	Meaning
ARRIVAL_ON_TIME / ARRIVAL_LATE / ACCEPTED_AFTER_CUTOFF	Arrival classified by window; the last one needs review
DEPARTURE_NORMAL / DEPARTURE_HALF_DAY / DEPARTURE_EARLY	Departure classified by window
REENTRY_RECORDED	Return after departure (policy allowing)
REPEAT_TAP_IGNORED / DUPLICATE_TAP_OTHER_DEVICE_IGNORED	Valid tap that does not change state
CLASSROOM_PRESENCE_RECORDED	Classroom reader evidence
FLAGGED_DEPARTURE_WITHOUT_ARRIVAL / FLAGGED_OUTSIDE_WINDOW / FLAGGED_TOO_EARLY	Stored and flagged for reconciliation
QUARANTINED_TIMESTAMP	Untrusted timestamp; not applied
NO_SCHOOL_DAY	Holiday or weekly off; not applied
STORED_PENDING_RULES	No applicable rule version (configuration error); stored for reprocessing
REJECTED_UNKNOWN_CARD / REJECTED_CARD_BLOCKED / REJECTED_NO_ACTIVE_ENROLLMENT	Refused

16.5 POST /api/v1/device/heartbeat

Example device heartbeat (request and response)

```
POST /api/v1/device/heartbeat
{
  "device_id": "DEV001",
  "boot_id": "c7d2e9a1-5b3f-4a86-9e10-3f8b6d2a7c45",
  "device_time": "2026-10-12T02:43:00.003Z",
  "uptime_ms": 5288466,
  "firmware_version": "1.2.0",
```

```
  "config_version": 3,
  "pending_events": 0,
  "oldest_pending_sequence": null,
  "last_sequence": 48213,
  "storage_used_pct": 4,
  "reader_status": "OK",
  "rtc_status": "OK",
  "last_ntp_sync": "2026-10-12T00:00:11Z",
  "wifi_rssi_dbm": -58,
  "reset_reason": "POWERON",
  "logs": [ { "ts": "2026-10-12T02:10:00Z", "level": "WARN", "code": "WIFI_RECONNECT", "detail": {
"attempts": 2 } } ]
}

HTTP/1.1 200 OK
{ "server_time": "2026-10-12T02:43:00.120Z", "skew_ms": -117, "config_version": 3,
  "ota": { "available": false }, "alerts": [] }
```

16.6 GET /api/v1/device/config

Device configuration

```
GET /api/v1/device/config            If-None-Match: "3"
HTTP/1.1 200 OK    ETag: "3"
{
  "config_version": 3,
  "heartbeat_interval_s": 60,
  "batch_max_events": 50,
  "batch_max_bytes": 100000,
  "tap_debounce_ms": 1500,
  "time_sync_interval_h": 6,
  "feedback": { "led_brightness": 60, "buzzer": true, "show_result_on_oled": false },
  "storage_warn_pct": 80,
  "location": { "name": "Main Gate", "type": "ENTRY" },
  "ota": { "channel": "stable" }
}   // 304 Not Modified when unchanged; never contains secrets or student data
```

16.7 POST /api/v1/device/sync-ack

Purge confirmation and gap detection

```
POST /api/v1/device/sync-ack
{ "device_id": "DEV001", "boot_id": "c7d2e9a1-5b3f-4a86-9e10-3f8b6d2a7c45",
  "purged_through_sequence": 48213, "pending_events": 0, "queue_corruption_count": 0 }

HTTP/1.1 200 OK
{ "server_time": "2026-10-12T02:42:06.010Z", "gap_detected": false, "next_expected_sequence": 48214 }
```

The batch response is the authoritative acknowledgement. `sync-ack` tells the server that the device has actually purged records and lets the server verify sequence continuity (`gap_detected` raises a DEV-REQ-012 alert).

16.8 POST /api/v1/device/ota/status

OTA state report (states: OFFERED, DOWNLOADING, VERIFYING, INSTALLED, FAILED, ROLLED_BACK)

```
POST /api/v1/device/ota/status
{ "device_id": "DEV001", "target_version": "1.3.0", "state": "VERIFYING",
  "detail": { "sha256_ok": true, "signature_ok": true }, "reported_at": "2026-10-12T03:10:00Z" }

HTTP/1.1 204 No Content
```

16.9 Batching, retries, idempotency and limits

- **Batching:** up to 100 events and 256 KB per request (device default 50 / 100 KB). Larger backlogs are sent as consecutive batches (Section 18).
- **Retries:** on timeout or network error the device resends the same events with the same `event_id`s; the server is idempotent. Exponential backoff with jitter; honour `Retry-After`.
- **Idempotency:** key = `(device_id, event_id)`; `batch_id` is informational and is not an idempotency key.

- **Acknowledgement:** a result with `device_action = DELETE` is returned only after the event's transaction has committed (ATT-REQ-014). Responses are never sent for events whose transaction rolled back; those yield `RETRY_LATER`.
- **Partial failure:** one event failing never fails the batch; HTTP 200 is returned with per-event results. HTTP 5xx is used only when the whole batch could not be processed.

16.10 Device error responses

HTTP	Code	Device behaviour
400 / 422	BAD_REQUEST, VALIDATION_ERROR	Log locally, report in heartbeat; do not retry unchanged payload; quarantine the offending record after N attempts.
401	UNAUTHENTICATED, DEVICE_REVOKED	Stop uploading, keep queue, show alert pattern; retry only with long backoff (1 h) for UNAUTHENTICATED; DEVICE_REVOKED is terminal until re-provisioned.
403	DEVICE_MISMATCH, DEVICE_DISABLED	As above; report.
413	PAYLOAD_TOO_LARGE	Halve batch size and retry.
429	RATE_LIMITED	Honour `Retry-After`.
5xx / network	INTERNAL, DEPENDENCY_UNAVAILABLE	Exponential backoff; keep queue.

> **Evolution**
> Endpoint names may be refined, but the contract elements above (idempotency key, per-event result, `device_action`, server-derived school, ack-after-commit) are fixed. A later version may add MQTT or signed-request (HMAC/ECDSA) authentication behind the same contract (Proposed).

17. Concurrency and Idempotency

17.1 Event processing steps

Each event is processed in its own transaction. The 12 steps required by the brief map to the implementation as follows; note that step 4 (idempotency) is implemented as an **atomic claim** with `INSERT … ON CONFLICT DO NOTHING`, not as a read-then-write, and step 9 completes the claimed row.

#	Step	Implementation
1	Authenticate device	Device auth middleware: header id + secret hash compare; status check.
2	Resolve device registration and school	`devices` row gives `school_id`, `location_id`, location type; tenant context set from it.
3	Validate payload	Zod schema; foreign school/student fields rejected; event id/time format; batch size.
4	Check idempotency	`INSERT INTO attendance_events … ON CONFLICT (device_id, device_event_id) DO NOTHING RETURNING id`. No row returned = duplicate → return stored result.
5	Identify active NFC card and student	`SELECT … FROM nfc_cards WHERE school_id = $1 AND card_uid = $2 AND status <> 'RETIRED'`; join to student and active enrolment.
6	Validate device location	Device's location type snapshot; ACTIVE device; location active.
7	Apply attendance rules	Resolve rule version for school-local date/class; time trust; pure `transition()` function.
8	Safely update daily attendance	`INSERT … ON CONFLICT DO NOTHING` the day row, `SELECT … FOR UPDATE`, fold/apply, `UPDATE`.
9	Record the event	`UPDATE attendance_events SET outcome, status, student_id, applied_to_daily, result …` for the claimed row.

#	Step	Implementation
10	Insert notification job	`INSERT INTO notification_outbox … ON CONFLICT (school_id, dedupe_key) DO NOTHING` only on meaningful transitions.
11	Commit	Single `COMMIT`; any error rolls back the whole event.
12	Return recorded result	Per-event result built from the committed row; `RETRY_LATER` if the transaction failed.

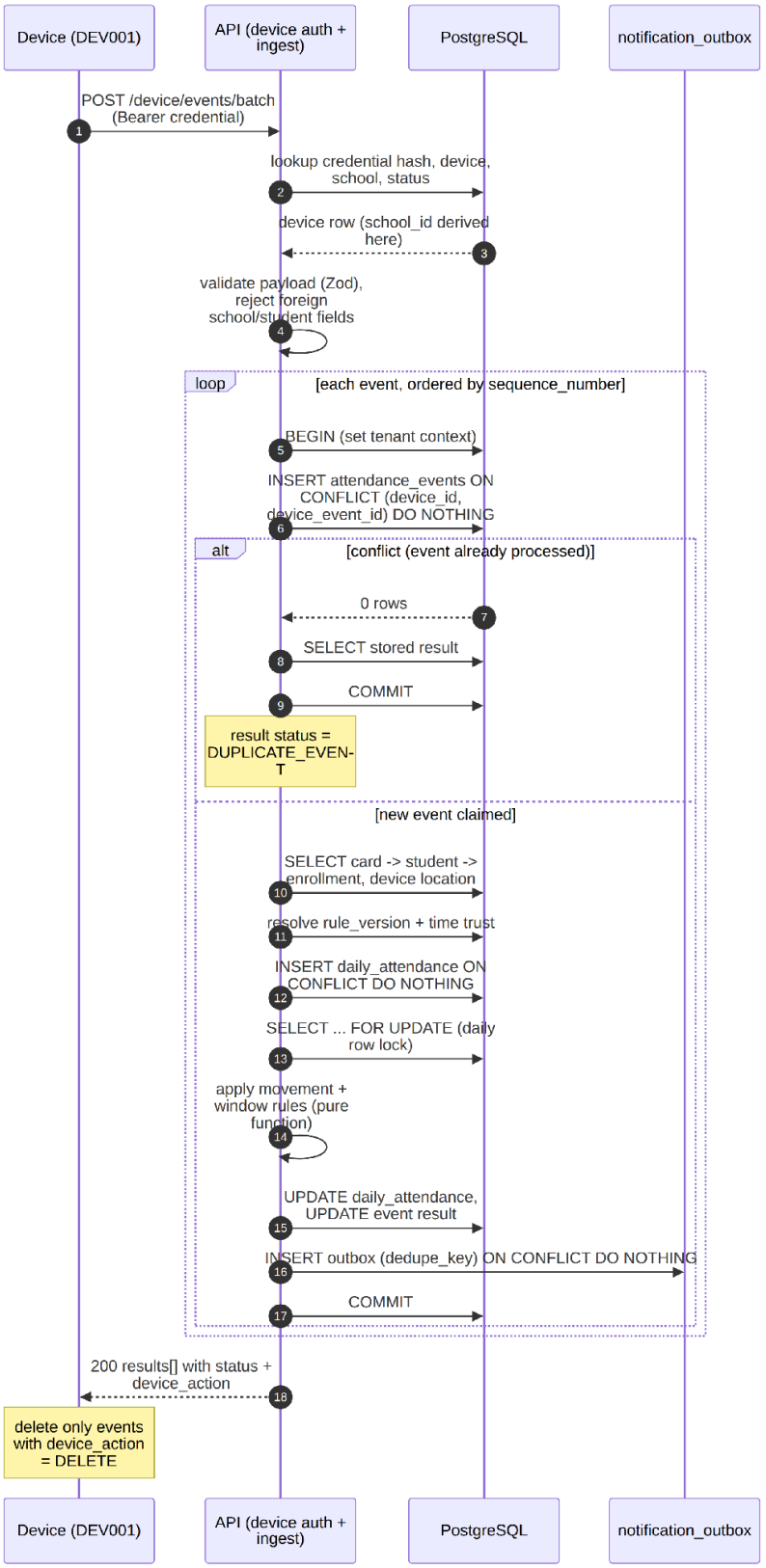


Figure 9 — Attendance transaction sequence

17.2 Transaction pseudocode

Event processing (TypeScript-style pseudocode)

```
async function processEvent(device: AuthedDevice, ev: DeviceEvent): Promise<EventResult> {
  try {
    return await withTenantTx({ schoolId: device.schoolId, actorType: 'DEVICE', userId: device.id }, async
(tx) => {
```

```
      // (4) atomic idempotency claim; concurrent identical events block here until the first commits
      const claimed = await tx.one(sql`
        INSERT INTO attendance_events (school_id, device_id, device_event_id, sequence_number, boot_id,
               card_uid, location_id, location_type, event_ts, effective_ts, time_source,
               timestamp_trust, firmware_version, status, outcome)
        VALUES (${device.schoolId}, ${device.id}, ${ev.eventId}, ${ev.seq}, ${ev.bootId},
               ${ev.cardUid}, ${device.locationId}, ${device.locationType}, ${ev.ts}, ${trust.effectiveTs},
               ${ev.timeSource}, ${trust.level}, ${ev.firmware}, 'RECORDED', 'PROCESSING')
        ON CONFLICT (device_id, device_event_id) DO NOTHING
        RETURNING id`).catch(noRows);
      if (!claimed) return await loadStoredResult(tx, device.id, ev.eventId);       // DUPLICATE_EVENT

      // (5) card -> student -> enrolment, always scoped by the device's school
      const card = await cardsRepo.findLive(tx, ev.cardUid);                         // WHERE school_id =
tenant
      if (!card)                 return finish(tx, claimed.id, 'REJECTED', 'REJECTED_UNKNOWN_CARD');
      if (card.status !== 'ACTIVE') return finish(tx, claimed.id, 'REJECTED', 'REJECTED_CARD_BLOCKED', {
card });
      const enrol = await enrolRepo.activeFor(tx, card.studentId, trust.effectiveTs);
      if (!enrol)                return finish(tx, claimed.id, 'REJECTED',
'REJECTED_NO_ACTIVE_ENROLLMENT', { card });

      // (7) rules + calendar
      const local = toSchoolLocal(trust.effectiveTs, device.schoolTz);               // date + time
      if (trust.level === 'SUSPECT') return finish(tx, claimed.id, 'RECORDED', 'QUARANTINED_TIMESTAMP', {
card });
      if (await calendar.isNonSchoolDay(tx, local.date, enrol.classId))
                                  return finish(tx, claimed.id, 'RECORDED', 'NO_SCHOOL_DAY', { card });
      const rules = await rulesRepo.resolve(tx, local.date, enrol.classId);
      if (!rules)                 return finish(tx, claimed.id, 'RECORDED', 'STORED_PENDING_RULES', {
card });

      // (8) one row per student per day; lock it so concurrent taps are serialised
      await tx.exec(sql`INSERT INTO daily_attendance (school_id, student_id, attendance_date,
academic_session_id,
                        class_id, rule_version_id) VALUES (${device.schoolId}, ${card.studentId},
${local.date},
                        ${enrol.sessionId}, ${enrol.classId}, ${rules.id})
                        ON CONFLICT (school_id, student_id, attendance_date) DO NOTHING`);
      const day = await tx.one(sql`SELECT * FROM daily_attendance
                        WHERE school_id = ${device.schoolId} AND student_id = ${card.studentId}
                          AND attendance_date = ${local.date} FOR UPDATE`);

      const evRow = { id: claimed.id, effectiveTs: trust.effectiveTs, locationType: device.locationType,
deviceId: device.id, seq: ev.seq };
      let next, change;
      if (day.lastAppliedEventTs == null || trust.effectiveTs >= day.lastAppliedEventTs) {
        ({ next, change } = engine.applyEvent(day, evRow, rules));                   // incremental
      } else {
        const events = await eventsRepo.forStudentDay(tx, card.studentId, local.date);   // includes the
new event
        ({ next, change } = engine.replay(events.concat(evRow), rules, day));            // out-of-order:
replay
      }
      await dailyRepo.update(tx, day.id, next, day.version);                         // version + 1

      // (9) complete the claimed event row
      await eventsRepo.complete(tx, claimed.id, { studentId: card.studentId, cardId: card.id,
            outcome: change.outcome, applied: change.applied });

      // (10) outbox only for meaningful transitions; dedupe key prevents repeats
      for (const n of change.notifications)
        await tx.exec(sql`INSERT INTO notification_outbox (school_id, kind, student_id, dedupe_key,
payload, source_event_id)
                          VALUES (${device.schoolId}, ${n.kind}, ${card.studentId}, ${n.dedupeKey},
${n.payload}, ${claimed.id})
                          ON CONFLICT (school_id, dedupe_key) DO NOTHING`);
      return buildResult(claimed.id, change);                                        // (12) after COMMIT
by withTenantTx
    });                                                                              // (11) COMMIT
  } catch (e) {
    if (isTransient(e)) return { eventId: ev.eventId, status: 'RETRY_LATER', deviceAction:
'RETAIN_AND_RETRY' };
    throw e;
  }
}
```

`finish()` updates the claimed row with the given status and outcome and returns the result. All early returns still commit the claimed row, so rejected and quarantined taps are audited.

17.3 Race conditions and how each is closed

Race	What could go wrong	Control
Same event retried concurrently (two requests with the same event id)	Both pass a "does it exist?" check and both process	Unique index + `ON CONFLICT DO NOTHING`; the second INSERT waits for the first transaction to finish, then gets zero rows and reads the stored result.
Two devices, same student, same instant	Both read `NOT_ARRIVED`; both mark arrival; two notifications	`SELECT … FOR UPDATE` on the single `daily_attendance` row serialises them; the second sees `ARRIVED`.
Day row does not exist yet	Two concurrent INSERTs create two rows	`UNIQUE (school_id, student_id, attendance_date)` + `ON CONFLICT DO NOTHING`; then both lock the same row.
Later-committed event has an earlier timestamp	Arrival time recorded wrongly	Compare with `last_applied_event_ts`; replay the day in event-time order.
Duplicate notification	Retry, replay or second worker creates two alerts	`UNIQUE (school_id, dedupe_key)` in the outbox; `UNIQUE (outbox_id, parent_id, channel)` in deliveries.
Deadlock between batches	Two requests lock rows in opposite order	Each transaction locks at most one student-day row (per-event transactions); deadlock errors are retried with bounded attempts.
Pooled connection leaks tenant	A later request inherits a previous tenant setting	Tenant settings are transaction-local (`set_config(..., true)`).
Response lost after commit	Device resends, fear of double processing	Idempotent replay returns `DUPLICATE_EVENT` with the original result.

17.4 Worked example: simultaneous taps at DEV001 and DEV002

Time (UTC)	Actor	Action	State after
02:42:03.412	STU1001	Taps card at DEV001 (Main Gate, ENTRY)	—
02:42:03.690	STU1001	Taps same card at DEV002 (Secondary Gate, BIDIRECTIONAL)	—
02:42:05.1	API ← DEV001	Claims event 9b2f…; creates day row; locks it	row locked by txn 1
02:42:05.1	API ← DEV002	Claims event 6e1a…; INSERT day row → no-op; `FOR UPDATE` **waits**	txn 2 blocked
02:42:05.2	txn 1	Applies arrival 08:12:03 IST → ON_TIME → PRESENT, ARRIVED; outbox `STUDENT_ARRIVED`; COMMIT	PRESENT / ARRIVED
02:42:05.2	txn 2	Lock acquired; sees ARRIVED; BIDIRECTIONAL tap outside departure window and inside debounce → `DUPLICATE_TAP_OTHER_DEVICE_IGNORED`; no outbox; COMMIT	unchanged
02:42:05.4	API → DEV001 / DEV002	Each device receives its own clear result (ARRIVAL_ON_TIME / DUPLICATE_TAP_OTHER_DEVICE_IGNORED)	both delete

Result: two raw events, one daily row, one arrival notification — regardless of which transaction wins the lock.

17.5 Isolation level and locking choices

- READ COMMITTED is sufficient because every shared decision is protected by a unique constraint or a row lock taken before the read of mutable state.
- Lock scope is a single row (the student-day); there is no table lock and no advisory lock in the hot path.
- `lock_timeout` = 3 s and `statement_timeout` = 10 s on the ingest role; timeouts map to `RETRY_LATER`.

17.6 Example SQL constraints used by the above

```
ALTER TABLE attendance_events ADD CONSTRAINT attendance_events_idem_uk UNIQUE (device_id, device_event_id);
ALTER TABLE daily_attendance  ADD CONSTRAINT daily_attendance_uk       UNIQUE (school_id, student_id,
attendance_date);
ALTER TABLE notification_outbox ADD CONSTRAINT notification_outbox_dedupe_uk UNIQUE (school_id,
dedupe_key);
ALTER TABLE notification_deliveries ADD CONSTRAINT notification_deliveries_uk
  UNIQUE (school_id, outbox_id, parent_id, channel);
```

17.7 Example attendance record and outbox entry

Example daily_attendance record

```
-- daily_attendance after the arrival above
{
  "id": "a2c5e8f1-7b34-4d06-9c18-5e0f3a7d2b64",
  "school_id": "7f3c2a10-5b1e-4c8a-9d2f-1a6e3b9c4d01",
  "student_id": "e4b2c9a7-1d53-4e8f-a6b0-9c3f7d2e5a14",     // STU1001
  "attendance_date": "2026-10-12",
  "attendance_status": "PRESENT",
  "movement_status": "ARRIVED",
  "arrival_classification": "ON_TIME",
  "departure_classification": null,
  "first_arrival_event_id": "f61c0b9e-2a47-4d83-8e15-b7a3c9d04e28",   // attendance_events.id for 9b2f...
  "first_arrival_at": "2026-10-12T02:42:03.412Z",
  "rule_version_id": "d9e3b1a5-6c28-4f70-a4b2-1e8f5c7a9d03",
  "needs_review": false, "source": "NFC", "version": 2
}
```

Example notification_outbox entry

```
-- notification_outbox row created in the same transaction
{
  "id": "4c8e1f27-9a63-4b50-8d12-e7f0a5b3c691",
  "school_id": "7f3c2a10-5b1e-4c8a-9d2f-1a6e3b9c4d01",
  "kind": "STUDENT_ARRIVED",
  "student_id": "e4b2c9a7-1d53-4e8f-a6b0-9c3f7d2e5a14",
  "dedupe_key": "STUDENT_ARRIVED:SCH001:STU1001:2026-10-12",
  "payload": { "arrived_at_local": "08:12", "date": "2026-10-12", "location": "Main Gate" },
  "source_event_id": "f61c0b9e-2a47-4d83-8e15-b7a3c9d04e28",
  "status": "PENDING", "attempts": 0, "next_attempt_at": "2026-10-12T02:42:05.2Z"
}
```

> **Consistency of the examples**
> The same student (STU1001), school (SCH001), devices (DEV001, DEV002), timestamps and event ids are used in Sections 14, 16, 17 and 19 so that engineers can follow one scenario end to end.

18. Offline Synchronization

Devices must keep recording taps through Wi-Fi, internet, server or database outages and synchronise afterwards without losing, duplicating or mis-timing events. The device queue is the buffer; the server's idempotency and replay logic make catching up safe.

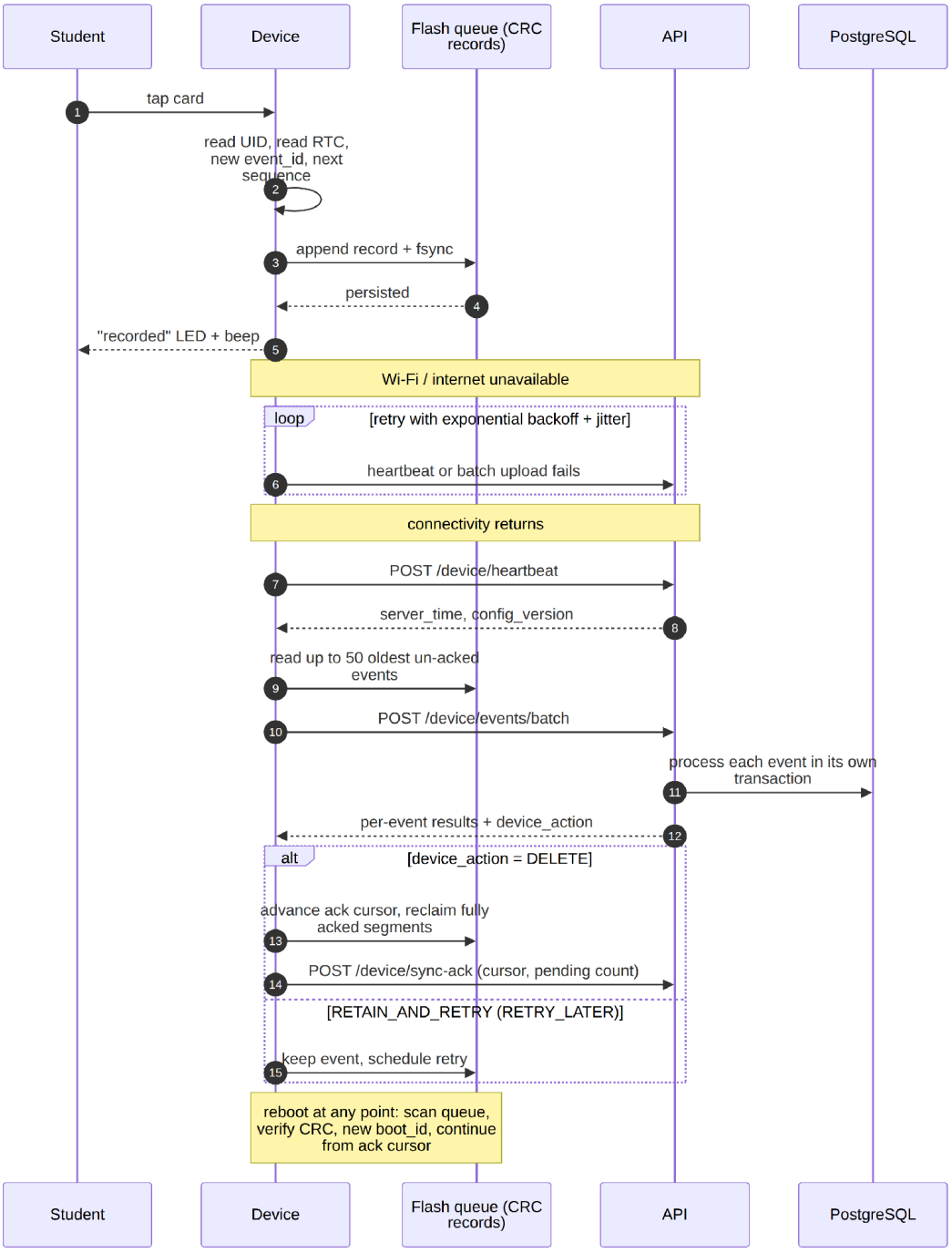


Figure 10 — Offline event synchronisation

18.1 Guarantees

Guarantee	Provided by
Accepted tap is never lost because Wi-Fi is down	Durable queue written before feedback; no deletion before DELETE result
Reboot or power loss does not lose acknowledged-pending events	CRC records, ack cursor, boot recovery
Retries never double-count	Idempotency key (device_id, event_id)
Late events are applied at the right time	Original timestamp, `effective_ts`, replay in event-time order
Wrong clocks do not corrupt attendance	Trust classification, reconstruction from `uptime_ms`, quarantine
Backlog does not overwhelm the server	Batch size limits, per-device rate limit, `Retry-After`, backoff with jitter

18.2 Synchronisation protocol

1. On reconnect the device sends a heartbeat to learn `server_time` and `config_version`.
2. It reads up to 50 of the oldest un-acknowledged events in sequence order and uploads them.
3. The server processes each event in its own transaction and returns per-event results.
4. The device marks `DELETE` results as acknowledged, advances its ack cursor over the contiguous prefix, reclaims fully acknowledged segments and sends `sync-ack`.
5. `RETAIN_AND_RETRY` events stay queued and are retried with backoff.
6. The loop continues until the queue is empty; throughput is limited to one in-flight batch per device.

18.3 Out-of-order and cross-device ordering

- Order **within** a device is by `sequence_number`; order **across** devices is by `effective_ts`. The server never assumes upload order equals tap order.
- If an event's `effective_ts` precedes the day row's `last_applied_event_ts`, the student-day is replayed (Section 14). Replay is deterministic, so applying the same set of events in any upload order yields the same final state (TEST-REQ-030).

18.4 Time reconstruction

If a boot session's RTC was untrustworthy but a later heartbeat of the same `boot_id` gave a reliable `(server_time, uptime_ms)` pair, the server computes `boot_reference_time = server_time − uptime_ms` and reconstructs `effective_ts = boot_reference_time + event.uptime_ms`, marking the event `RECONSTRUCTED`. Reconstruction error is bounded by heartbeat latency and clock drift; events exceeding tolerance are marked for review.

18.5 Backlog bursts

Situation	Handling
One device uploads 5,000 events after an outage	100 batches of 50; one in flight; ≈ 20 events/s per device is within target; other devices unaffected because per-event transactions are short and locks are per student-day.
Several devices reconnect simultaneously after a school-wide outage	Server rate limiter returns 429 with `Retry-After`; devices back off with jitter, so arrivals of the backlog spread out. Worker and DB pool headroom keep dashboards responsive (PERF-REQ-008).
Server or database down for hours	Devices keep queueing (capacity ≥ 72 h); on recovery the same flow applies. `RETRY_LATER` events are retained.
Sequence gap detected	`sync-ack` returns `gap_detected`; alert raised; events missing from the queue are reported by firmware via `device_logs`.

18.6 Effects on notifications and absent finalisation

- Notifications for events older than the staleness limit (default 24 h) are suppressed (`EXPIRED_STALE`); events within the limit are sent with the **event time**, e.g. "arrived at 8:12 AM", never with the upload time.
- Absent finalisation for a school is deferred while any of its devices is OFFLINE or reports pending events (Section 14). A late-synchronised arrival retracts a provisional ABSENT and sends one correction notice.
- Dashboards show a "data may be delayed" banner when any device is offline.

18.7 Outage matrix

Failure	Device	Server	Recovery
Wi-Fi down	Queues, amber idle LED	Watchdog marks OFFLINE	Auto reconnect, drain

Failure	Device	Server	Recovery
Internet down at school	Same	Same	Same
API down / deploying	Upload fails, backoff	Alert on error rate	Retry; idempotent
Database down	5xx or RETRY_LATER	Alert, health degraded	Retry; no partial state
Power cut at device	Queue persists (CRC)	OFFLINE alert	Reboot, new boot_id, resume
RTC wrong	Events flagged by time_source	Quarantine/reconstruct, alert	Admin resets/time sync; review queue
Flash nearly full	Heartbeat warning; stop at 100 %	Alert	Restore connectivity; replace storage

19. Notification Architecture

19.1 Design

Parent notifications use the **transactional outbox pattern**. The attendance transaction inserts an outbox row in the same commit as the attendance change, so a notification exists if and only if the change committed. A separate worker process delivers from the outbox. Attendance is therefore never blocked or lost by a provider outage (NOT-REQ-001, NOT-REQ-005).

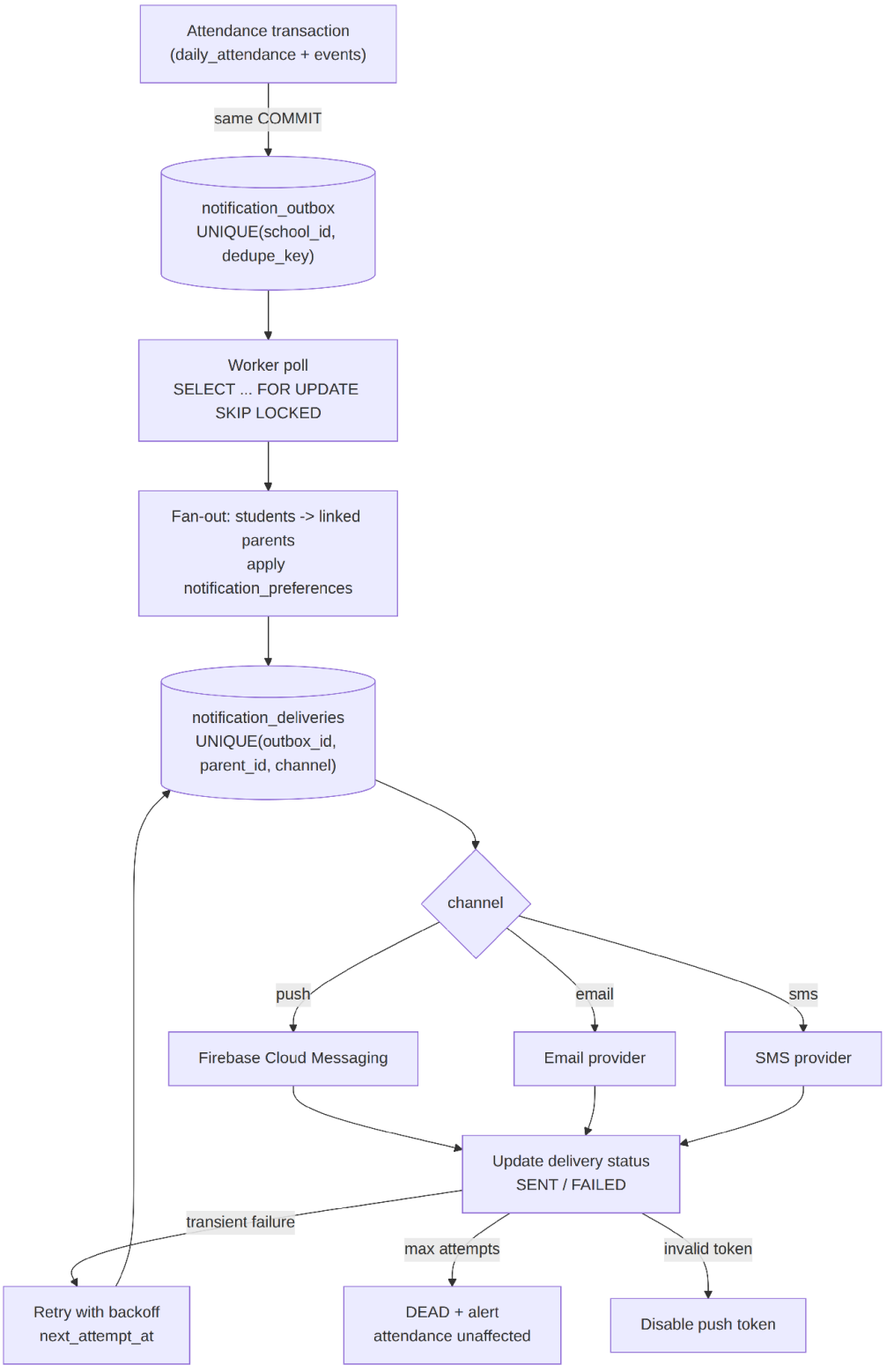


Figure 11 — Parent notification outbox and delivery pipeline

Sender address

vidyalayaainfo@gmail.com is the official platform contact and support address. For automated parent emails, send from a verified custom domain (for example no-reply@<your-domain>) with SPF, DKIM and DMARC through the email provider; consumer Gmail addresses are not suitable as bulk transactional senders because of deliverability and sending limits (Validate with the chosen provider).

19.2 Channels and messages

Channel	Provider	Use	Notes
Push	Firebase Cloud Messaging	Default for attendance alerts	Tokens in `push_tokens`; invalid tokens disabled on provider error. FCM is a delivery channel only.
Email	Transactional provider (adapter)	Fallback and summaries	Verified sender domain; unsubscribe applies only to non-critical kinds.
SMS	SMS provider (adapter)	Opt-in or fallback for parents without the app	Cost-controlled; Indian operator template/sender registration must be completed (Validate).

Event	Example message
Arrival	"Your child arrived at school at 8:12 AM."
Late arrival	"Your child arrived at school at 9:05 AM (late)."
Departure	"Your child left school at 2:31 PM."
Absent	"Your child has been marked absent today."
Correction	"Attendance for today was updated to Present."

Templates are keyed by `kind` and locale; the child's first name is included only if the parent has not chosen a "no names on lock screen" preference (Proposed).

19.3 Worker algorithm

Outbox claim query

```
-- claim (runs every 2 s per worker, several workers may run concurrently)
WITH picked AS (
  SELECT id FROM notification_outbox
   WHERE next_attempt_at <= now()
     AND (status = 'PENDING' OR (status = 'PROCESSING' AND locked_until < now()))   -- recover crashed
workers
   ORDER BY next_attempt_at
   FOR UPDATE SKIP LOCKED
   LIMIT 50)
UPDATE notification_outbox o
   SET status = 'PROCESSING', attempts = o.attempts + 1, locked_until = now() + interval '2 minutes'
  FROM picked WHERE o.id = picked.id
RETURNING o.*;
```

1. **Claim** a batch with `SKIP LOCKED` so parallel workers never take the same row.
2. **Expand**: load linked parents from `student_parents` (status ACTIVE, `can_receive_alerts`), apply `notification_preferences` and quiet hours, choose channels.
3. **Create deliveries** with `INSERT … ON CONFLICT (outbox_id, parent_id, channel) DO NOTHING` (safe for re-runs).
4. **Send** each PENDING delivery through its adapter with a timeout; record `provider_message_id`.
5. **Record** SENT, or FAILED with `next_attempt_at` per the retry schedule; mark the outbox row COMPLETED when all deliveries are final.
6. **Suppress** deliveries older than the staleness limit (`EXPIRED_STALE`) or for disabled preferences (`SUPPRESSED`).

19.4 Retry, failure and dead-letter handling

Attempt	Delay after failure	Applies to
1 → 2	30 seconds	Push, email, SMS
2 → 3	2 minutes	Push, email, SMS
3 → 4	10 minutes	Push, email, SMS
4 → 5	30 minutes	Push, email, SMS
5 → 6	2 hours	Email; push/SMS go DEAD if the event is older than 2 hours
> 6 or permanent error	DEAD	Alert to platform on-call; visible in admin "failed notifications" list

- Permanent errors (invalid or unregistered token, invalid phone) are not retried; the token or address is disabled and the parent's other channels continue.
- Provider rate-limit responses honour `Retry-After`; a per-provider token-bucket limits sending rate; a circuit breaker pauses a failing provider for 60 s.
- Alerts: DEAD count > 0, backlog age > 5 minutes, provider error rate > 5 % over 10 minutes (thresholds Proposed).

19.5 Preferences, multiple parents and duplicate prevention

- `notification_preferences (parent, kind, channel, enabled, quiet hours)`; defaults: arrival/departure/absent on push and email, SMS off.
- A student with two linked parents produces two independent delivery sets from one outbox row; failure for one parent never affects the other (TEST-REQ-044).
- **No fresh notification per duplicate tap:** only transitions with a dedupe key produce outbox rows; repeated, duplicate and ignored taps do not.
- Dedupe keys: `STUDENT_ARRIVED:{school}:{student}:{date}`; `STUDENT_DEPARTED:{school}:{student}:{date}:{n}`; `STUDENT_ABSENT:…`; `ATTENDANCE_CORRECTED:…:{version}`.
- Per-student daily cap (default 8 notifications) protects parents from malfunctioning devices (Proposed).

19.6 Audit history

`notification_deliveries` retains every attempt, status, provider message id and error for the audit period (default 180 days, Proposed). Parents can see a notification history in the app. Support staff may view delivery status under an access grant without seeing message bodies beyond what policy allows.

19.7 Push token lifecycle

- The parent app registers an FCM token after login (`push_tokens`, hashed lookup, platform, last_seen).
- Tokens are deactivated on logout, provider "unregistered" responses, and 60 days of inactivity.
- A user may have several active tokens (phone, tablet).

Notification requirements

ID	Requirement	Status
NOT-REQ-001	The notification outbox row SHALL be inserted in the same database transaction as the attendance change that caused it.	**Confirmed**
NOT-REQ-002	The worker SHALL claim jobs with FOR UPDATE SKIP LOCKED; delivery is at-least-once and duplicates are suppressed by dedupe keys.	**Confirmed**

ID	Requirement	Status
NOT-REQ-003	Failed deliveries SHALL retry with backoff and, after the maximum attempts, move to DEAD with an alert.	**Confirmed**
NOT-REQ-004	Parent preferences SHALL be honoured per channel and every linked parent SHALL be notified independently.	**Confirmed**
NOT-REQ-005	Attendance SHALL remain saved when any notification provider is unavailable.	**Confirmed**
NOT-REQ-006	Notifications older than a configurable staleness limit (default 24 h) SHALL be suppressed and recorded as EXPIRED_STALE.	**Proposed**
NOT-REQ-007	Every delivery attempt and outcome SHALL be retained for audit for a configurable period.	**Proposed**

20. Subscription Architecture

20.1 Packages and entitlements

Packages are rows in `subscription_plans` with `features` (flags) and `limits` (numbers). Contract-specific overrides are stored in `subscriptions.limit_overrides`. Prices are **not** part of this design: they live in the billing provider and contract and must not be hard-coded.

Capability / limit	Basic	Professional	Enterprise
Student and parent management, teachers	Included	Included	Included
Marks, results, report cards	Included	Included	Included
Attendance reports (manual attendance)	Included	Included	Included
Notices / announcements	Included	Included	Included
NFC devices and automated attendance	—	Included	Included
Automated parent alerts (push/email/SMS)	—	Included	Included
Timetable	—	Included	Included
Homework and study materials	—	Included	Included
Advanced reporting	—	Included	Included + advanced analytics
Multiple campuses (organizations)	—	—	Included
Custom integrations, priority support	—	—	Included
`max_students`, `max_devices`, `max_campuses`	Plan limits (set per contract)	Plan limits (larger)	Contract-defined

20.2 Enforcement points

Point	Mechanism	On violation
Every API route	`entitlements.requireFeature(flag)` reads the school's plan (cached 60 s)	403 `SUBSCRIPTION_FEATURE_UNAVAILABLE`; UI shows upgrade message
Create student / device / campus	Service checks current count vs limit inside the same transaction	409 `PLAN_LIMIT_REACHED`
Device registration	Counts non-revoked devices against `max_devices`	Registration refused; existing devices unaffected
Worker	Skips alert kinds not in plan (records `SUPPRESSED`)	No delivery; outbox retained per policy
Frontend	Hides modules by `features` from `/me`	UX only; API is authoritative

20.3 Subscription lifecycle and grace policy

The values below are **Proposed defaults** to be fixed in each contract (Validate with legal and finance).

State	Entered when	Access	Devices	Exit
TRIALING	Onboarding	Full (limited by trial plan)	Ingest	Payment set up → ACTIVE
ACTIVE	Paid	Full per plan	Ingest	Payment failure → PAST_DUE
PAST_DUE	Payment failed or period ended unpaid (day 0)	Full access; banners; reminders on day 0, 3, 7, 12	Ingest	Paid → ACTIVE; day 15 → SUSPENDED_READONLY
SUSPENDED_READONLY	Day 15	Read-only for staff and parents; no new marks/fees/announcements; school admin can still pay and export	Ingest continues; alerts paused (outbox kept; stale ones expire)	Paid → ACTIVE; day 45 → SUSPENDED
SUSPENDED	Day 45	Only School Admin: billing and data export	Ingest continues for non-revoked devices	Paid → ACTIVE; cancellation → CANCELLED
CANCELLED / EXPIRED	Contract ended	No access except export window	Devices disabled after export window	Retention period then deletion (Section 21)

Never erase, never silently stop recording

Subscription state changes **features and access**, not data. Device event ingestion continues for non-revoked devices in every state (SUB-REQ-003), so taps already made are never lost to a billing dispute. Deletion happens only through the documented retention/offboarding job after the contractual period, never as a side effect of expiry.

20.4 Operations

- A daily `subscriptionState` job evaluates `current_period_end` and `grace_ends_at`, moves states, writes `subscription_events` and audit logs, and notifies the School Admin and platform support.
- Billing integration (provider TBD; for INR billing candidates include Razorpay or Stripe — Validate availability, fees and compliance) updates subscription state through a signed webhook endpoint with idempotency keys. Webhooks never touch school data.
- Super Admin can extend grace, override limits, or reinstate with a reason; each action is audited.
- Usage metering (active students, devices, notifications sent) is stored for plan reviews and capacity planning.

Subscription requirements

ID	Requirement	Status
SUB-REQ-001	Subscription plans SHALL define feature flags and numeric limits enforced server-side.	**Confirmed**
SUB-REQ-002	Payment failure SHALL follow a documented grace and suspension policy; expiry SHALL never delete school data.	**Confirmed**
SUB-REQ-003	Device event ingestion SHALL continue in every subscription state for non-revoked devices so that recorded taps are never lost.	**Confirmed**
SUB-REQ-004	Unavailable features SHALL return a stable error code (SUBSCRIPTION_FEATURE_UNAVAILABLE) and the UI SHALL show an upgrade message.	**Proposed**
SUB-REQ-005	All subscription state changes SHALL be written to audit_logs and subscription_events.	**Proposed**

21. Security and Privacy

21.1 Threat overview

Threat	Example	Primary controls
Cross-school data leakage	Admin of School A reads School B students	Tenant gatekeeper, RLS, composite FKs, route/table sweep tests
Parent overreach	Parent reads another child	`student_parents` check on every request; 404 responses
Stolen/forged device	Attacker with a device credential posts fake taps	Per-device credential scoped to one device; school derived server-side; revocation; anomaly alerts; physical hardening
Card cloning / sharing	Friend taps with a copied card	Documented risk; lost-card block; authenticated cards in production; anomaly rules (two gates in impossible time)
Credential theft	Phished admin	MFA, short sessions, step-up auth, audit alerts
Secret leakage	Service-role key in browser or firmware	Secrets only in host stores; CI secret scans; separate roles
Injection / XSS	Malicious announcement text	Parameterised queries, Zod validation, sanitised Markdown, CSP
Abuse / DoS	Request floods, oversize payloads	Rate limits, body limits, WAF/CDN options, backoff
Malicious firmware	Unauthorised OTA	Signed images, secure boot, anti-rollback
Insider misuse	Support staff browsing children's data	No platform role on student tables; grants + audit
Data loss	Bad migration or deletion	Backups, PITR, restore drills, forward-only migrations, immutable raw events

21.2 Controls

21.2.1 Passwords, sessions and MFA

- Supabase Auth handles password hashing, recovery email, session and refresh token rotation. Configure minimum length (≥ 10), leaked-password protection and email confirmation where the plan supports it (Validate).
- Access tokens short-lived (default ≈ 1 hour); refresh tokens rotated; privileged roles use shorter session limits (Proposed).
- Optional TOTP-based MFA; required for Super Admin, recommended/enforceable for School Admin and Principal.
- Account lockout/back-off and rate limits on login and recovery endpoints.

21.2.2 Authorisation and tenant isolation

- Server-side RBAC with permission codes; fail-closed routes; scope checks for teacher-class and parent-child.
- Multi-tenant defence in depth (Section 8): membership-derived school, `withTenantTx`, FORCED RLS, composite FKs, automated tests.
- Parent-child validation on every request, no client-supplied relationship trusted (SEC-REQ-004).

21.2.3 Transport, secrets and rate limiting

- HTTPS/TLS 1.2+ everywhere; HSTS on web and API; certificate pinning to a CA bundle on devices.
- Secrets in Render/Firebase/GitHub secret stores, separate per environment; no `.env` committed; rotation procedure documented (database password, JWT secrets, provider keys, device credentials).

- Database service-role key is never present in browser code or firmware. The API uses a least-privilege Postgres role; the service-role key is used only by the API/worker for Auth admin and Storage operations, if at all.
- Rate limits per IP, user and device (Section 10).

21.2.4 Per-device authentication

- 256-bit random secret per device; stored as SHA-256 hash; constant-time compare; rotation with overlap; revocation within 60 s target.
- Compromise of one device reveals only that device's credential, which can submit that device's events and fetch its config — it cannot read student data or reach other schools.
- Upgrade path (Proposed): per-device asymmetric key pair held in a secure element (e.g. ATECC608-class — Validate), signed requests or mutual TLS, and a short-lived token endpoint.

21.2.5 Audit logs

Sensitive actions are logged append-only with actor, school, entity, before/after, request id and support-grant id (SEC-REQ-007). Examples: login of privileged users, membership/role changes, parent-student link changes, card issue/block, attendance rule changes, corrections, exports, device lifecycle, subscription changes and support access.

21.2.6 Firmware updates

Firmware images are signed offline with a protected signing key (HSM or CI-held key with approvals). Devices verify signature and hash, enforce anti-rollback, and only install matching hardware images (DEV-REQ-009, TEST-REQ-027).

21.2.7 Backups

Backups are encrypted at rest by the database provider; restore permissions are limited to named platform engineers with MFA; backup/restore actions are logged; exports for schools are encrypted in transit and at rest.

21.3 NFC cards: cloning risk and lost-card handling

> **UID-only cards can be cloned**
> Many NFC cards expose a fixed UID that is trivial to read and emulate with inexpensive hardware, and the UID is not a secret. A UID-only deployment therefore proves "someone presented this identifier", not "this student is present". This is acceptable for convenience attendance with parent notification (where a clone is quickly noticed) but not for high-assurance use.

- **Production recommendation:** evaluate cards with cryptographic authentication (challenge-response or dynamic authenticated messages, e.g. DESFire/NTAG 424 DNA class) so that a copied UID is useless (Validate: card cost, reader support, read speed).
- **Lost or suspected-cloned card:** the School Admin blocks it instantly (`nfc_cards.status = BLOCKED/LOST`); the next tap returns `REJECTED_CARD_BLOCKED`; the admin is alerted; a replacement card is issued (the old UID can be retired).
- **Detection heuristics:** same card at two distant devices within seconds, taps while the student is marked LEFT, taps on days the student is on approved leave → flagged for review.
- Cards store no personal data; the mapping UID → student is only in the database.

21.4 Parent-child and teacher-class validation

See Section 7. These checks run in `authorize()` for every route that takes a student or class identifier and in repository queries that filter by the link tables, so even a missing middleware does not allow a data read.

21.5 Secure report downloads

1. Report generation is asynchronous; the file is stored in a private bucket under `school_id/…` and recorded in `report_documents`.
2. A download request goes to the API, which authorises (role, scope, school, subscription) and then issues a signed URL with a short TTL (default 60 s).
3. No public buckets; no guessable paths; URLs are not logged; downloads are audited.

21.6 Data retention, deletion and child privacy

> **Legal validation required**
> The platform processes children's personal data, including location-like arrival/departure events. Applicable law (for deployments in India, the Digital Personal Data Protection Act, 2023 and its Rules, and any education-sector guidance) must be reviewed by counsel before pilot go-live. The items below are engineering defaults, not legal advice.

Topic	Proposed default	Status
Data minimisation	Collect only fields needed for school operations; no student photos or biometrics in V1; no student names on devices	**Proposed**
Lawful basis and consent	School is the data controller; platform is processor under a data-processing agreement; parental consent/notice captured by the school as required	**Validate**
Access control	Least privilege, audited support access, no routine staff access	**Confirmed**
Raw attendance events	Retained for the academic year + 2 years (configurable by contract)	**Validate**
Heartbeats / device logs	30 / 90 days	**Proposed**
Notification delivery logs	180 days	**Proposed**
Audit logs	≥ 2 years	**Validate**
After contract end	Export window (e.g. 60 days), then deletion within a defined period; backups age out per backup retention	**Validate**
Data subject requests	Admin tooling to export/correct/delete a student's data within a school; deletion respects legal retention	**Validate**
Data location	Choose Supabase/Render regions consistent with contractual and legal requirements	**Validate**

21.7 Application security hygiene

- Dependency, container and secret scanning in CI; renovate/dependabot; lockfiles committed.
- Security headers (CSP, HSTS, X-Content-Type-Options, Referrer-Policy, frame-ancestors) on frontend and API.
- Input validation everywhere; parameterised SQL only; output encoding in React; sanitised Markdown.
- Error messages never reveal internals; `404` for foreign-school resources.
- External penetration test and a cross-school/IDOR focused review before pilot (SEC-REQ-013).

Security and privacy requirements

ID	Requirement	Status
SEC-REQ-001	Passwords SHALL be handled by Supabase Auth; a minimum length and leaked-password checking SHALL be configured where the plan supports it.	**Validate**
SEC-REQ-002	Multi-factor authentication SHALL be available and SHOULD be enforced for Super Admin, School Admin and Principal accounts.	**Proposed**
SEC-REQ-003	Every API route SHALL enforce role-based authorisation server-side through shared middleware; routes without an explicit policy SHALL fail closed.	**Confirmed**
SEC-REQ-004	Parent and student access to student data SHALL be validated through student_parents (or the student's own link) on every request.	**Confirmed**
SEC-REQ-005	Teacher access to classes, marks and students SHALL be validated through teacher_class_assignments.	**Confirmed**
SEC-REQ-006	Secrets SHALL live only in host secret stores; no secret SHALL be committed to source control or shipped in a browser bundle.	**Confirmed**
SEC-REQ-007	Sensitive actions (logins of privileged users, role changes, corrections, exports, device lifecycle, support access) SHALL be audit-logged.	**Confirmed**
SEC-REQ-008	Report and document downloads SHALL use short-lived signed URLs issued only after an authorisation check.	**Confirmed**
SEC-REQ-009	Backups SHALL be encrypted at rest and restore access SHALL be limited to named platform engineers.	**Validate**
SEC-REQ-010	The risks of UID-only NFC cards (cloning, relay) SHALL be documented; lost cards SHALL be blockable instantly; production SHALL evaluate cryptographically authenticated cards.	**Proposed**
SEC-REQ-011	Data retention and deletion periods SHALL be defined per contract and applicable law before pilot go-live.	**Validate**
SEC-REQ-012	Children's data SHALL be minimised and processed under a lawful basis, including parental consent where required; legal review of applicable Indian data-protection law (DPDP Act 2023 and Rules) is required.	**Validate**
SEC-REQ-013	Dependency, container and secret scanning SHALL run in CI and an external penetration test SHALL precede pilot go-live.	**Proposed**
SEC-REQ-014	Security headers (CSP, HSTS, X-Content-Type-Options, frame-ancestors) SHALL be set on the frontend host and API.	**Proposed**
SEC-REQ-015	Production hardware SHALL enable flash encryption and secure boot so a captured device cannot be trivially cloned or re-flashed.	**Proposed**
SEC-REQ-016	Authentication endpoints SHALL be protected against brute force by rate limiting and lockout/back-off.	**Proposed**

22. Performance and Scalability

> **Estimates and targets, not results**
> Every number in this section is either arithmetic from stated assumptions or a **target to validate through load tests**. None is a measured result or a guarantee.

22.1 Workload model (per school, ≈ 1,000 students)

Quantity	Assumption / calculation	Estimate
Baseline events per school day	1,000 students × 2 events (arrival + departure)	2,000
Baseline events per year	2,000 × 220 school days	≈ 440,000

Quantity	Assumption / calculation	Estimate
Realistic raw events (repeat taps, classroom readers, rejected cards)	Baseline × 1.5 to 3 (assumption)	≈ 0.66 – 1.32 million / year
Morning arrival peak	70 % of students (700) arrive in a 20-minute window = 1,200 s	≈ 0.6 events/s average
Burst factor	Assume 10× the average within the window (gate clustering)	≈ 6 events/s per school
Device uploads at peak	Batches every few seconds per busy device; 1–5 events per batch	≈ 3–6 requests/s per school
Heartbeats	10 devices × 1/min (all day)	≈ 14,400 requests/day; DB history sampled every 5 min ≈ 2,880 rows/day
Notification deliveries	2 alerts × ≈ 1.5 parents × ≈ 1.5 channels per student	≈ 4,500 deliveries / day; peak ≈ 1 delivery/s
Raw event storage	≈ 0.5 KB per row including indexes (assumption)	≈ 0.2 GB/yr baseline; ≈ 0.3–0.7 GB/yr realistic (Validate)
Dashboard concurrency	100 concurrent staff/parent sessions per school during morning	Mostly cached/paginated reads

22.1.1 Multi-school scaling thought experiment

Schools in the same timezone with the same start time	Peak average (events/s)	Burst at 10× (events/s)	Comment
1	0.6	6	Trivial for a single API instance and small database
10	6	60	Comfortable; verify pool settings
100	58	≈ 580	Requires multiple API instances, pooler tuning and load-test evidence; consider read replica for dashboards

This is why the design uses batch endpoints, short per-event transactions, single-row locks and a separate worker: the workload is small per school and scales mostly by adding API instances. No Kafka, Kubernetes or microservices are justified at these figures (ARCH-REQ-001).

22.2 Database performance measures

- **Indexes:** documented in Section 12; hot paths are (`school_id`, `card_uid`) card lookup, the unique day key, `student_parents` by parent and by student, class rosters, outbox claim.
- **Connection pooling:** transaction-mode pooler; small per-instance pools; peak use ≤ 70 % of limits (PERF-REQ-008).
- **Pagination:** all lists server-paginated (default 25, max 100); cursor pagination for events and audit logs; no unbounded `SELECT`.
- **Batch uploads:** devices send up to 100 events per request; per-event short transactions keep locks brief.
- **Background processing:** notifications, reports, exports, absent finalisation and retention run in the worker, not in request threads.
- **Read-heavy dashboards:** aggregate tables or materialised views refreshed by the worker if live queries over `daily_attendance` prove slow; a read replica is an option after measurement.
- **Caching:** API-side short TTL caches for plan/entitlements and device auth (30–60 s); TanStack Query on the client.

22.3 Initial performance targets (to be validated)

Performance targets

ID	Requirement	Status
PERF-REQ-001	TARGET: p95 server-side latency of POST /device/events/batch ≤ 500 ms for batches ≤ 20 events (p99 ≤ 1.5 s) at morning-peak load.	**Validate**
PERF-REQ-002	TARGET: ≥ 99.9 % of valid events are committed within 5 s of receipt during the morning peak.	**Validate**
PERF-REQ-003	TARGET: p95 ≤ 400 ms for paginated dashboard read endpoints with 100 concurrent users per school.	**Validate**
PERF-REQ-004	TARGET: p95 time from attendance commit to hand-off to the push provider ≤ 30 s.	**Validate**
PERF-REQ-005	TARGET: a device backlog of 5,000 events drains in ≤ 10 minutes without degrading other devices.	**Validate**
PERF-REQ-006	TARGET: a term report run for 1,000 students completes asynchronously within 2 minutes (p95).	**Validate**
PERF-REQ-007	TARGET: a 500-row bulk marks upload validates and commits in ≤ 5 s.	**Validate**
PERF-REQ-008	TARGET: peak database connection use stays ≤ 70 % of the configured pool/pooler limit.	**Validate**

22.4 Load-testing plan

Tool: k6 (HTTP scenarios, thresholds in code) against a staging environment with production-like data volumes (seed ≥ 3 schools × 1,000 students × 1 year of events). Run from a separate load generator, observe database and API metrics during each run.

Scenario	Setup and load profile	Pass criteria	Related
Morning arrival peak	3 schools; 5 devices each; 700 students per school within 20 min with 10× bursts; 1–5 events per batch; plus 100 dashboard users per school	PERF-REQ-001/002/003; no deadlocks; pool ≤ 70 %	TEST-REQ-038
Multiple devices uploading simultaneously	Ramp 5 → 50 devices uploading every 2 s with repeating cards (forces same student-day contention)	One daily row per student-day; zero duplicate notifications; no 5xx	TEST-REQ-011
Large class lists	Rosters and attendance sheets for classes of 60–80 and school-wide lists of 1,000	p95 ≤ 400 ms with pagination	PERF-REQ-003
Parent notification burst	1,000 arrival outbox rows in 20 min; FCM/SMS stubs with injected latency and failures	Hand-off p95 ≤ 30 s; retries/dead-letter behave; no duplicates	PERF-REQ-004, TEST-REQ-021
Attendance report generation	Term attendance + marks report for 1,000 students, run 5 concurrently with live traffic	Completes ≤ 2 min p95; live latency unaffected	PERF-REQ-006
Bulk mark upload	500-row uploads, 10 teachers concurrently	≤ 5 s each; atomic	PERF-REQ-007, TEST-REQ-037
Device reconnection after outage	10 devices reconnect within 1 minute with 500 events each	All recorded; ordering correct; 429/backoff behaves	PERF-REQ-005, TEST-REQ-014
Thousands of queued events together	One device uploads 5,000 events; another device taps normally	Drain ≤ 10 min; other device unaffected	PERF-REQ-005
Soak	24 h at 20 % of peak with random failures (API restarts, DB blips)	No leaks; error rate within budget; queue depth returns to zero	TEST-REQ-019/020

22.5 Capacity triggers

Observation	Response
API CPU > 60 % sustained at peak	Add API instance; confirm pooler headroom
DB CPU > 60 % or connection use > 70 %	Tune queries/indexes; raise plan; consider pooler settings
Dashboard queries dominate	Aggregate tables/materialised views; read replica

Observation	Response
Raw events > ≈ 100 M rows	Evaluate partitioning/archival (mind the unique idempotency key)
Outbox backlog age > 5 min repeatedly	Add worker instance; review provider limits

23. Deployment Architecture

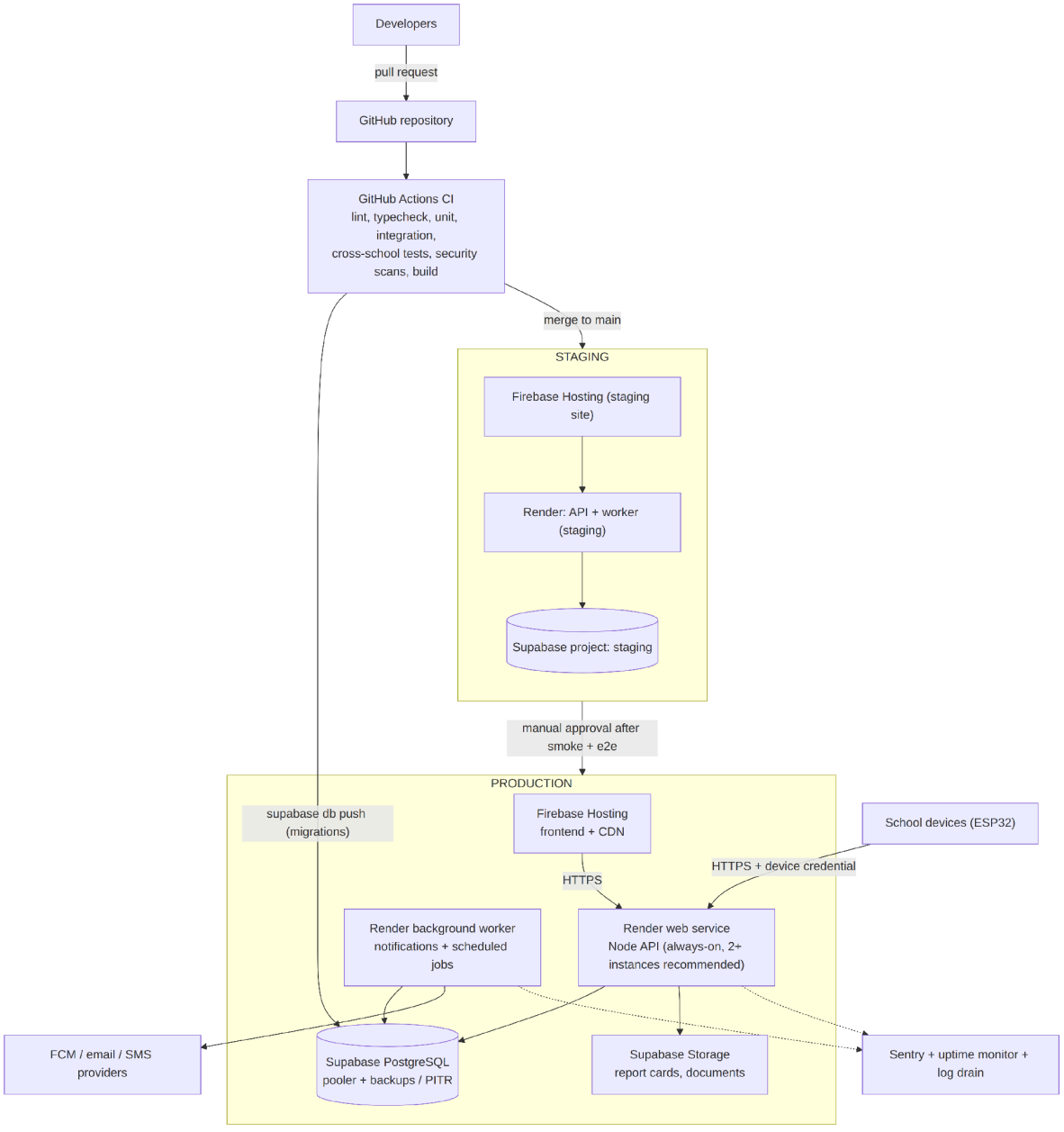


Figure 12 — Deployment architecture

23.1 Environments

Environment	Purpose	Frontend	API / worker	Database	Data
Local	Developer machines	Vite dev server	Node local	`supabase start`	Seeded demo schools
CI	Automated checks	—	Test containers	Ephemeral Postgres	Fixtures

Environment	Purpose	Frontend	API / worker	Database	Data
Staging	Integration, E2E, load tests, restore drills	Firebase staging site	Render staging service + worker	Separate Supabase project	Synthetic only
Production	Live schools	Firebase production site	Render production service (always-on) + worker	Supabase production project	Real data

Staging and production have separate secrets, separate Supabase projects, separate FCM/provider projects and separate device credential sets (ARCH-REQ-007).

23.2 Host responsibilities

Concern	Host	Configuration highlights
Frontend	Firebase Hosting	SPA rewrite to `index.html`; immutable caching for hashed assets; security headers; custom domain with TLS
Backend API	Render web service	Node LTS; health check `/readyz`; ≥ 2 instances recommended in production (validate plan options); autoscale on CPU after load tests; env secrets; custom API domain
Background worker	Render background worker	Same repo, separate start command; no public port; restart on failure; scheduled jobs via internal scheduler with DB-based leader lock (`job_runs`)
Database	Supabase PostgreSQL	Pooler URLs for API/worker; direct URL only for CI migrations; network restrictions where available
File storage	Supabase Storage	Private buckets: `reports`, `materials`, `firmware`
Source control	GitHub	Protected `main`; required reviews and checks; signed commits optional

23.3 CI/CD pipeline

1. **Pull request:** install, lint, type-check, unit tests, API integration tests against ephemeral Postgres (including cross-school sweeps and migration apply), frontend build, bundle secret scan, dependency scan (OPS-REQ-001).
2. **Merge to main:** build artefacts, generate OpenAPI and contract types, publish container/build.
3. **Staging deploy:** apply migrations (`supabase db push`), deploy API and worker, deploy frontend to the staging site, run smoke + Playwright E2E.
4. **Approval gate:** manual approval by a maintainer after staging checks and, for risky releases, a load or restore check.
5. **Production deploy:** migrations (expand phase) → API/worker rolling deploy → frontend → post-deploy smoke tests and alert watch for 30 minutes.
6. **Rollback:** redeploy previous API/worker build and previous Firebase release (one step); database changes follow expand/contract so the previous build still runs against the new schema.

23.4 Configuration and secrets

Setting	Where	Exposure
`VITE_API_BASE_URL`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, FCM web config	Frontend build env	Public by design
`DATABASE_URL` (pooler, role `app_api`), `DATABASE_URL_WORKER` (role `app_worker`)	Render env secrets	Server only
`SUPABASE_SERVICE_ROLE_KEY` (only if Auth admin/Storage admin are needed)	Render env secret on API/worker	Server only; never browser/firmware
`SUPABASE_JWT_SECRET` / JWKS URL	Render env	Server only

Setting	Where	Exposure
FCM service account, email/SMS keys	Worker env secrets	Server only
Firmware signing key	Offline HSM or protected CI secret with approvals	Never on devices
Device secrets	Generated per device; plaintext shown once; hash in DB	Device flash only

23.5 Release management for firmware

- Channels: `dev`, `beta` (pilot school devices), `stable`. Promotion requires bench tests (TEST-REQ-015/016/027).
- Staged rollout by percentage/device group; automatic halt if > 2 % of updated devices report FAILED/ROLLED_BACK (Proposed).
- Release notes list queue-format changes; format changes require migration code and a rollback test.

> **No vendor prices**
> Hosting plan sizes and prices are deliberately not specified. Choose plans after the load tests in Section 22 and confirm limits (connections, PITR, always-on instances) against each vendor's current documentation.

24. Backups and Disaster Recovery

24.1 Objectives

Objective	Initial target	Notes
Database RPO	≤ 5 minutes	Requires PITR/WAL archiving. Without PITR the RPO equals the backup interval (Validate plan features). Device queues mean taps made during the gap are re-sent and re-recorded, which narrows the *effective* attendance data loss.
Database RTO	≤ 4 hours	Restore to a new project + repoint API/worker; validated in drills.
API / worker RTO	≤ 30 minutes	Redeploy from GitHub/Render; stateless.
Frontend RTO	≤ 15 minutes	Redeploy previous Firebase release.
Storage RPO/RTO	Per provider durability; weekly export of critical buckets (Proposed)	Report cards are regenerable; study materials are not.

24.2 Backup design

- Automated daily database backups and PITR where available; backup status monitored (alert if the last backup is older than 26 hours).
- Additional logical export (`pg_dump` of critical tables or full) weekly to separate, encrypted object storage in another account/region (Proposed) to guard against account-level failure.
- On-demand snapshot before risky migrations.
- Backups encrypted at rest; access limited to named engineers with MFA; restores audited (SEC-REQ-009).
- Git holds code, migrations and infrastructure configuration; secrets are recoverable from the secret stores/password manager with documented break-glass access.

24.3 Restore procedure and drills

1. Declare incident; freeze deployments; notify school admins if impact expected.
2. Choose recovery point (latest, or timestamp before corruption).
3. Restore to a **new** database project (never overwrite production in place).
4. Run integrity checks: row counts per table, latest event timestamps, FK/unique constraint checks, RLS policy assertions, a sample cross-school test.
5. Point API/worker `DATABASE_URL` to the restored project; run smoke tests; re-enable devices (they resend queued events; idempotency prevents duplicates).
6. Reconcile: compare device `last_sequence` and `sync-ack` data against restored events to find any gap; re-process or review.
7. Post-incident review; update runbooks.

Restore drills run at least quarterly and before pilot go-live, in staging, recording measured RTO and data-loss window (TEST-REQ-026, OPS-REQ-005).

24.4 Disaster scenarios

Scenario	Impact	Response
Database provider outage	API returns RETRY_LATER; devices queue; web read/write fails	Wait/vendor status; if prolonged restore elsewhere; devices drain later
Bad migration / data corruption	Wrong data	PITR to before change into new project; replay devices
Accidental deletion by a user	Missing records	Status-based retirement prevents most; restore from PITR for specific tables into scratch DB and copy back
Region outage (Render or Supabase)	Service down	Stateless API redeploy in another region if DB reachable; otherwise wait/restore
Account compromise	Secrets exposed	Rotate all secrets, revoke sessions, rotate device credentials, forensic audit-log review

24.5 Incident-response checklists

Database downtime

- Confirm via provider status and `/readyz`; check connection saturation and failed migrations.
- Post incident status; confirm devices are queueing (RETRY_LATER / 5xx) and storage-full risk is low.
- If saturation: reduce worker concurrency, scale pooler/plan, kill runaway queries.
- If provider outage: wait; if corruption: execute restore procedure.
- After recovery verify event flow, outbox drain, absent-finalisation state, then close with review.

Backend (API) downtime

- Check Render service health, last deploy and Sentry.
- Rollback to previous build if deploy-related.
- Verify devices are retrying with backoff (no storms) and rate limiter is healthy.
- After recovery watch error rate, queue depth and duplicate-event metrics.

Device failure

- Watchdog alert: confirm which device and location; check last heartbeat, RSSI, reset reason, storage and RTC status.
- Remote steps: config refresh, OTA rollback; on-site steps: power cycle, network check, reader replacement.
- If device lost/stolen: revoke credentials immediately; block affected cards if relevant.

- After restoration verify sync-ack gap check and review quarantined/flagged events.

Notification failure

- Check outbox backlog age, DEAD count and provider status pages.
- Pause the failing channel (circuit breaker); keep outbox rows; switch parents to alternative channels if configured.
- After recovery let retries drain; confirm stale-suppression behaved and no duplicates were sent.
- Inform schools if a notification gap exceeded the agreed service level.

25. Monitoring and Logging

25.1 Signals

Area	Metrics and logs	Source
API	Request rate, error rate (4xx/5xx), latency p50/p95/p99 by route, rate-limit hits, auth failures	Structured logs, host metrics, Sentry performance
Attendance ingest	Events/s, outcomes by code, RETRY_LATER rate, duplicate events, quarantined events, replay count, lock wait time	Application metrics/logs, DB
Devices	Heartbeat age, online count, pending events, storage %, RTC skew, reader status, firmware versions, sequence gaps	`devices`, `device_heartbeats`, `device_logs`
Worker / notifications	Outbox backlog age, deliveries by status/channel, retries, DEAD, provider latency/errors	`notification_*` tables, worker logs
Database	Connections vs limit, CPU, memory, disk, WAL/backups, slow queries, lock waits, long transactions	Provider dashboard, `pg_stat_statements`
Frontend	JS errors, web vitals, failed API calls	Sentry / analytics
Security	Failed logins, privileged actions, support grants, cross-school denials, revoked-device attempts	Audit logs, API logs

25.2 Alerts (initial thresholds — tune after load tests)

Alert	Condition	Severity	Notify
API unavailable	Uptime check fails 2 consecutive minutes	Critical	On-call page
API error rate	5xx > 2 % for 5 min	High	On-call
Database capacity	Disk > 80 %, connections > 70 %, CPU > 80 % for 10 min	High	On-call
Backup stale	Last successful backup > 26 h	High	On-call
Device heartbeat loss	ACTIVE device without heartbeat > 3 min	Medium	School admin (email/push)
Device offline long	> 15 min during school hours	High	Platform support + school admin
Failed synchronisation	Batch failures for a device > 10 min, or sequence gap detected	High	Platform support
Notification backlog	Oldest PENDING outbox row > 5 min	High	On-call
Dead notifications	Any DEAD row	Medium	Platform support
Quarantine spike	> 5 quarantined events/device/day	Medium	Platform support
Storage-full risk (device)	Queue ≥ 80 %	Medium	School admin
Absent finalisation deferred past hard limit	Provisional ABSENT written	Medium	School admin

25.3 Logging standards

- JSON logs to stdout; the host ships them to a log drain with 30-day retention (Proposed); audit logs are in the database.
- Every log line carries `request_id`; device events add `device_code`, `event_id`; no PII bodies; card UIDs hashed; secrets redacted.
- Business-event log names are stable (e.g. `attendance.event.recorded`, `outbox.delivery.dead`) so alerts and dashboards do not depend on message text.

25.4 Uptime and dashboards

- External uptime monitor on `/healthz` (API) and the frontend URL from at least two regions; status page for schools (Proposed).
- Dashboards: *Morning Peak* (ingest rate, latency, queue depth, outbox), *Devices* (fleet map by school), *Database*, *Notifications*, *Security*.
- On-call rota and escalation documented; severity definitions and response targets agreed with partner schools in the support terms.

Operations requirements

ID	Requirement	Status
OPS-REQ-001	CI SHALL run lint, type-check, unit, integration (including cross-school), build and security scans on every pull request.	**Proposed**
OPS-REQ-002	Deployments SHALL go to staging first and be promoted to production with manual approval; rollback SHALL be possible in one step.	**Proposed**
OPS-REQ-003	Initial targets: RPO ≤ 5 minutes (needs PITR) and RTO ≤ 4 hours for the database; both SHALL be validated by restore drills.	**Validate**
OPS-REQ-004	Alerts SHALL exist for database capacity, device heartbeat loss, notification backlog, failed event synchronisation and API error rate.	**Confirmed**
OPS-REQ-005	Restore drills SHALL be performed at least quarterly and before pilot go-live.	**Proposed**
OPS-REQ-006	Runbooks SHALL exist for database downtime, API downtime, device failure and notification failure.	**Proposed**
OPS-REQ-007	API and worker SHALL expose liveness and readiness endpoints used by the host and uptime monitor.	**Proposed**

26. Project Directory Structure

A single GitHub monorepo (npm/pnpm workspaces) keeps contracts, migrations, services and firmware versioned together while each deployable keeps its own build and release pipeline.

Repository layout

```
vidyalayaa/
  apps/
    web/                    # React + Vite frontend (Section 9)
    api/                    # Express API (Section 10)
    worker/                 # background worker: outbox + scheduled jobs
  packages/
    contracts/              # Zod schemas, OpenAPI generation, event codes, enums (NO secrets)
    ui-tokens/              # design tokens shared by web and docs (optional)
    config/                 # eslint, tsconfig, prettier presets
  db/
    migrations/             # NNNN_*.sql forward-only migrations
    seeds/                  # demo schools for dev/test only
    tests/                  # schema assertions, RLS/FK/unique tests
  firmware/
    nfc-device/             # ESP32 firmware (PlatformIO / ESP-IDF), partitions, tests
    tools/                  # provisioning tool, signing scripts (keys NOT stored here), emulator
  tests/
```

```
    e2e/                      # Playwright
    load/                     # k6 scenarios
    security/                 # cross-school sweeps, header and secret scans
    chaos/                    # fault-injection scripts (DB, network, provider)
  infra/
    firebase/                 # firebase.json (headers, rewrites), .firebaserc
    render/                   # render.yaml (services, worker, env groups without secret values)
    supabase/                 # config.toml, policies notes
    monitoring/               # alert definitions, dashboards as code
  docs/
    architecture/             # this SAS, ADRs (decision records), diagrams (.mmd/.dot sources)
    runbooks/                 # incident, restore, rotation procedures
    api/                      # generated OpenAPI + device protocol guide
  .github/workflows/          # ci.yml, deploy-staging.yml, deploy-prod.yml, firmware.yml
  package.json  pnpm-workspace.yaml  .nvmrc  .env.example  README.md
```

Area	Contents and notes
Frontend	`apps/web` as specified in Section 9; imports types and Zod schemas from `packages/contracts`.
Backend API	`apps/api` as in Section 10; builds with `tsc`; exposes `app.ts` for tests.
Background worker	`apps/worker`: `main.ts` starts the outbox loop and a scheduler that runs job modules from `apps/api/src/jobs` (shared) under a DB lock in `job_runs`.
Database migrations	`db/migrations` with a naming rule `NNNN_verb_subject.sql`; every migration reviewed for locking and RLS.
Device firmware	`firmware/nfc-device`; unit tests on host (queue, CRC, sync logic) and hardware-in-the-loop tests; reads generated constants.
Automated tests	Unit tests live beside code; cross-cutting suites in `tests/`.
Deployment configuration	`infra/*` plus GitHub workflows; no secret values committed.
Documentation	`docs/*`; diagram sources kept as Mermaid / DOT so they can be edited and re-rendered.

26.1 Sharing API contracts without sharing secrets

- `packages/contracts` contains **only** public shape information: Zod schemas, enumerations (`attendance_status`, `outcome`, `device_action`), limits (max batch size) and example payloads. It contains no keys, URLs with credentials or environment values.
- The API imports the schemas to validate; the web app imports them for forms and types; CI generates `openapi.json` and a **C header** (`contracts_generated.h`) with event codes and limits for the firmware, plus a human-readable device protocol guide.
- A CI check regenerates artefacts and fails if they differ from the committed ones, so web, API and firmware cannot drift.
- Secrets are injected per environment by the host (Render/Firebase/GitHub) and, for devices, at provisioning time; they are never part of any shared package or generated file.

27. Development Roadmap

Phases are ordered by dependency. Effort is not estimated here (no invented durations); the team should size each phase after Phase 1 spikes. Each phase ends only when its acceptance criteria and tests pass.

27.1 Phase 1 — Project setup, database schema, authentication and school isolation

Goal: A secure skeleton with two isolated demo schools.

Tasks

- Create monorepo, CI, environments (staging/prod), Supabase projects, secrets.
- Migrations for identity, schools, memberships, roles/permissions, academic basics; RLS template and composite-FK helpers.
- Supabase Auth integration, `/me`, tenant middleware, `withTenantTx`, authorisation middleware, error/logging/rate-limit middleware.
- Frontend shell: auth pages, layouts, protected routes, API client.
- Cross-school test harness and schema-assertion tests.

Dependencies	Deliverables
None	Running staging environment; migrations pipeline; login for each role; seeded Schools A and B; CI gates.

Acceptance criteria	Tests required
Users sign in and see only their school; automated cross-school tests pass for every route/table; migrations apply from empty.	TEST-REQ-001, 002, 003, 036, 040, 045

27.2 Phase 2 — School admin, teachers, students, parents and academic records

Goal: Complete the non-attendance SaaS functions.

Tasks

- School profile, academic sessions, classes, subjects, enrolments.
- Students, parents, teachers CRUD, `student_parents`, `teacher_class_assignments`, bulk import.
- Exams, marks (incl. bulk upload), homework/materials, timetable, announcements, fees.
- Report cards (async) and secure downloads; principal/teacher/parent/student dashboards.
- Audit logging for sensitive actions; subscription entitlement middleware (feature flags only).

Dependencies	Deliverables
Phase 1	Role portals with academic features; report generation; audit log viewer.

Acceptance criteria	Tests required
Teacher edits only assigned classes; parent sees only own children; marks upload atomic; WCAG checks on core flows.	TEST-REQ-004, 005, 006, 007, 037, 039, 041

27.3 Phase 3 — NFC device registration and basic attendance

Goal: One device, one school, end-to-end taps.

Tasks

- Device inventory, credentials, claim and location assignment; device auth middleware.
- Cards (assign/block), rule versions, calendar/holidays.
- Event ingest endpoint, attendance engine (movement + windows + time trust), daily projection, day-open and absent jobs.
- Firmware v0: read card, RTC, durable queue, batch upload, heartbeat, config.
- Admin UI: devices, cards, rules, daily attendance, corrections.

Dependencies	Deliverables
Phase 2 (students/classes), Phase 1	Working prototype device recording attendance; engine unit test suite; corrections workflow.

Acceptance criteria	Tests required
Taps at one device produce correct daily states under boundary tests; unknown/blocked cards rejected; power-loss queue tests pass.	TEST-REQ-009, 010, 015, 017, 018, 029, 032, 033, 034, 042, 043

27.4 Phase 4 — Multiple devices per school, concurrency control, idempotency and offline synchronisation

Goal: Prove correctness with 5 concurrent devices and outages.

Tasks

- Idempotent claim, row-locked projection, replay for out-of-order events, sequence-gap detection, `sync-ack`.
- Multi-location rules (ENTRY/EXIT/BIDIRECTIONAL/CLASSROOM) and movement scenarios.
- Device watchdog, health UI, revocation within target, credential rotation.
- Firmware: backoff/jitter, corruption handling, storage thresholds, signed OTA with rollback.
- Concurrency and chaos test suites.

Dependencies	Deliverables
Phase 3	Five-device school simulation; concurrency harness; OTA pipeline.

Acceptance criteria	Tests required
1,000-iteration simultaneous-tap tests yield one daily row and one notification; same-event retries safe; DB/API outages lose nothing.	TEST-REQ-011, 012, 013, 014, 016, 019, 020, 025, 027, 028, 030, 031

27.5 Phase 5 — Parent notifications and background workers

Goal: Reliable alerts with retries and audit.

Tasks

- Outbox writer in the attendance transaction; worker claim/expand/send/retry; FCM, email, SMS adapters.
- Preferences, push tokens, multiple parents, templates, staleness suppression, per-day caps.
- Parent mobile-first notification centre and history; admin failed-notification list.
- Alerts for backlog and DEAD notifications.

Dependencies	Deliverables
Phase 3–4	Working push/email/SMS pipeline with dashboards.

Acceptance criteria	Tests required
Provider failures never lose attendance; no duplicate alerts under replay and parallel workers.	TEST-REQ-021, 022, 044

27.6 Phase 6 — Subscription billing, monitoring and reports

Goal: Commercial readiness.

Tasks

- Plans, subscriptions, lifecycle job, grace/suspension policy, entitlement enforcement, billing webhook (provider chosen).
- Platform admin console: schools, plans, device inventory, health, support grants.
- Monitoring/alerting/log drain, uptime checks, status page.
- Advanced reports and exports; usage metering.

Dependencies	Deliverables
Phase 1–5	Billing integration; platform console; monitoring stack; runbooks.

Acceptance criteria	Tests required
All subscription states behave per Section 20 with devices still ingesting; alerts fire in drills.	TEST-REQ-008, 023, 024

27.7 Phase 7 — Security testing, performance testing, backup recovery and pilot deployment

Goal: Production readiness for the first partner school.

Tasks

- External penetration test and IDOR/cross-school review; fix findings.
- Load tests (Section 22) and tuning; soak test.
- Backup/PITR configuration and restore drill; incident tabletop exercises.
- Hardware field survey, device installation at pilot school, staff/parent onboarding, legal/privacy sign-off.
- Go-live checklist (Section 30) and hyper-care period.

Dependencies	Deliverables
Phase 1–6	Pilot go-live; measured performance and recovery figures; signed checklist.

Acceptance criteria	Tests required
Section 30 checklist complete; PERF targets met or documented exceptions; measured RTO/RPO recorded.	TEST-REQ-026, 038 and the full suite

28. Testing Strategy

> **Happy path is not production-ready**
> A system must not be declared production-ready merely because tapping a card produces a green light and a notification. Production readiness requires the failure, concurrency, isolation, recovery and security tests below to pass.

28.1 Test pyramid and tooling

Level	Scope	Tools
Unit	Attendance engine (pure), time trust, rule resolution, permission checks, firmware queue/CRC/sync on host	Vitest, firmware unit framework (Unity/GoogleTest)
Integration	API + real Postgres: routes, RLS, composite FKs, transactions, outbox, worker	Vitest + Supertest + Testcontainers / `supabase start`
Concurrency / chaos	Parallel taps, retries, kill DB/network/providers	Custom harness, Toxiproxy-style fault injection, k6
End-to-end	Role journeys in the browser, auth expiry, accessibility	Playwright + axe
Hardware-in-the-loop	Power loss, OTA, RTC, reader faults	Test rig, relay power switching
Load / soak	Section 22 scenarios	k6
Security	Cross-school sweeps, header scan, secret scan, pen test	Custom suite, ZAP/Burp, gitleaks/trufflehog
DR	Restore drills	Runbook-driven

28.2 Test cases

Each test case is a requirement (`TEST-REQ-NNN`) and is traced to the requirements it verifies in Appendix A.

ID	Scenario	Method	Expected result	Level	Verifies
TEST-REQ-001	School A user reads School B records	As a School A admin, call GET/PATCH/DELETE on every resource type using valid School B IDs. Repeat with School B IDs in query, body and path.	Always 404; zero data returned; attempt logged. No difference in response timing/shape from a non-existent ID.	Integration (CI)	TEN-REQ-003, TEN-REQ-005, SEC-REQ-003
TEST-REQ-002	School A writes reference School B parent rows	Directly in SQL (as app role), insert enrollment/marks/attendance rows pairing School A school_id with School B student/class IDs.	Composite foreign key violation on every attempt.	DB integration	TEN-REQ-002
TEST-REQ-003	RLS is the last line of defence	Run queries as the API role with no tenant setting, then with School A setting while selecting School B rows, including a deliberately buggy query without WHERE school_id.	Zero rows (or error) without setting; only School A rows with setting; INSERT of foreign-school rows rejected by WITH CHECK.	DB integration	TEN-REQ-004, DB-REQ-007
TEST-REQ-004	Parent reads an unrelated student	As Parent P (linked to student S1), request S2 profile, marks, attendance, fees, homework, report downloads.	404 on all; no data; audit log entry.	Integration	SEC-REQ-004, FE-REQ-002
TEST-REQ-005	Parent-child link removed	Remove the student_parents link while the parent has an active session, then repeat requests.	Access denied on the very next request; cached UI data not refetchable; push notifications for the child stop.	Integration	SEC-REQ-004
TEST-REQ-006	Teacher accesses an unassigned class	As Teacher T (assigned to class 8A) write marks, view roster and post homework for class 8B.	403/404; no rows changed; audit entry.	Integration	SEC-REQ-005
TEST-REQ-007	Student reads another student	As Student S1 request S2 data and write endpoints.	Denied; students have read-only access to self where enabled.	Integration	SEC-REQ-004
TEST-REQ-008	Super admin access to student data	As platform admin request student records (a) without a grant, (b) with an active grant, (c) after grant expiry.	(a) denied + audited; (b) allowed, scoped, audited; (c) denied.	Integration	TEN-REQ-007, SEC-REQ-007
TEST-REQ-009	Repeated tap at the same device	Tap the same card 5 times within 3 s at an ENTRY device.	5 raw events stored; 1 state change (arrival); 1 outbox row; later taps return REPEAT_TAP_IGNORED.	Integration	ATT-REQ-008
TEST-REQ-010	Repeated tap after debounce and location rules	While ARRIVED: tap ENTRY reader after 5 min; tap BIDIRECTIONAL reader at 10:00; tap CLASSROOM reader; tap EXIT reader at 14:05; then tap ENTRY again (re-entry).	ENTRY → ignored+flag; BIDIRECTIONAL outside window → ignored+flag; CLASSROOM → presence evidence only; EXIT → departure; re-entry follows policy and is audited.	Integration	ATT-REQ-007, ATT-REQ-013, ATT-REQ-016

ID	Scenario	Method	Expected result	Level	Verifies
TEST-REQ-011	Simultaneous taps from two devices	Fire two parallel batch requests (DEV001, DEV002) carrying the same student's card within 300 ms; repeat 1,000 times with random interleaving and pool sizes.	Exactly 2 raw events, 1 daily_attendance row, 1 arrival outbox row, 1 PRESENT/LATE decision, no deadlocks, no 5xx.	Concurrency/integration	ATT-REQ-006, DB-REQ-003, DEV-REQ-015
TEST-REQ-012	Same event uploaded repeatedly	Send the same event_id 10 times sequentially and 10 times in parallel.	One event row; identical stored result; later responses status DUPLICATE_EVENT; no extra notification.	Integration	ATT-REQ-005, DB-REQ-004
TEST-REQ-013	Network drops after commit, before response	Kill the connection after the DB commit but before the HTTP response; device retries.	Retry returns DUPLICATE_EVENT with original outcome; device deletes the event; no duplicate side effects.	Chaos/integration	ATT-REQ-005, ATT-REQ-014
TEST-REQ-014	Long outage and backlog synchronisation	Device offline 24 h accumulating 3,000–5,000 events; reconnect while other devices stay active.	All events recorded in order, no loss, no duplicates; backlog drains within PERF target; stale notifications suppressed per policy; other devices unaffected.	Load/soak	DEV-REQ-008, DEV-REQ-012, NOT-REQ-006, PERF-REQ-005
TEST-REQ-015	Power loss during queue write	Cut power at random points while appending and while purging the queue (fault-injection rig or emulator), 500 iterations.	No acknowledged event lost; torn record detected by CRC and quarantined; queue mounts; sequence continues.	Hardware/firmware	DEV-REQ-004, DEV-REQ-010
TEST-REQ-016	Power loss during OTA	Interrupt power at several points of download, flash and first boot.	Device boots previous firmware (or rolls back) and keeps its queue; OTA status reports failure.	Hardware/firmware	DEV-REQ-009
TEST-REQ-017	Incorrect RTC time	Set RTC to 2000-01-01, to +2 days future, and clear its oscillator-stop-flag handling; tap cards.	Events marked SUSPECT or RECONSTRUCTED; SUSPECT events quarantined; time-drift alert raised; rules never use upload time.	Integration/firmware	ATT-REQ-003, ATT-REQ-004
TEST-REQ-018	Device restart with pending events	Reboot with 200 pending events; also brown-out mid-upload.	New boot_id, sequence continues, pending events uploaded exactly once; none lost.	Firmware integration	DEV-REQ-004
TEST-REQ-019	Database failure during processing	Stop the DB / kill connections during batch processing.	Affected events return RETRY_LATER (device_action RETAIN_AND_RETRY); no partial state; full recovery after DB returns.	Chaos	ATT-REQ-014
TEST-REQ-020	Database failover / pooler restart	Restart pooler / force failover while traffic flows.	API reconnects automatically; health endpoint reflects degraded then healthy; no stuck connections.	Chaos	OPS-REQ-007
TEST-REQ-021	Notification provider failure	Make FCM/email/SMS fail (timeouts, 5xx, invalid token).	Attendance saved; deliveries retry with backoff; DEAD after max attempts with alert; other channels/parents unaffected; outbox survives worker restart.	Integration/chaos	NOT-REQ-001, NOT-REQ-003, NOT-REQ-005, ARCH-REQ-010
TEST-REQ-022	Duplicate notification prevention	Replay events, run out-of-order replay of the day, run worker concurrently (2 instances).	Exactly one arrival and one departure notification per student per episode.	Integration	ATT-REQ-008, NOT-REQ-002
TEST-REQ-023	Expired subscription	Move a school to PAST_DUE, SUSPENDED_READONLY and EXPIRED via time travel; use UI, API and devices.	Data never deleted; devices keep recording; gated features return stable error code; read access per policy.	Integration	SUB-REQ-001, SUB-REQ-003, SUB-REQ-004
TEST-REQ-024	Grace-period state machine	Simulate payment failure, retry success, and no payment through every transition.	States and timings follow Section 20; each change audited; notifications to school admin sent.	Integration	SUB-REQ-002, SUB-REQ-005
TEST-REQ-025	Device revocation	Revoke DEV003 while it has pending events and active traffic.	Requests rejected 401 within 60 s; other devices unaffected; events on device preserved; alert raised; reinstatement requires new credential.	Integration/E2E	DEV-REQ-003
TEST-REQ-026	Backup restoration	Restore latest backup / PITR to a new project; run integrity checks, RLS policy checks and smoke tests; record duration and data-loss window.	Measured RTO/RPO recorded and compared with targets; application runs against the restored DB.	DR drill	DB-REQ-008, OPS-REQ-003, OPS-REQ-005, SEC-REQ-009
TEST-REQ-027	Unauthorised firmware update	Offer unsigned, wrongly signed, tampered-hash, downgraded-version and wrong-hardware images.	All rejected before activation; device reports failure; current firmware keeps running.	Firmware	DEV-REQ-009, SEC-REQ-015
TEST-REQ-028	Device spoofs another school	DEV001 (School A) sends events with school_id of School B, another device's device_id and a foreign student_id.	Foreign IDs ignored/rejected; events stored (if valid) only under School A and DEV001; mismatching device_id → 403.	Integration	TEN-REQ-003, DEV-REQ-002, API-REQ-002
TEST-REQ-029	Lost or cloned card	Block a card, then tap it at several devices; try issuing the same UID to a second student.	REJECTED_CARD_BLOCKED, admin alert, no attendance change; duplicate active assignment prevented by unique indexes.	Integration	DB-REQ-012, SEC-REQ-010

ID	Scenario	Method	Expected result	Level	Verifies
TEST-REQ-030	Out-of-order events across devices	Upload an EXIT event (Gate B) before the earlier ARRIVAL (Gate A) arrives.	Replay yields ARRIVED then LEFT in event-time order; attendance status retained; one arrival and one departure notification.	Integration	ATT-REQ-001, ATT-REQ-003, ATT-REQ-007, ATT-REQ-009
TEST-REQ-031	One device offline, others online	Cut DEV003 network; keep DEV001/002 active.	DEV003 marked OFFLINE after threshold with alert; others normal; absent finalisation deferral engaged; later merge is correct.	Integration	DEV-REQ-001, DEV-REQ-006, DEV-REQ-013
TEST-REQ-032	Window boundaries	Taps at 07:59:59.999, 08:00:00.000, 08:29:59.999, 08:30:00.000, 11:59:59.999, 12:00:00.000 school-local; also server in a different timezone.	Start-inclusive/end-exclusive classification exactly as configured; independent of server timezone.	Unit	ATT-REQ-002
TEST-REQ-033	Holiday tap	Tap on a configured holiday and on a Sunday.	Event stored with NO_SCHOOL_DAY outcome; no daily row changes; no notification.	Integration	ATT-REQ-011
TEST-REQ-034	Absent finalisation and retraction	Run job at cutoff+grace with some students lacking taps; then sync a delayed on-time arrival.	Absent set only after grace and device-online check; late arrival retracts ABSENT, sets PRESENT/LATE, sends correction notice once.	Integration	ATT-REQ-010
TEST-REQ-035	Rate and payload limits	Send oversize body, 101-event batch, burst above rate limit, repeated bad logins.	413/422/429 with stable codes and Retry-After; no processing of rejected payloads; device backs off.	Integration	API-REQ-004, API-REQ-005, DEV-REQ-007, SEC-REQ-016
TEST-REQ-036	Session expiry in the browser	Expire access token mid-form; revoke session server-side; offline then online.	Silent refresh; otherwise redirect with return path and unsaved form draft retained.	E2E	FE-REQ-004
TEST-REQ-037	Bulk marks upload	Upload a 500-row file with valid and invalid rows; repeat the same upload.	Atomic per policy; row-level error report; idempotent on repeat; within PERF target.	Integration /load	PERF-REQ-007
TEST-REQ-038	Morning-peak load test	k6 scenario: multi-school, 5–10 devices/school, arrival burst, parallel dashboard traffic and notification burst (Section 22).	Targets PERF-REQ-001 to 008 met or a documented remediation plan is produced.	Load	PERF-REQ-001 to PERF-REQ-008
TEST-REQ-039	Report download security	Use another user's signed URL, an expired URL, and a URL for a different school.	Denied; URLs expire quickly; no listing of storage paths.	Integration	SEC-REQ-008
TEST-REQ-040	Secret leakage scan	CI scans built bundle, repository history and firmware images for service-role keys, device secrets, private keys.	No findings; build fails on detection.	CI	ARCH-REQ-004, FE-REQ-006, SEC-REQ-006
TEST-REQ-041	Accessibility	Automated axe checks plus manual keyboard/screen-reader pass on login, dashboards, attendance and marks entry.	No critical violations; keyboard-operable flows.	E2E/manual	FE-REQ-003
TEST-REQ-042	Exam-day and class-specific rules	Configure exam-day and class-level rule versions; tap on those days; change a rule mid-year.	Correct precedence; historical rows keep original rule_version_id.	Integration	ATT-REQ-002, ATT-REQ-015
TEST-REQ-043	Manual correction workflow	Teacher requests correction, principal approves/rejects, admin applies directly (if allowed).	Daily record updated with audit trail; raw events unchanged; parent correction notice sent once.	Integration	ATT-REQ-012, SEC-REQ-007
TEST-REQ-044	Multiple parents, partial failure	Student with two parents; one has invalid push token and email-only preference.	Each parent handled independently; invalid token disabled; other parent notified.	Integration	NOT-REQ-004
TEST-REQ-045	Migrations and policy existence	Apply all migrations from empty DB and from the previous release; run schema assertions (RLS enabled/forced, composite FKs, unique constraints, indexes).	Clean apply; assertions pass; seed data loads.	CI	ARCH-REQ-002, ARCH-REQ-007, TEN-REQ-001, DB-REQ-001, DB-REQ-002, DB-REQ-006

28.3 Quality gates

- Pull requests cannot merge if cross-school, schema-assertion, unit or integration tests fail.
- Releases to production require staging smoke + E2E pass.

- The pilot cannot start until TEST-REQ-011, 012, 013, 014, 015, 019, 021, 025, 026, 027, 028 and 038 pass and the pen-test critical/high findings are closed.

29. Risks and Mitigation

ID	Risk	Likelihood	Impact	Mitigation	Owner
R1	Cross-school data leakage through a missed filter	Medium	Critical	Defence in depth (Section 8); route/table sweeps; code review checklist; pen test	Backend lead
R2	Lost or duplicated attendance under concurrency/retries	Medium	High	Unique keys, row locks, idempotent claim, chaos tests, per-event transactions	Backend lead
R3	Devices lose events on power cut or flash wear	Medium	High	CRC queue, durable-before-feedback, FRAM option, power-fail rig tests	Firmware lead
R4	RTC drift or wrong clock misclassifies attendance	Medium	Medium	Trust classification, `uptime_ms` reconstruction, NTP discipline, quarantine, alerts	Firmware / backend
R5	Notification provider outage or SMS regulatory blocking	Medium	Medium	Outbox, retries, multiple channels, early DLT/sender registration	Product / backend
R6	Card cloning / card sharing	High	Medium	Documented risk, anomaly flags, block lost cards, authenticated cards in production	Product / security
R7	Wi-Fi coverage poor at gates	High	Medium	Site survey, queueing, Ethernet/PoE option, RSSI alerts	Deployment
R8	Child-data privacy/legal non-compliance	Medium	Critical	Legal review, minimisation, retention policy, DPA, consent flows	Product owner / counsel
R9	Hosting limits (connections, PITR, always-on) differ from assumptions	Medium	High	Validate vendor docs early; load tests; plan upgrade path	DevOps
R10	Single-region outage	Low	High	Device queues, restore drills, documented rebuild, consider multi-region later	DevOps
R11	Scope creep delays core attendance	High	Medium	Phased roadmap; strict V1 scope; ADRs for changes	Product owner
R12	Small-team key-person dependency	Medium	Medium	Documentation, runbooks, modular code, pair reviews	Management
R13	Unauthorised firmware or credential theft from device	Low	High	Secure boot, flash encryption, scoped credentials, revocation, signed OTA	Firmware / security
R14	Performance targets not met at scale	Medium	Medium	Load testing, indexes, scaling levers (Section 22)	Backend / DevOps

30. Production Readiness Checklist

Mark each item with evidence (link to test run, document or screenshot). The pilot go-live requires every item to be complete or formally waived by the product owner and security reviewer.

Isolation and security

Item	Done	Evidence
TEST-REQ-001/002/003 green in CI on every merge	☐	
Pen test completed; critical/high issues closed	☐	
Secrets scan clean; service-role key absent from bundle	☐	
MFA enforced for Super Admin; recommended for Admin/Principal	☐	
Security headers verified	☐	
Support access grant flow tested and audited	☐	

Attendance correctness

Item	Done	Evidence
Concurrency, idempotency, out-of-order and boundary tests green	☐	
Rule versions configured and reviewed by the pilot school	☐	
Absent finalisation and retraction verified	☐	
Manual correction workflow signed off by Principal	☐	

Devices

Item	Done	Evidence
Power-loss and OTA tests passed on production hardware	☐	
Signed firmware; secure boot / flash encryption enabled on production units	☐	
Device credentials provisioned, unique, recorded; revocation drill done	☐	
Site survey done; Wi-Fi coverage adequate at every device location	☐	
Spare units and replacement procedure available	☐	

Notifications

Item	Done	Evidence
Providers configured in production (FCM, email, SMS incl. regulatory registration)	☐	
Failure/retry/dead-letter behaviour verified; alerts routed	☐	
Parent consent/notice and preferences flow approved	☐	

Data and recovery

Item	Done	Evidence
Backups and PITR enabled; last backup alert in place	☐	
Restore drill performed; RTO/RPO measured and recorded	☐	
Retention and deletion policy approved by counsel	☐	
Migration rollback / expand-contract procedure documented	☐	

Performance and capacity

Item	Done	Evidence
Load tests run; results documented against PERF-REQ targets	☐	
Pool/connection headroom verified; autoscaling or capacity plan set	☐	
Production plans sized from measurements	☐	

Operations

Item	Done	Evidence
Monitoring, alerts and on-call rota live; fire-drill performed	☐	
Runbooks for DB, API, device and notification incidents	☐	

Item	Done	Evidence
Status page and school-admin communications prepared	☐	
Support process and escalation agreed	☐	

Commercial

Item	Done	Evidence
Subscription plans, limits and grace policy configured; contract terms aligned	☐	
Billing webhook tested	☐	
Offboarding/export procedure tested	☐	

Accessibility and usability

Item	Done	Evidence
Axe checks and manual keyboard/screen-reader pass on core flows	☐	
Parent portal tested on common low-end phones	☐	
Admin/teacher training materials ready	☐	

31. Final Engineering Recommendations

1. **Build the attendance engine first as a pure library with an exhaustive test table**, then wrap it in the transaction. Most bugs in this domain are rule-boundary and ordering bugs, which are cheap to find in pure functions.
2. **Make concurrency tests a permanent part of CI**, not a one-off exercise: the simultaneous-tap and duplicate-upload suites protect the most important guarantee of the product.
3. **Keep PostgreSQL authoritative and simple**: unique constraints, composite FKs, FORCED RLS, forward-only SQL migrations. Resist adding a second datastore or queue until measured need.
4. **Prove tenant isolation continuously** with route and table sweeps; add a new table or route only through the helpers that apply RLS and tests.
5. **Treat devices as untrusted reporters**: server-derived school, server-side decisions, per-device revocable credentials, signed firmware, no personal data on devices.
6. **Decide the card technology early** (UID-only vs authenticated cards) because it affects reader choice, cost and the security claim made to schools. Start with UID for the pilot, with the risk documented, while piloting authenticated cards in parallel.
7. **Invest in field reliability**: site surveys, Ethernet/PoE where Wi-Fi is weak, protected power, FRAM or well-tested flash, and a clear "storage full" behaviour.
8. **Validate vendor assumptions in week one**: Supabase plan features (PITR, pooler limits), Render always-on instances, Firebase Hosting headers, SMS regulatory requirements, and FCM behaviour for parent devices.
9. **Involve counsel before the pilot** for child-data processing, consent, retention and cross-border considerations.
10. **Measure before scaling**: run the Section 22 load tests at each growth step and let measurements, not fashion, justify any move to replicas, partitions or services.

31.1 Open decisions requiring owner input

Decision	Options	Needed by
Billing provider and plan prices	Razorpay / Stripe / other (Validate)	Phase 6
Email and SMS providers	Comparison on deliverability, DLT support, cost	Phase 5
Card technology for production	UID cards vs authenticated cards	Before hardware order
Late/half-day policy details per school	Rule versions to configure	Phase 3 pilot workshops
Data retention periods	Contractual and legal review	Before pilot
Region selection	Closest Supabase/Render regions that satisfy legal needs	Phase 1
Student login policy by age	Enabled/disabled per school	Phase 2

32. Glossary

Term	Definition
ADR	Architecture Decision Record: a short document recording a significant technical decision and its rationale.
Anti-rollback	Firmware mechanism that refuses installing images with a lower security version than the current one.
Attendance status	The school's recorded status for a student on a day (PENDING, PRESENT, LATE, HALF_DAY, ABSENT, EXCUSED).
Backoff with jitter	Retry delays that grow exponentially with random variation to avoid synchronised retry storms.
Bidirectional reader	A reader used for both entry and exit; interprets taps using state and time windows.
Boot ID	Random identifier created at each device boot; groups events and time references of one power-on session.
Composite foreign key	A foreign key spanning several columns, here `(school_id, id)`, which forces child and parent into the same school.
Dedupe key	Deterministic string making a notification unique for one meaningful state change.
Device action	Server instruction in each result: DELETE (device may purge) or RETAIN_AND_RETRY.
Effective timestamp	The trusted time of a tap used by the rule engine (device time, reconstructed time or, for informational purposes only, receive time).
Idempotency	Property that repeating a request has the same effect as doing it once.
Modular monolith	One deployable application organised into well-separated modules.
Movement status	Where the student is: NOT_ARRIVED, ARRIVED, LEFT.
NFC	Near-Field Communication, short-range wireless used by cards and readers.
Outbox pattern	Writing messages to a database table in the same transaction as the cause, delivered later by a worker.
PITR	Point-in-time recovery: restoring a database to a chosen moment using continuous log archiving.
RLS	Row-Level Security: PostgreSQL policies that restrict which rows a role can see or change.
RPO / RTO	Recovery Point Objective (acceptable data loss) / Recovery Time Objective (acceptable downtime).
RTC	Real-time clock hardware (here DS3231) that keeps time without power from the main supply.
SKIP LOCKED	PostgreSQL clause letting concurrent workers skip rows already locked by another worker.
Super Admin	Platform operator role of the SaaS company.
Tenant	One partner school and its data within the shared platform.
UID	Unique identifier of an NFC card; not secret and clonable on many card types.
WCAG	Web Content Accessibility Guidelines.

Appendix A — Requirement Traceability Matrix

Every requirement is mapped to the implementing module(s) and the verifying test(s). "Design review" and similar entries are non-automated verification. Test case definitions are in Section 28.

Requirement ID	Requirement (abridged)	Implementing module(s)	Verified by	Status
ARCH-REQ-001	The platform SHALL be delivered as one modular-monolith API plus one separate background worker. Microservices, Kubernetes and Kafka SHALL NOT be…	apps/api, apps/worker	Design review	**Confirmed**
ARCH-REQ-002	PostgreSQL (Supabase) SHALL be the single authoritative store for school, student, attendance and academic records. No second source of truth…	db/migrations, apps/api	TEST-REQ-045	**Confirmed**
ARCH-REQ-003	Frontend host, API host, database host, file storage and background worker SHALL be distinct deployables with independent release and scaling.	infra/	Design review	**Confirmed**
ARCH-REQ-004	Browser bundles SHALL contain only public configuration. Database service-role credentials and device secrets SHALL never appear in browser code.	apps/web, CI	TEST-REQ-040	**Confirmed**
ARCH-REQ-005	All business rules (attendance, school isolation, marks, notifications, device registration, subscription enforcement) SHALL execute on the…	apps/api/modules/*	TEST-REQ-001, TEST-REQ-011	**Confirmed**
ARCH-REQ-006	A single machine-readable API contract (OpenAPI / JSON Schema generated from Zod schemas) SHALL be the shared source for web client types, API…	packages/contracts	CI contract check	**Proposed**
ARCH-REQ-007	Staging and production SHALL use separate Supabase projects, separate secrets and separate device credential sets.	infra/	TEST-REQ-045	**Proposed**
ARCH-REQ-008	Adding a new partner school SHALL require configuration/data only (no code change and no schema change).	apps/api/modules/schools	TEST-REQ-001	**Confirmed**
ARCH-REQ-009	The public REST API SHALL be versioned under /api/v1 and breaking changes SHALL ship under a new version prefix.	apps/api/app.ts	Contract tests	**Proposed**
ARCH-REQ-010	The worker SHALL be restart-safe: a worker crash or deploy SHALL NOT lose or duplicate notification jobs.	apps/worker	TEST-REQ-021	**Confirmed**
TEN-REQ-001	Every school-owned table SHALL carry a NOT NULL school_id referencing schools(id).	db/migrations	TEST-REQ-045	**Confirmed**
TEN-REQ-002	References between school-owned rows SHALL use composite foreign keys (school_id, id) so a row cannot reference a parent that belongs to another…	db/migrations	TEST-REQ-002	**Confirmed**
TEN-REQ-003	The API SHALL derive school context from the authenticated membership (users) or the authenticated device registration (devices). A school_id…	middleware/tenant, middleware/deviceAuth	TEST-REQ-001, TEST-REQ-028	**Confirmed**
TEN-REQ-004	PostgreSQL Row-Level Security SHALL be enabled and FORCED on every school-owned table, with policies keyed on a transaction-local tenant setting.	db/migrations/rls	TEST-REQ-003	**Confirmed**
TEN-REQ-005	Automated cross-school access tests SHALL run in CI against every API route and every school-owned table; a failing test SHALL block merge.	tests/security	TEST-REQ-001, TEST-REQ-002, TEST-REQ-003	**Confirmed**
TEN-REQ-006	Requests for another school's resource SHALL return 404 (not 403) to avoid confirming that the resource exists.	middleware/errors	TEST-REQ-001	**Proposed**
TEN-REQ-007	Platform staff access to school-owned student data SHALL require a time-boxed, reason-bearing support_access_grant and SHALL be written to audit_logs.	modules/schools, modules/audit	TEST-REQ-008	**Confirmed**
FE-REQ-001	The web app SHALL provide role-specific layouts (platform, admin, principal, teacher, parent, student) that share common components.	apps/web/layouts	E2E role smoke tests	**Confirmed**
FE-REQ-002	Protected routes and permission-aware UI SHALL be treated as user-experience aids only; every action SHALL be re-authorised by the API.	apps/web/routes	TEST-REQ-001, TEST-REQ-004	**Confirmed**
FE-REQ-003	The UI SHALL target WCAG 2.1 AA conformance for core workflows (keyboard navigation, contrast, labels, focus management).	apps/web/components	TEST-REQ-041	**Validate**

Requirement ID	Requirement (abridged)	Implementing module(s)	Verified by	Status
FE-REQ-004	Expired or revoked sessions SHALL be handled without data loss: silent refresh first, then redirect to sign-in preserving the user's return path.	apps/web/services/apiClient	TEST-REQ-036	**Proposed**
FE-REQ-005	Layouts SHALL be responsive for desktop dashboards, tablets and mobile phones; the parent portal is designed mobile-first.	apps/web/layouts	E2E viewport tests	**Confirmed**
FE-REQ-006	Only variables prefixed VITE_ and containing public values (API base URL, Supabase URL, Supabase anon key, FCM web config) SHALL be bundled.	apps/web, CI	TEST-REQ-040	**Confirmed**
API-REQ-001	All client and device traffic SHALL use HTTPS (TLS 1.2 or later); plain HTTP SHALL be redirected or refused.	infra/, apps/api	Deployment check	**Confirmed**
API-REQ-002	Every endpoint SHALL validate request bodies, params and query strings with Zod schemas. Client/device-supplied school identifiers SHALL be…	modules/*/schemas	TEST-REQ-028	**Proposed**
API-REQ-003	Errors SHALL use one envelope containing a stable machine code, a human message and a request_id; internal details SHALL NOT leak.	middleware/errors	Unit tests	**Proposed**
API-REQ-004	Rate limits SHALL apply per IP, per authenticated user and per device, with tighter limits on authentication endpoints.	middleware/rateLimit	TEST-REQ-035	**Proposed**
API-REQ-005	Payload limits SHALL be enforced: 1 MB default JSON body, 256 KB and 100 events for device batches.	middleware/bodyLimit	TEST-REQ-035	**Proposed**
API-REQ-006	List endpoints SHALL paginate (default 25, max 100); very large or append-only lists SHALL use cursor pagination.	utils/pagination	Unit tests	**Proposed**
DB-REQ-001	Primary keys SHALL be UUIDs generated by gen_random_uuid(); instants SHALL be timestamptz in UTC; enumerations SHALL be text with CHECK constraints.	db/migrations	TEST-REQ-045	**Proposed**
DB-REQ-002	Foreign keys SHALL be declared for every relationship; ON DELETE defaults to RESTRICT; academic and attendance data SHALL be retired by status,…	db/migrations	TEST-REQ-045	**Proposed**
DB-REQ-003	daily_attendance SHALL carry UNIQUE (school_id, student_id, attendance_date).	db/migrations/attendance	TEST-REQ-011	**Confirmed**
DB-REQ-004	attendance_events SHALL carry UNIQUE (device_id, device_event_id) as the idempotency key.	db/migrations/attendance	TEST-REQ-012	**Confirmed**
DB-REQ-005	Every index SHALL be justified by a documented query; new indexes SHALL be verified with EXPLAIN (ANALYZE) on realistic data volumes.	db/migrations	Review checklist	**Proposed**
DB-REQ-006	Schema changes SHALL be forward-only, version-controlled SQL migrations applied by CI, using expand/contract for breaking changes.	db/migrations, CI	TEST-REQ-045	**Proposed**
DB-REQ-007	The API SHALL connect through the pooler using a least-privilege role (not superuser, not BYPASSRLS).	config/database	TEST-REQ-003	**Confirmed**
DB-REQ-008	Automated backups and, where the plan supports it, point-in-time recovery SHALL be enabled and verified by restore drills.	infra/, runbooks	TEST-REQ-026	**Validate**
DB-REQ-009	audit_logs SHALL be append-only: application roles SHALL hold INSERT and SELECT only.	db/migrations/audit	Privilege test	**Confirmed**
DB-REQ-010	Child tables SHALL not duplicate student or school attributes; snapshots are allowed only where historical accuracy is required (e.g.…	db/migrations	Schema review	**Proposed**
DB-REQ-011	attendance_events rows SHALL be immutable except for explicitly listed processing-state columns; DELETE SHALL be revoked.	db/migrations/attendance	Privilege test	**Proposed**
DB-REQ-012	A student SHALL have at most one ACTIVE NFC card, and a card UID SHALL belong to at most one non-retired card record per school (partial unique…	db/migrations/cards	TEST-REQ-029	**Confirmed**
ATT-REQ-001	Attendance status and movement status SHALL be stored in separate columns. A departure event SHALL NOT erase attendance status or the recorded…	modules/attendance/engine	TEST-REQ-030	**Confirmed**
ATT-REQ-002	Attendance windows SHALL be configurable per school and per class, versioned, timezone-aware, start-inclusive and end-exclusive.	modules/attendance/rules	TEST-REQ-032, TEST-REQ-042	**Confirmed**

Requirement ID	Requirement (abridged)	Implementing module(s)	Verified by	Status
ATT-REQ-003	Rules SHALL be evaluated against the event's trusted original timestamp (effective_ts), never its upload time.	modules/attendance/engine	TEST-REQ-017, TEST-REQ-030	**Confirmed**
ATT-REQ-004	Events with untrustworthy timestamps SHALL be stored, flagged and quarantined for review rather than applied automatically.	modules/attendance/timeTrust	TEST-REQ-017	**Confirmed**
ATT-REQ-005	Processing SHALL be idempotent per (device, event_id); a retried event SHALL return the stored result without reprocessing.	modules/attendance/ingest	TEST-REQ-012, TEST-REQ-013	**Confirmed**
ATT-REQ-006	Simultaneous taps from several devices SHALL be stored as separate raw events yet resolve to exactly one daily record, using a row lock inside one…	modules/attendance/ingest	TEST-REQ-011	**Confirmed**
ATT-REQ-007	A tap SHALL NOT be blindly treated as departure; location type, current movement state and time window SHALL decide the outcome.	modules/attendance/engine	TEST-REQ-010, TEST-REQ-030	**Confirmed**
ATT-REQ-008	Parent notifications SHALL be created only on meaningful state changes and SHALL carry a deterministic dedupe key.	modules/attendance, modules/notifications	TEST-REQ-009, TEST-REQ-022	**Confirmed**
ATT-REQ-009	daily_attendance SHALL be a deterministic projection of the ordered attendance events of that student and date; an out-of-order event SHALL…	modules/attendance/engine	TEST-REQ-030	**Proposed**
ATT-REQ-010	An absent-finalisation job SHALL run after the absent cutoff plus a synchronisation grace period, defer for schools with offline devices holding…	jobs/absentFinalise	TEST-REQ-034	**Proposed**
ATT-REQ-011	Taps on holidays or non-school days SHALL be recorded but not applied to daily attendance.	modules/attendance/calendar	TEST-REQ-033	**Proposed**
ATT-REQ-012	Manual corrections SHALL use attendance_corrections with requester, approver and reason; raw events SHALL never be edited.	modules/attendance/corrections	TEST-REQ-043	**Confirmed**
ATT-REQ-013	Classroom-reader events SHALL be stored as presence evidence and SHALL NOT change gate movement status unless the school enables…	modules/attendance/engine	TEST-REQ-010	**Proposed**
ATT-REQ-014	An event SHALL be acknowledged to the device only after its database transaction has committed.	modules/attendance/ingest	TEST-REQ-013, TEST-REQ-019	**Confirmed**
ATT-REQ-015	Each daily_attendance row SHALL record the rule_version_id used, so historical results remain explainable after rule changes.	db/migrations/attendance	TEST-REQ-042	**Proposed**
ATT-REQ-016	Re-entry after departure (e.g. returning for an appointment) SHALL follow a configurable policy and SHALL be audited.	modules/attendance/engine	TEST-REQ-010	**Proposed**
DEV-REQ-001	Each device SHALL have a unique device_code, unique credentials, school association, location, firmware version, online status, last heartbeat,…	modules/devices, firmware/	TEST-REQ-031	**Confirmed**
DEV-REQ-002	A device's school SHALL be derived from its authenticated registration; a device SHALL never select an arbitrary school or student.	middleware/deviceAuth	TEST-REQ-028	**Confirmed**
DEV-REQ-003	Device secrets SHALL be stored only as hashes server-side, support rotation with overlap, and revocation SHALL take effect within a target of 60…	modules/devices/credentials	TEST-REQ-025	**Proposed**
DEV-REQ-004	The device SHALL persist each tap durably before giving "recorded" feedback and SHALL NOT delete any event before explicit server acknowledgement.	firmware/queue	TEST-REQ-015, TEST-REQ-018	**Confirmed**
DEV-REQ-005	Each event SHALL include event_id, device_id, card_uid, event_timestamp, time_source, sequence_number, boot_id and firmware_version.	packages/contracts, firmware/event	TEST-REQ-012	**Confirmed**
DEV-REQ-006	The device SHALL send periodic heartbeats containing health, queue depth, storage, RTC and connectivity data.	firmware/health, modules/devices	TEST-REQ-031	**Confirmed**
DEV-REQ-007	Batch uploads SHALL contain at most 100 events and 256 KB; the response SHALL contain a per-event result and an explicit device_action (DELETE or…	modules/attendance/ingest	TEST-REQ-035	**Proposed**
DEV-REQ-008	Uploads SHALL retry with exponential backoff plus jitter and SHALL resume automatically after reboot or network loss.	firmware/sync	TEST-REQ-014	**Confirmed**

Requirement ID	Requirement (abridged)	Implementing module(s)	Verified by	Status
DEV-REQ-009	Firmware updates SHALL be cryptographically signed, version-checked against rollback and installed with automatic rollback on failed self-test.	firmware/ota, modules/devices/ota	TEST-REQ-016, TEST-REQ-027	**Confirmed**
DEV-REQ-010	The queue SHALL use per-record CRC, quarantine corrupt records, raise alerts at defined fill levels and never overwrite unacknowledged events.	firmware/queue	TEST-REQ-015	**Confirmed**
DEV-REQ-011	Devices SHALL NOT store student personal data or backend privileged secrets; the only secret on a device is its own credential.	firmware/	Security review	**Confirmed**
DEV-REQ-012	The server SHALL detect gaps in a device's sequence numbers and raise an alert for review.	modules/devices/sync	TEST-REQ-014	**Proposed**
DEV-REQ-013	A device with no heartbeat for the configured threshold (default 3 minutes) SHALL be marked OFFLINE and alerted to the school admin.	jobs/deviceWatchdog	TEST-REQ-031	**Proposed**
DEV-REQ-014	Device configuration SHALL be versioned, fetched over authenticated TLS and SHALL NOT contain secrets of other systems.	modules/devices/config	Unit tests	**Proposed**
DEV-REQ-015	A school SHALL be able to operate 1..N devices, limited only by its subscription plan.	modules/devices, modules/subscriptions	TEST-REQ-011	**Confirmed**
NOT-REQ-001	The notification outbox row SHALL be inserted in the same database transaction as the attendance change that caused it.	modules/attendance/ingest	TEST-REQ-021	**Confirmed**
NOT-REQ-002	The worker SHALL claim jobs with FOR UPDATE SKIP LOCKED; delivery is at-least-once and duplicates are suppressed by dedupe keys.	apps/worker/outbox	TEST-REQ-022	**Confirmed**
NOT-REQ-003	Failed deliveries SHALL retry with backoff and, after the maximum attempts, move to DEAD with an alert.	apps/worker/outbox	TEST-REQ-021	**Confirmed**
NOT-REQ-004	Parent preferences SHALL be honoured per channel and every linked parent SHALL be notified independently.	modules/notifications	TEST-REQ-044	**Confirmed**
NOT-REQ-005	Attendance SHALL remain saved when any notification provider is unavailable.	modules/attendance, apps/worker	TEST-REQ-021	**Confirmed**
NOT-REQ-006	Notifications older than a configurable staleness limit (default 24 h) SHALL be suppressed and recorded as EXPIRED_STALE.	apps/worker/outbox	TEST-REQ-014	**Proposed**
NOT-REQ-007	Every delivery attempt and outcome SHALL be retained for audit for a configurable period.	notification_deliveries	Review	**Proposed**
SUB-REQ-001	Subscription plans SHALL define feature flags and numeric limits enforced server-side.	modules/subscriptions	TEST-REQ-023	**Confirmed**
SUB-REQ-002	Payment failure SHALL follow a documented grace and suspension policy; expiry SHALL never delete school data.	modules/subscriptions, jobs/subscriptionState	TEST-REQ-023, TEST-REQ-024	**Confirmed**
SUB-REQ-003	Device event ingestion SHALL continue in every subscription state for non-revoked devices so that recorded taps are never lost.	modules/attendance/ingest	TEST-REQ-023	**Confirmed**
SUB-REQ-004	Unavailable features SHALL return a stable error code (SUBSCRIPTION_FEATURE_UNAVAILABLE) and the UI SHALL show an upgrade message.	middleware/entitlements	TEST-REQ-023	**Proposed**
SUB-REQ-005	All subscription state changes SHALL be written to audit_logs and subscription_events.	modules/subscriptions	Review	**Proposed**
SEC-REQ-001	Passwords SHALL be handled by Supabase Auth; a minimum length and leaked-password checking SHALL be configured where the plan supports it.	Supabase Auth config	Config review	**Validate**
SEC-REQ-002	Multi-factor authentication SHALL be available and SHOULD be enforced for Super Admin, School Admin and Principal accounts.	Supabase Auth, apps/web	E2E	**Proposed**
SEC-REQ-003	Every API route SHALL enforce role-based authorisation server-side through shared middleware; routes without an explicit policy SHALL fail closed.	middleware/authorize	TEST-REQ-001 to TEST-REQ-007	**Confirmed**
SEC-REQ-004	Parent and student access to student data SHALL be validated through student_parents (or the student's own link) on every request.	modules/parents	TEST-REQ-004, TEST-REQ-005, TEST-REQ-007	**Confirmed**
SEC-REQ-005	Teacher access to classes, marks and students SHALL be validated through teacher_class_assignments.	modules/teachers	TEST-REQ-006	**Confirmed**

Requirement ID	Requirement (abridged)	Implementing module(s)	Verified by	Status
SEC-REQ-006	Secrets SHALL live only in host secret stores; no secret SHALL be committed to source control or shipped in a browser bundle.	CI, infra/	TEST-REQ-040	**Confirmed**
SEC-REQ-007	Sensitive actions (logins of privileged users, role changes, corrections, exports, device lifecycle, support access) SHALL be audit-logged.	modules/audit	TEST-REQ-008, TEST-REQ-043	**Confirmed**
SEC-REQ-008	Report and document downloads SHALL use short-lived signed URLs issued only after an authorisation check.	modules/reports, Storage	TEST-REQ-039	**Confirmed**
SEC-REQ-009	Backups SHALL be encrypted at rest and restore access SHALL be limited to named platform engineers.	infra/	TEST-REQ-026	**Validate**
SEC-REQ-010	The risks of UID-only NFC cards (cloning, relay) SHALL be documented; lost cards SHALL be blockable instantly; production SHALL evaluate…	modules/cards	TEST-REQ-029	**Proposed**
SEC-REQ-011	Data retention and deletion periods SHALL be defined per contract and applicable law before pilot go-live.	Policy	Legal review	**Validate**
SEC-REQ-012	Children's data SHALL be minimised and processed under a lawful basis, including parental consent where required; legal review of applicable…	Policy	Legal review	**Validate**
SEC-REQ-013	Dependency, container and secret scanning SHALL run in CI and an external penetration test SHALL precede pilot go-live.	CI	Pen-test report	**Proposed**
SEC-REQ-014	Security headers (CSP, HSTS, X-Content-Type-Options, frame-ancestors) SHALL be set on the frontend host and API.	firebase.json, middleware/helmet	Header scan	**Proposed**
SEC-REQ-015	Production hardware SHALL enable flash encryption and secure boot so a captured device cannot be trivially cloned or re-flashed.	firmware/	TEST-REQ-027	**Proposed**
SEC-REQ-016	Authentication endpoints SHALL be protected against brute force by rate limiting and lockout/back-off.	Supabase Auth, middleware/rateLimit	TEST-REQ-035	**Proposed**
PERF-REQ-001	TARGET: p95 server-side latency of POST /device/events/batch ≤ 500 ms for batches ≤ 20 events (p99 ≤ 1.5 s) at morning-peak load.	modules/attendance/ingest	TEST-REQ-038	**Validate**
PERF-REQ-002	TARGET: ≥ 99.9 % of valid events are committed within 5 s of receipt during the morning peak.	modules/attendance/ingest	TEST-REQ-038	**Validate**
PERF-REQ-003	TARGET: p95 ≤ 400 ms for paginated dashboard read endpoints with 100 concurrent users per school.	apps/api	TEST-REQ-038	**Validate**
PERF-REQ-004	TARGET: p95 time from attendance commit to hand-off to the push provider ≤ 30 s.	apps/worker	TEST-REQ-038	**Validate**
PERF-REQ-005	TARGET: a device backlog of 5,000 events drains in ≤ 10 minutes without degrading other devices.	firmware/sync, ingest	TEST-REQ-014	**Validate**
PERF-REQ-006	TARGET: a term report run for 1,000 students completes asynchronously within 2 minutes (p95).	modules/reports	TEST-REQ-038	**Validate**
PERF-REQ-007	TARGET: a 500-row bulk marks upload validates and commits in ≤ 5 s.	modules/academics	TEST-REQ-037	**Validate**
PERF-REQ-008	TARGET: peak database connection use stays ≤ 70 % of the configured pool/pooler limit.	config/database	TEST-REQ-038	**Validate**
OPS-REQ-001	CI SHALL run lint, type-check, unit, integration (including cross-school), build and security scans on every pull request.	.github/workflows	CI	**Proposed**
OPS-REQ-002	Deployments SHALL go to staging first and be promoted to production with manual approval; rollback SHALL be possible in one step.	.github/workflows, infra/	Release drill	**Proposed**
OPS-REQ-003	Initial targets: RPO ≤ 5 minutes (needs PITR) and RTO ≤ 4 hours for the database; both SHALL be validated by restore drills.	infra/, runbooks	TEST-REQ-026	**Validate**
OPS-REQ-004	Alerts SHALL exist for database capacity, device heartbeat loss, notification backlog, failed event synchronisation and API error rate.	infra/monitoring	Alert fire-drill	**Confirmed**
OPS-REQ-005	Restore drills SHALL be performed at least quarterly and before pilot go-live.	runbooks	TEST-REQ-026	**Proposed**
OPS-REQ-006	Runbooks SHALL exist for database downtime, API downtime, device failure and notification failure.	docs/runbooks	Tabletop exercise	**Proposed**
OPS-REQ-007	API and worker SHALL expose liveness and readiness endpoints used by the host and uptime monitor.	apps/api, apps/worker	Integration	**Proposed**

Reverse view: test case to requirements

Test	Scenario	Requirements verified
TEST-REQ-001	School A user reads School B records	TEN-REQ-003, TEN-REQ-005, SEC-REQ-003
TEST-REQ-002	School A writes reference School B parent rows	TEN-REQ-002
TEST-REQ-003	RLS is the last line of defence	TEN-REQ-004, DB-REQ-007
TEST-REQ-004	Parent reads an unrelated student	SEC-REQ-004, FE-REQ-002
TEST-REQ-005	Parent-child link removed	SEC-REQ-004
TEST-REQ-006	Teacher accesses an unassigned class	SEC-REQ-005
TEST-REQ-007	Student reads another student	SEC-REQ-004
TEST-REQ-008	Super admin access to student data	TEN-REQ-007, SEC-REQ-007
TEST-REQ-009	Repeated tap at the same device	ATT-REQ-008
TEST-REQ-010	Repeated tap after debounce and location rules	ATT-REQ-007, ATT-REQ-013, ATT-REQ-016
TEST-REQ-011	Simultaneous taps from two devices	ATT-REQ-006, DB-REQ-003, DEV-REQ-015
TEST-REQ-012	Same event uploaded repeatedly	ATT-REQ-005, DB-REQ-004
TEST-REQ-013	Network drops after commit, before response	ATT-REQ-005, ATT-REQ-014
TEST-REQ-014	Long outage and backlog synchronisation	DEV-REQ-008, DEV-REQ-012, NOT-REQ-006, PERF-REQ-005
TEST-REQ-015	Power loss during queue write	DEV-REQ-004, DEV-REQ-010
TEST-REQ-016	Power loss during OTA	DEV-REQ-009
TEST-REQ-017	Incorrect RTC time	ATT-REQ-003, ATT-REQ-004
TEST-REQ-018	Device restart with pending events	DEV-REQ-004
TEST-REQ-019	Database failure during processing	ATT-REQ-014
TEST-REQ-020	Database failover / pooler restart	OPS-REQ-007
TEST-REQ-021	Notification provider failure	NOT-REQ-001, NOT-REQ-003, NOT-REQ-005, ARCH-REQ-010
TEST-REQ-022	Duplicate notification prevention	ATT-REQ-008, NOT-REQ-002
TEST-REQ-023	Expired subscription	SUB-REQ-001, SUB-REQ-003, SUB-REQ-004
TEST-REQ-024	Grace-period state machine	SUB-REQ-002, SUB-REQ-005
TEST-REQ-025	Device revocation	DEV-REQ-003
TEST-REQ-026	Backup restoration	DB-REQ-008, OPS-REQ-003, OPS-REQ-005, SEC-REQ-009
TEST-REQ-027	Unauthorised firmware update	DEV-REQ-009, SEC-REQ-015
TEST-REQ-028	Device spoofs another school	TEN-REQ-003, DEV-REQ-002, API-REQ-002
TEST-REQ-029	Lost or cloned card	DB-REQ-012, SEC-REQ-010
TEST-REQ-030	Out-of-order events across devices	ATT-REQ-001, ATT-REQ-003, ATT-REQ-007, ATT-REQ-009
TEST-REQ-031	One device offline, others online	DEV-REQ-001, DEV-REQ-006, DEV-REQ-013
TEST-REQ-032	Window boundaries	ATT-REQ-002
TEST-REQ-033	Holiday tap	ATT-REQ-011
TEST-REQ-034	Absent finalisation and retraction	ATT-REQ-010
TEST-REQ-035	Rate and payload limits	API-REQ-004, API-REQ-005, DEV-REQ-007, SEC-REQ-016
TEST-REQ-036	Session expiry in the browser	FE-REQ-004
TEST-REQ-037	Bulk marks upload	PERF-REQ-007
TEST-REQ-038	Morning-peak load test	PERF-REQ-001 to PERF-REQ-008
TEST-REQ-039	Report download security	SEC-REQ-008
TEST-REQ-040	Secret leakage scan	ARCH-REQ-004, FE-REQ-006, SEC-REQ-006
TEST-REQ-041	Accessibility	FE-REQ-003

Test	Scenario	Requirements verified
TEST-REQ-042	Exam-day and class-specific rules	ATT-REQ-002, ATT-REQ-015
TEST-REQ-043	Manual correction workflow	ATT-REQ-012, SEC-REQ-007
TEST-REQ-044	Multiple parents, partial failure	NOT-REQ-004
TEST-REQ-045	Migrations and policy existence	ARCH-REQ-002, ARCH-REQ-007, TEN-REQ-001, DB-REQ-001, DB-REQ-002, DB-REQ-006

Appendix B — Editable Diagram Sources

All diagrams are kept as text sources so they can be edited and re-rendered (Mermaid CLI `mmdc`, Graphviz `dot`). Store these files under `docs/architecture/diagrams/`.

Figure — Complete system architecture (Graphviz DOT)

```
digraph G {
  rankdir=TB; nodesep=0.35; ranksep=0.75; bgcolor="white"; newrank=true; compound=true;
  node [shape=box, style="rounded,filled", fillcolor="#EAF1F8", color="#2E75B6", fontname="Helvetica",
fontsize=13, margin="0.16,0.11"];
  edge [fontname="Helvetica", fontsize=11, color="#44546A", penwidth=1.4];
  subgraph cluster_users { label="USERS"; labeljust=l; style="rounded,filled"; fillcolor="#F6F8FA";
color="#9FB3C8"; fontname="Helvetica-Bold"; fontsize=12;
    WEB [label="Web app (React SPA)\nAdmin · Principal · Teacher\nParent · Student"];
    PH  [label="Phone / tablet browser\n(responsive, PWA-ready)"]; }
  subgraph cluster_school { label="SCHOOL PREMISES"; labeljust=l; style="rounded,filled";
fillcolor="#F6F8FA"; color="#9FB3C8"; fontname="Helvetica-Bold"; fontsize=12;
    DV1 [label="DEV001\nMain Gate"]; DV2 [label="DEV002\nSecondary Gate"]; DVN [label="DEV00n\nClassroom /
Bus"]; }
  subgraph cluster_fe { label="FRONTEND HOST"; labeljust=l; style="rounded,filled"; fillcolor="#FFF9E6";
color="#C98A00"; fontname="Helvetica-Bold"; fontsize=12;
    FH [fillcolor="#FFF4D6", color="#C98A00", label="Firebase Hosting\nstatic SPA + CDN"]; }
  subgraph cluster_be { label="BACKEND HOST (Render)"; labeljust=l; style="rounded,filled";
fillcolor="#FFF9E6"; color="#C98A00"; fontname="Helvetica-Bold"; fontsize=12;
    API [fillcolor="#FFF4D6", color="#C98A00", label="Node.js / Express API\nmodular monolith · REST
/api/v1\nauth · tenancy · attendance engine"];
    WRK [fillcolor="#FFF4D6", color="#C98A00", label="Background worker\nnotification outbox · scheduled
jobs"]; }
  subgraph cluster_db { label="DATABASE + STORAGE HOST (Supabase)"; labeljust=l; style="rounded,filled";
fillcolor="#EEF8F1"; color="#2E8B57"; fontname="Helvetica-Bold"; fontsize=12;
    AUTH [fillcolor="#E3F4E8", color="#2E8B57", label="Supabase Auth"];
    PG [shape=cylinder, fillcolor="#E3F4E8", color="#2E8B57", label="PostgreSQL\nauthoritative data +
RLS"];
    STO [fillcolor="#E3F4E8", color="#2E8B57", label="Supabase Storage\nreport cards · documents"]; }
  subgraph cluster_ext { label="EXTERNAL SERVICES"; labeljust=l; style="rounded,filled";
fillcolor="#F6F8FA"; color="#9FB3C8"; fontname="Helvetica-Bold"; fontsize=12;
    FCM [label="Firebase Cloud\nMessaging"]; EML [label="Email provider"]; SMS [label="SMS provider"]; OBS
[label="Error + uptime\nmonitoring"]; }

  {rank=same; WEB; PH; DV1; DV2; DVN;}
  {rank=same; FH; API;}
  {rank=same; WRK;}
  {rank=same; AUTH; PG; STO;}
  WEB -> FH [label=" loads SPA"];
  PH -> FH;
  WEB -> API [label=" HTTPS + user JWT"];
  PH -> API;
  DV1 -> API [label=" HTTPS + device credential"]; DV2 -> API; DVN -> API;
  FH -> AUTH [style=invis];
  API -> PG [label=" pooled SQL, tenant context", penwidth=2];
  API -> AUTH [label=" JWT verify"];
  API -> STO [label=" signed URLs"];
  WRK -> PG [label=" SKIP LOCKED claims"];
  API -> WRK [style=invis];
  WRK -> FCM; WRK -> EML; WRK -> SMS;
  API -> OBS [style=dashed]; WRK -> OBS [style=dashed];
  WEB -> AUTH [label=" sign-in / refresh", style=dashed];
}
```

Figure — Frontend and backend modules (Mermaid)

```
flowchart LR
  subgraph FE["FRONTEND (apps/web)"]
    direction TB
    F1["routes + layouts<br/>(role-specific)"]
    F2["features/*<br/>auth, students, attendance,<br/>marks, devices, fees ..."]
    F3["services/apiClient<br/>+ TanStack Query"]
    F4["schemas (Zod) + shared<br/>components (shadcn/ui)"]
    F1 --> F2 --> F3
    F2 --> F4
  end
  subgraph BE["BACKEND (apps/api) - modular monolith"]
    direction TB
    M0["middleware<br/>requestId, logging, rateLimit,<br/>authenticate, tenant,
authorize,<br/>entitlements, validate, errors"]
    subgraph MOD["modules"]
      direction LR
      M1["auth / schools /<br/>subscriptions"]
```

```
      M2["students / parents /<br/>teachers / academics"]
      M3["attendance engine /<br/>devices / cards"]
      M4["notifications / fees /<br/>reports / audit"]
    end
    M5["controller -> service -> repository"]
    M0 --> MOD --> M5
  end
  subgraph DBL["DATA"]
    PG[("PostgreSQL + RLS")]
    ST["Storage"]
  end
  WK["apps/worker<br/>jobs: outbox, absent finalise,<br/>device watchdog, subscription state"]
  F3 -->|"REST /api/v1 + JWT"| M0
  M5 --> PG
  M5 --> ST
  WK --> PG
```

Figure — Database entity relationships (Graphviz DOT)

```
digraph G {
  rankdir=LR; nodesep=0.28; ranksep=0.9; bgcolor="white"; newrank=true;
  node [shape=box, style="rounded,filled", fillcolor="#EAF1F8", color="#2E75B6", fontname="Helvetica",
fontsize=12, margin="0.12,0.07"];
  edge [color="#44546A", arrowhead=crow, arrowtail=tee, dir=both, penwidth=1.2, arrowsize=0.8];
  subgraph cluster_id { label="Platform and identity"; style="rounded,filled"; fillcolor="#F6F8FA";
color="#9FB3C8"; fontname="Helvetica-Bold";
    schools [fillcolor="#1F3A5F", fontcolor=white]; users; school_memberships; roles; subscriptions;
subscription_plans; audit_logs; }
  subgraph cluster_ac { label="Academic"; style="rounded,filled"; fillcolor="#F6F8FA"; color="#9FB3C8";
fontname="Helvetica-Bold";
    academic_sessions; classes; enrollments; students; parents; student_parents; teachers;
teacher_class_assignments; subjects; exams; marks; timetable_entries; assignments; fee_records; }
  subgraph cluster_att { label="NFC, devices and attendance"; style="rounded,filled"; fillcolor="#EEF8F1";
color="#2E8B57"; fontname="Helvetica-Bold";
    device_locations; devices; device_credentials; device_heartbeats; nfc_cards; attendance_events
[fillcolor="#E3F4E8"]; daily_attendance [fillcolor="#E3F4E8"]; attendance_rule_versions;
attendance_corrections; }
  subgraph cluster_not { label="Notifications"; style="rounded,filled"; fillcolor="#FFF9E6";
color="#C98A00"; fontname="Helvetica-Bold";
    notification_outbox [fillcolor="#FFF4D6"]; notification_deliveries [fillcolor="#FFF4D6"]; }
  schools -> school_memberships; users -> school_memberships; roles -> school_memberships;
  schools -> subscriptions; subscription_plans -> subscriptions; schools -> audit_logs;
  schools -> academic_sessions; academic_sessions -> classes; classes -> enrollments; students ->
enrollments;
  students -> student_parents; parents -> student_parents; teachers -> teacher_class_assignments; classes
-> teacher_class_assignments; subjects -> teacher_class_assignments;
  classes -> exams; exams -> marks; students -> marks; classes -> timetable_entries; classes ->
assignments; students -> fee_records;
  schools -> device_locations; device_locations -> devices; devices -> device_credentials; devices ->
device_heartbeats;
  students -> nfc_cards; devices -> attendance_events; nfc_cards -> attendance_events; students ->
attendance_events;
  students -> daily_attendance; attendance_rule_versions -> daily_attendance; daily_attendance ->
attendance_corrections;
  attendance_events -> notification_outbox; notification_outbox -> notification_deliveries; parents ->
notification_deliveries;
}
```

Figure — Attendance entities (Graphviz DOT)

```
digraph G {
  rankdir=TB; nodesep=0.5; ranksep=0.7; bgcolor="white";
  node [shape=plain, fontname="Helvetica", fontsize=11];
  edge [color="#44546A", arrowhead=crow, arrowtail=tee, dir=both, penwidth=1.3, fontname="Helvetica",
fontsize=10];
  DL [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#2E8B57"><TR><TD
BGCOLOR="#2E8B57"><FONT COLOR="white"><B>device_locations</B></FONT></TD></TR><TR><TD ALIGN="LEFT">id PK ·
school_id FK</TD></TR><TR><TD ALIGN="LEFT">name · location_type</TD></TR></TABLE>>];
  DV [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#2E8B57"><TR><TD
BGCOLOR="#2E8B57"><FONT COLOR="white"><B>devices</B></FONT></TD></TR><TR><TD ALIGN="LEFT">id PK · school_id
FK</TD></TR><TR><TD ALIGN="LEFT">device_code UNIQUE</TD></TR><TR><TD ALIGN="LEFT">location_id FK ·
status</TD></TR></TABLE>>];
  NC [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#2E8B57"><TR><TD
BGCOLOR="#2E8B57"><FONT COLOR="white"><B>nfc_cards</B></FONT></TD></TR><TR><TD ALIGN="LEFT">id PK ·
school_id FK</TD></TR><TR><TD ALIGN="LEFT">card_uid · student_id FK</TD></TR><TR><TD
ALIGN="LEFT">status</TD></TR></TABLE>>];
  ST [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#2E75B6"><TR><TD
BGCOLOR="#2E75B6"><FONT COLOR="white"><B>students</B></FONT></TD></TR><TR><TD ALIGN="LEFT">id PK ·
school_id FK</TD></TR><TR><TD ALIGN="LEFT">UNIQUE (school_id, id)</TD></TR></TABLE>>];
```

```
  AE [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#C98A00"><TR><TD
BGCOLOR="#C98A00"><FONT COLOR="white"><B>attendance_events (raw taps)</B></FONT></TD></TR><TR><TD
ALIGN="LEFT">id PK · school_id FK</TD></TR><TR><TD ALIGN="LEFT">device_id FK ·
device_event_id</TD></TR><TR><TD ALIGN="LEFT"><B>UNIQUE (device_id, device_event_id)</B></TD></TR><TR><TD
ALIGN="LEFT">student_id FK · effective_ts · outcome</TD></TR></TABLE>>];
  DA [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#C98A00"><TR><TD
BGCOLOR="#C98A00"><FONT COLOR="white"><B>daily_attendance</B></FONT></TD></TR><TR><TD ALIGN="LEFT">id PK ·
school_id FK · student_id FK</TD></TR><TR><TD ALIGN="LEFT"><B>UNIQUE (school_id, student_id,
attendance_date)</B></TD></TR><TR><TD ALIGN="LEFT">attendance_status · movement_status</TD></TR><TR><TD
ALIGN="LEFT">rule_version_id FK</TD></TR></TABLE>>];
  RV [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#C98A00"><TR><TD
BGCOLOR="#C98A00"><FONT COLOR="white"><B>attendance_rule_versions</B></FONT></TD></TR><TR><TD
ALIGN="LEFT">id PK · school_id · class_id FK</TD></TR><TR><TD ALIGN="LEFT">effective_from · rules
jsonb</TD></TR></TABLE>>];
  CO [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#C98A00"><TR><TD
BGCOLOR="#C98A00"><FONT COLOR="white"><B>attendance_corrections</B></FONT></TD></TR><TR><TD
ALIGN="LEFT">daily_attendance_id FK · status</TD></TR><TR><TD ALIGN="LEFT">requested_by ·
approved_by</TD></TR></TABLE>>];
  OB [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#7A3E9D"><TR><TD
BGCOLOR="#7A3E9D"><FONT COLOR="white"><B>notification_outbox</B></FONT></TD></TR><TR><TD ALIGN="LEFT">id PK
· school_id · kind</TD></TR><TR><TD ALIGN="LEFT"><B>UNIQUE (school_id, dedupe_key)</B></TD></TR></TABLE>>];
  ND [label=<<TABLE BORDER="1" CELLBORDER="0" CELLSPACING="0" CELLPADDING="3" COLOR="#7A3E9D"><TR><TD
BGCOLOR="#7A3E9D"><FONT COLOR="white"><B>notification_deliveries</B></FONT></TD></TR><TR><TD
ALIGN="LEFT">outbox_id FK · parent_id FK</TD></TR><TR><TD ALIGN="LEFT">UNIQUE (outbox, parent,
channel)</TD></TR></TABLE>>];
  DL -> DV [label=" hosts"]; DV -> AE [label=" records"]; NC -> AE [label=" tapped"]; ST -> NC [label="
carries"];
  ST -> DA [label=" has"]; RV -> DA [label=" governs"]; DA -> CO [label=" corrected by"];
  AE -> OB [label=" triggers"]; OB -> ND [label=" fans out"];
  {rank=same; DL; ST;}
  {rank=same; DV; NC;}
}
```

Figure — Attendance transaction sequence (Mermaid)

```
sequenceDiagram
  autonumber
  participant D as Device (DEV001)
  participant A as API (device auth + ingest)
  participant DB as PostgreSQL
  participant O as notification_outbox
  D->>A: POST /device/events/batch (Bearer credential)
  A->>DB: lookup credential hash, device, school, status
  DB-->>A: device row (school_id derived here)
  A->>A: validate payload (Zod), reject foreign school/student fields
  loop each event, ordered by sequence_number
    A->>DB: BEGIN (set tenant context)
    A->>DB: INSERT attendance_events ON CONFLICT (device_id, device_event_id) DO NOTHING
    alt conflict (event already processed)
      DB-->>A: 0 rows
      A->>DB: SELECT stored result
      A->>DB: COMMIT
      Note over A: result status = DUPLICATE_EVENT
    else new event claimed
      A->>DB: SELECT card -> student -> enrollment, device location
      A->>DB: resolve rule_version + time trust
      A->>DB: INSERT daily_attendance ON CONFLICT DO NOTHING
      A->>DB: SELECT ... FOR UPDATE (daily row lock)
      A->>A: apply movement + window rules (pure function)
      A->>DB: UPDATE daily_attendance, UPDATE event result
      A->>O: INSERT outbox (dedupe_key) ON CONFLICT DO NOTHING
      A->>DB: COMMIT
    end
  end
  A-->>D: 200 results[] with status + device_action
  Note over D: delete only events with device_action = DELETE
```

Figure — Offline synchronisation (Mermaid)

```
sequenceDiagram
  autonumber
  participant S as Student
  participant D as Device
  participant Q as Flash queue (CRC records)
  participant A as API
  participant DB as PostgreSQL
  S->>D: tap card
  D->>D: read UID, read RTC, new event_id, next sequence
  D->>Q: append record + fsync
  Q-->>D: persisted
  D-->>S: "recorded" LED + beep
```

```
  Note over D,A: Wi-Fi / internet unavailable
  loop retry with exponential backoff + jitter
    D->>A: heartbeat or batch upload fails
  end
  Note over D,A: connectivity returns
  D->>A: POST /device/heartbeat
  A-->>D: server_time, config_version
  D->>Q: read up to 50 oldest un-acked events
  D->>A: POST /device/events/batch
  A->>DB: process each event in its own transaction
  A-->>D: per-event results + device_action
  alt device_action = DELETE
    D->>Q: advance ack cursor, reclaim fully acked segments
    D->>A: POST /device/sync-ack (cursor, pending count)
  else RETAIN_AND_RETRY (RETRY_LATER)
    D->>Q: keep event, schedule retry
  end
  Note over D,Q: reboot at any point: scan queue, verify CRC, new boot_id, continue from ack cursor
```

Figure — Notification outbox (Mermaid)

```
flowchart TB
  T["Attendance transaction<br/>(daily_attendance + events)"] -->|"same COMMIT"|
OB[("notification_outbox<br/>UNIQUE(school_id, dedupe_key)")]
  OB --> W1["Worker poll<br/>SELECT ... FOR UPDATE SKIP LOCKED"]
  W1 --> FAN["Fan-out: students -> linked parents<br/>apply notification_preferences"]
  FAN --> DL[("notification_deliveries<br/>UNIQUE(outbox_id, parent_id, channel)")]
  DL --> CH{"channel"}
  CH -->|"push"| FCM["Firebase Cloud Messaging"]
  CH -->|"email"| EM["Email provider"]
  CH -->|"sms"| SM["SMS provider"]
  FCM --> RES["Update delivery status<br/>SENT / FAILED"]
  EM --> RES
  SM --> RES
  RES -->|"transient failure"| RT["Retry with backoff<br/>next_attempt_at"]
  RT --> DL
  RES -->|"max attempts"| DEAD["DEAD + alert<br/>attendance unaffected"]
  RES -->|"invalid token"| TOK["Disable push token"]
```

Figure — Device registration and revocation (Mermaid)

```
sequenceDiagram
  autonumber
  participant SA as Super Admin
  participant API as API
  participant DB as PostgreSQL
  participant BN as Bench provisioning
  participant DEV as Device
  participant AD as School Admin
  SA->>API: register hardware for school (serial, MAC, device_code)
  API->>DB: check plan device limit, insert device (PROVISIONED)
  API->>DB: store SHA-256 hash of new 256-bit secret
  API-->>SA: secret + claim code (shown ONCE)
  SA->>BN: flash signed firmware, write secret to encrypted NVS
  BN-->>DEV: provisioned unit shipped to school
  AD->>API: claim device (claim code), set name, location, type
  API->>DB: bind location, status ACTIVE, config v1
  DEV->>API: heartbeat + batches (Bearer secret)
  API-->>DEV: server_time, config_version
  Note over SA,DEV: Revocation (lost, stolen or compromised unit)
  AD->>API: revoke DEV003 (reason required)
  API->>DB: credential REVOKED, device REVOKED, audit log
  DEV->>API: next request (auth cache TTL 30 s)
  API-->>DEV: 401 DEVICE_REVOKED
  Note over DEV: pending events stay on device, re-enrolment needs a new credential
```

Figure — Device lifecycle (Mermaid)

```
stateDiagram-v2
  [*] --> PROVISIONED: registered by platform
  PROVISIONED --> ACTIVE: claimed + location assigned
  ACTIVE --> DISABLED: school admin pauses (e.g. maintenance)
  DISABLED --> ACTIVE: re-enabled
  ACTIVE --> REVOKED: lost / stolen / compromised
  DISABLED --> REVOKED
  REVOKED --> [*]: credential permanently invalid
```

Figure — Deployment architecture (Mermaid)

```
flowchart TB
```

```
  DEV["Developers"] -->|"pull request"| GH["GitHub repository"]
  GH --> CI["GitHub Actions CI<br/>lint, typecheck, unit, integration,<br/>cross-school tests, security
scans, build"]
  CI -->|"merge to main"| STG
  subgraph STG["STAGING"]
    direction TB
    SF["Firebase Hosting (staging site)"]
    SA["Render: API + worker (staging)"]
    SD[("Supabase project: staging")]
    SF --> SA --> SD
  end
  STG -->|"manual approval after smoke + e2e"| PRD
  subgraph PRD["PRODUCTION"]
    direction TB
    PF["Firebase Hosting<br/>frontend + CDN"]
    PA["Render web service<br/>Node API (always-on, 2+ instances recommended)"]
    PW["Render background worker<br/>notifications + scheduled jobs"]
    PD[("Supabase PostgreSQL<br/>pooler + backups / PITR")]
    PS["Supabase Storage<br/>report cards, documents"]
    PF -->|"HTTPS"| PA
    PA --> PD
    PW --> PD
    PA --> PS
  end
  DEVS["School devices (ESP32)"] -->|"HTTPS + device credential"| PA
  PW --> FCM["FCM / email / SMS providers"]
  PA -.-> MON["Sentry + uptime monitor + log drain"]
  PW -.-> MON
  CI -->|"supabase db push (migrations)"| PD
```

Figure — Movement state machine (Mermaid)

```
stateDiagram-v2
  [*] --> NOT_ARRIVED
  NOT_ARRIVED --> ARRIVED: ENTRY or BIDIRECTIONAL tap inside arrival window<br/>(sets
arrival_classification, attendance status)
  ARRIVED --> ARRIVED: repeat tap (debounce) / classroom tap / tap at ENTRY<br/>(stored, no state change)
  ARRIVED --> LEFT: EXIT tap, or BIDIRECTIONAL tap inside a departure window<br/>(sets
departure_classification)
  LEFT --> ARRIVED: ENTRY tap when re-entry allowed<br/>(audited, no new arrival notification)
  LEFT --> LEFT: repeat exit tap (stored, no state change)
  NOT_ARRIVED --> NOT_ARRIVED: EXIT tap without arrival<br/>(stored, flagged
FLAGGED_DEPARTURE_WITHOUT_ARRIVAL)
```

Figure — Multi-school tenant architecture (Graphviz DOT)

```
digraph G {
  rankdir=TB; nodesep=0.45; ranksep=0.6; bgcolor="white"; compound=true;
  node [shape=box, style="rounded,filled", fillcolor="#EAF1F8", color="#2E75B6", fontname="Helvetica",
fontsize=13, margin="0.18,0.12"];
  edge [fontname="Helvetica", fontsize=12, color="#44546A", penwidth=1.6];

  SAAS [label="SaaS PLATFORM\none web app, one API, one background worker,\none PostgreSQL cluster",
fillcolor="#1F3A5F", fontcolor="white", color="#1F3A5F", fontsize=14];
  GATE [shape=box, fillcolor="#FFF4D6", color="#C98A00", label="TENANT GATEKEEPER (applied to every
request)\l1  verified identity: user JWT or device credential\l2  membership / device-registration lookup
(never client school_id)\l3  server-side RBAC + scope checks (class, child, self)\l4  tenant context set
per transaction (app.current_school_id)\l5  PostgreSQL RLS + composite foreign keys (last line of
defence)\l"];

  SA [fillcolor="#E3F4E8", color="#2E8B57", label="SCHOOL A  (school_id = A)\l\n• Students, Parents,
Teachers\l• Classes, Academic results\l• NFC cards, Devices\l• Attendance, Notifications\l• Subscription
configuration\l"];
  SB [fillcolor="#E3F4E8", color="#2E8B57", label="SCHOOL B  (school_id = B)\l\n• Students, Parents,
Teachers\l• Classes, Academic results\l• NFC cards, Devices\l• Attendance, Notifications\l• Subscription
configuration\l"];
  SC [fillcolor="#E3F4E8", color="#2E8B57", label="SCHOOL C  (school_id = C)\l\n• Students, Parents,
Teachers\l• Classes, Academic results\l• NFC cards, Devices\l• Attendance, Notifications\l• Subscription
configuration\l"];

  SAAS -> GATE;
  GATE -> SA [label=" school A only"];
  GATE -> SB [label=" school B only"];
  GATE -> SC [label=" school C only"];
  {rank=same; SA; SB; SC;}
  SA -> SB [style=invis]; SB -> SC [style=invis];
}
```

Figure — Multi-device attendance flow (Graphviz DOT)

```
digraph G {
  rankdir=TB; nodesep=0.35; ranksep=0.6; bgcolor="white"; newrank=true;
  node [shape=box, style="rounded,filled", fillcolor="#EAF1F8", color="#2E75B6", fontname="Helvetica",
fontsize=13, margin="0.16,0.11"];
  edge [fontname="Helvetica", fontsize=12, color="#44546A", penwidth=1.8];

  subgraph cluster_school {
    label="SCHOOL SCH001  -  five devices registered to ONE school, each with a unique device ID,
credential and location";
    labeljust=l; fontname="Helvetica-Bold"; fontsize=13; style="rounded,filled"; fillcolor="#F6F8FA";
color="#9FB3C8"; margin=18;
    D1 [label="DEV001\nMain Gate\nlocation type: ENTRY\nlocal durable queue"];
    D2 [label="DEV002\nSecondary Gate\nBIDIRECTIONAL\nlocal durable queue"];
    D3 [label="DEV003\nClassroom Block A\nCLASSROOM\nlocal durable queue"];
    D4 [label="DEV004\nClassroom Block B\nCLASSROOM\nlocal durable queue"];
    D5 [label="DEV005\nBus Entrance\nENTRY / EXIT\nlocal durable queue"];
    {rank=same; D1; D2; D3; D4; D5;}
  }
  API [fillcolor="#FFF4D6", color="#C98A00", label="SHARED AUTHENTICATED API   (HTTPS / TLS)\nper-device
credential  →  device resolves to its school (no school_id from the device)\nidempotency claim: UNIQUE
(device_id, device_event_id)"];
  ENG [fillcolor="#FFF4D6", color="#C98A00", label="SHARED ATTENDANCE ENGINE\ncard lookup · location rules
· time trust · window rules · movement state"];
  PG [shape=cylinder, fillcolor="#E3F4E8", color="#2E8B57", label="PostgreSQL  -  one logically
authoritative database\nattendance_events (every raw tap)   ·   daily_attendance (one row per student per
day, row-locked)\nnotification_outbox (written in the same transaction)"];
  DASH [label="School dashboard\nPrincipal reports"];
  WRK [label="Notification worker\n(SKIP LOCKED claims)"];
  PAR [fillcolor="#E3F4E8", color="#2E8B57", label="Parent notifications\npush · email · SMS"];

  D1 -> API; D2 -> API; D3 -> API; D4 -> API; D5 -> API [label="  concurrent batches"];
  API -> ENG;
  ENG -> PG [label="  one transaction per event", penwidth=2.4];
  PG -> DASH;
  PG -> WRK [label=" outbox"];
  WRK -> PAR;
  {rank=same; DASH; WRK;}
}
```

Appendix C — Items Requiring Validation

These items are tagged **Validate**. None may be relied upon as a fact or guarantee until confirmed.

ID	What must be validated	How
FE-REQ-003	The UI SHALL target WCAG 2.1 AA conformance for core workflows (keyboard navigation, contrast, labels, focus management).	Vendor documentation / review
DB-REQ-008	Automated backups and, where the plan supports it, point-in-time recovery SHALL be enabled and verified by restore drills.	Vendor documentation + restore drill
SEC-REQ-001	Passwords SHALL be handled by Supabase Auth; a minimum length and leaked-password checking SHALL be configured where the plan supports it.	Vendor documentation / review
SEC-REQ-009	Backups SHALL be encrypted at rest and restore access SHALL be limited to named platform engineers.	Vendor documentation + restore drill
SEC-REQ-011	Data retention and deletion periods SHALL be defined per contract and applicable law before pilot go-live.	Legal review
SEC-REQ-012	Children's data SHALL be minimised and processed under a lawful basis, including parental consent where required; legal review of applicable Indian data-protection…	Legal review
PERF-REQ-001	TARGET: p95 server-side latency of POST /device/events/batch ≤ 500 ms for batches ≤ 20 events (p99 ≤ 1.5 s) at morning-peak load.	Load test (TEST-REQ-038)
PERF-REQ-002	TARGET: ≥ 99.9 % of valid events are committed within 5 s of receipt during the morning peak.	Load test (TEST-REQ-038)
PERF-REQ-003	TARGET: p95 ≤ 400 ms for paginated dashboard read endpoints with 100 concurrent users per school.	Load test (TEST-REQ-038)
PERF-REQ-004	TARGET: p95 time from attendance commit to hand-off to the push provider ≤ 30 s.	Load test (TEST-REQ-038)
PERF-REQ-005	TARGET: a device backlog of 5,000 events drains in ≤ 10 minutes without degrading other devices.	Load test (TEST-REQ-038)
PERF-REQ-006	TARGET: a term report run for 1,000 students completes asynchronously within 2 minutes (p95).	Load test (TEST-REQ-038)
PERF-REQ-007	TARGET: a 500-row bulk marks upload validates and commits in ≤ 5 s.	Load test (TEST-REQ-038)
PERF-REQ-008	TARGET: peak database connection use stays ≤ 70 % of the configured pool/pooler limit.	Load test (TEST-REQ-038)
OPS-REQ-003	Initial targets: RPO ≤ 5 minutes (needs PITR) and RTO ≤ 4 hours for the database; both SHALL be validated by restore drills.	Vendor documentation + restore drill

Additional validation items called out in the text: ESP32 chip revision security features; production card technology and reader support; SMS operator registration (DLT) requirements; hosting plan limits (connections, always-on instances, PITR); billing provider availability and fees; legal retention periods; regional data-location requirements; radio/EMC certification of the production device.